

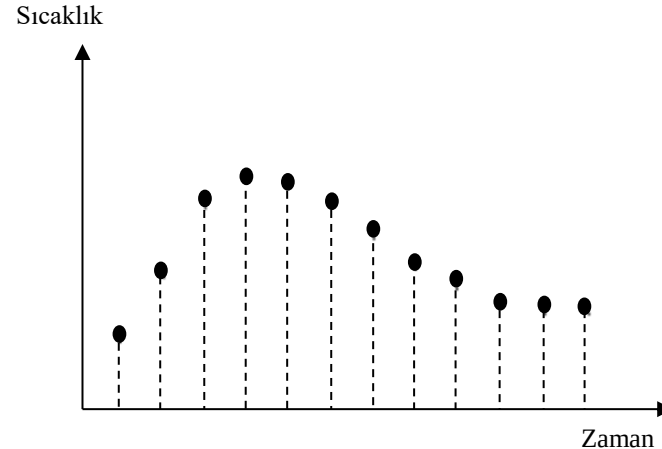
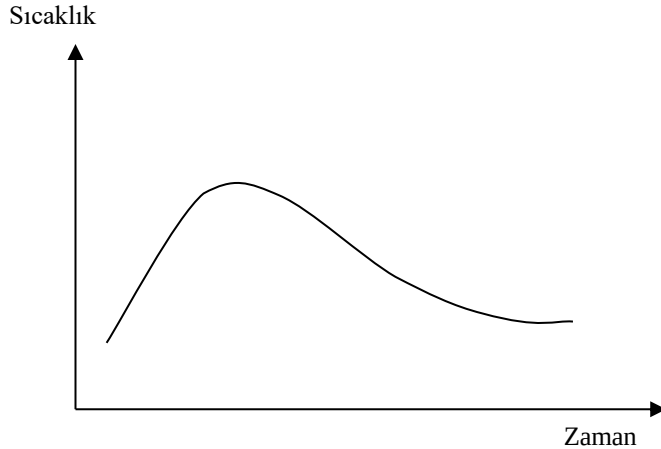
ANALOG-SAYISAL BÜYÜKLÜK VE SAYI SİSTEMLERİ

- **Analog ve Sayısal Büyüklük Kavramı**
- **Sayı Sistemleri**
- **İkilik Sistemde Aritmetik İşlemler**
- **İkili Sayıların 1'e ve 2'ye Tümleyeninin Bulunması**
- **İşaretili Sayılar**

ANALOG-SAYISAL BÜYÜKLÜK VE SAYI SİSTEMLERİ

Analog ve Sayısal Büyüklük Kavramı :

Elektronik devreler analog ve sayısal olmak üzere iki kategoride incelenir. Analog elektronik büyüklüklerin sürekli değerlerini, sayısal elektronik ise kesikli veya ayırık değerlerini kullanır.



Sayısal gösterim şeklinin, analog gösterime göre bilginin işlenebilirliği, yorumlanması, saklanması ve daha güvenilir olarak taşınması bakımından avantajları vardır.

Analog ve Sayısal Büyüklük Kavramı

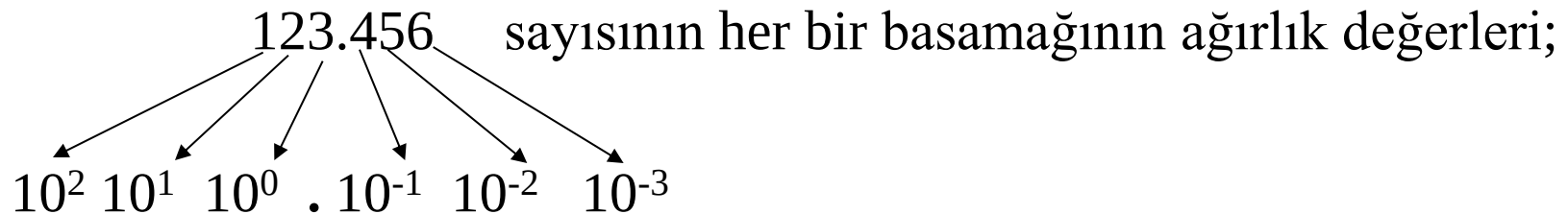
Sayısal bir değer, ‘ON’ ve ‘OFF’ voltaj seviyelerinin (genellikle +5 Volt=ON, 0 Volt=OFF) bir kombinasyonu ile ifade edilebilir.

Örneğin sıcaklık değeri 25 derece ise, bu değer sayısal bir devre ile ‘ON’ ve ‘OFF’ voltaj seviyelerinden oluşan bir string olarak sunulabilir.

Bu voltaj seviyeleri mantıksal olarak ‘1’ ve ‘0’ lar olarak yorumlanır ve en yalın halde ikili sayı sisteminde ‘11001’ olarak ifade edilebilir.

Sayı Sistemleri

Onluk Sistem: Günlük hayatta kullandığımız sayı sistemidir. 0 ile 9 arasındaki rakamlar kullanılarak 10 farklı büyüklük elde edilir.



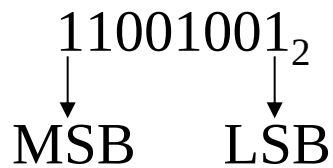
Aynı sayının ağırlıklı toplamı;

$$123.456 = (1 \times 10^2) + (2 \times 10^1) + (3 \times 10^0) + (4 \times 10^{-1}) + (5 \times 10^{-2}) + (6 \times 10^{-3})$$

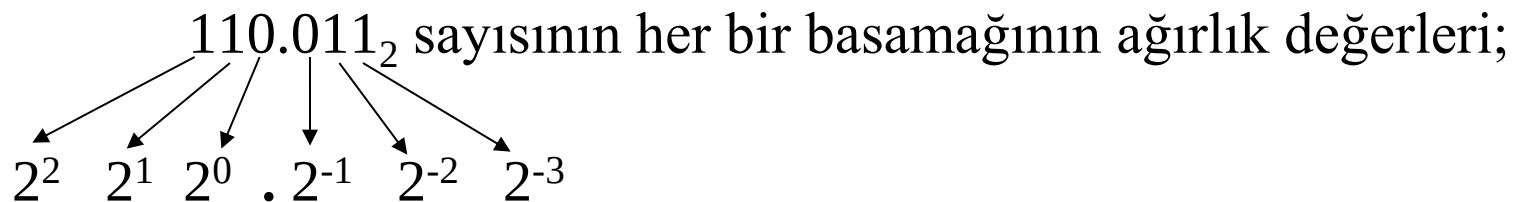
Sayı Sistemleri

İkilik Sistem: Büyüklükleri göstermenin diğer bir yolu da ikilik sayı sistemidir. İkilik sistem, sadece 1 ve 0 değerlerini içerir. İkili sayılarda basamak ağırlığı, 2'nin pozitif katları şeklinde artar. Kesirli sayılarda için ise 2'nin negatif katları şeklinde azalır.

İkili sayılarda en sağdaki bit LSB (Least Significant Bit) olarak, en soldaki bit ise MSB (Most Significant Bit) olarak anılır.



n bit içeren bir ikili sayı ile 0 ile $(2^n - 1)$ arasındaki sayıları gösterilebilir.



Sayı Sistemleri

İkilik sistemden onluk sisteme dönüşüm: Herhangi bir ikili sayının onluk sistemdeki karşılığını bulmak için, sayı içerisindeki 1'lerin basamak değerleri toplanır, 0'lar hesaba katılmaz.

10110_2 ikili sayısının onluk sistemdeki karşılığı; $2^4+2^2+2^1=22$ 'dir.

Yani $(10110)_2 = (22)_{10}$

10.101_2 ikili sayısının onluk sistemdeki karşılığı; $2^1+2^{-1}+2^{-3} = 2.625$ 'dir.

Sayı Sistemleri

Onluk sistemden ikilik sisteme dönüşüm:

İki yöntem kullanılabilir:

- İkili sayının basamak değerlerinin toplamı onluk sayıyı verecek şekilde düzenlemeye gidilir. Özellikle küçük sayıları için pratik bir yöntemdir.

Örneğin onluk sistemdeki bir sayı ikilik sistemde 7 bit ile ifade edilebiliyorsa, bu ikili sayının basamak değerleri (64 32 16 8 4 2 1) dir.

Onluk sistemdeki 98 sayısının ikilik sistemdeki karşılığı 64, 32 ve 2 sayılarının toplamıyla pratik bir şekilde bulunabilir. O halde bu basamak değerlerine karşılık gelen bitler 1 alınarak $(1100010)_2$ sayısı elde edilir.

Sayı Sistemleri

Onluk sistemden ikilik sisteme dönüşüm:

- Onluk sistemde verilen sayıyı sürekli 2'ye bölerek, bölme işlemi sonucunda oluşan kalan kısımlarından ikili sayıyı oluşturmaktır. İlk bölme işlemi sonucunda kalan LSB'yi, son bölme işleminden sonra oluşan kalan ise MSB'yi oluşturur.

Örnek: Onluk sistemdeki 13 sayısının 2'lik sistemdeki karşılığı,

<u>Bölme işlemi</u>	<u>Kalan</u>	
$13/2 = 6$	1 (LSB)	} $(13)_{10} = (1101)_2$
$6/2 = 3$	0	
$3/2 = 1$	1	
$1/2 = 0$	1 (MSB)	

Sayı Sistemleri

Ondalıklı sayıların ikilik sisteme dönüştürülmesi:

Yine iki yöntem kullanılabilir:

- İkili sayıların kesirli kısımlarının basamak değerlerinin toplamı onluk sistemdeki kesirli sayıyı verecek şekilde düzenlemeye gidilir.

İkilik sistemde kesirli sayıların basamak ağırlıkları 0.5, 0.25, 0.125, 0.0625 şeklinde gider.

Örneğin 0.625 sayısı $(0.5+0.125)$ şeklinde ifade edilebileceğinden, bu sayının ikilik sistemdeki karşılığı, 2^{-1} ve 2^{-3} ağırlıklarına sahip bitlerin 1 yapılması suretiyle elde edilir. Yani $0.625_{10} = (0.101)_2$ olur.

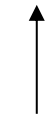
Sayı Sistemleri

Ondalık sayıların ikilik sisteme dönüştürülmesi:

- Tekrarlı çarpma yöntemidir: Onluk sistemdeki kesirli kısım sürekli 2 ile çarpılır ve çarpım işleminin sonucunda oluşan tamsayı kısımlarından (1 veya 0'dır) ikili sayı elde edilir. Bu çarpma işlemine kesirli kısım sıfır oluncaya kadar devam edilir.

Örnek: 0.625 sayısının ikilik sistemdeki karşılığı,

$$\left. \begin{array}{l} 0.625 \times 2 = 1.25 \text{ (MSB)} \\ 0.25 \times 2 = 0.50 \\ 0.5 \times 2 = 1.00 \text{ (LSB)} \end{array} \right\} (0.625)_{10} = (0.101)_2$$



Ondalık kısmı 0 olduğu için işlem tamamlanmıştır.

İkilik Sistemde Aritmetik İşlemler

İki sistemde toplama:

İkili sayıların toplama işleminde 4 temel kural vardır;

$0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$ ve $1 + 1 = 10$ (elde biti oluşur)

İki sayıyı toplarken elde biti 1 ise 4 farklı durum oluşur;

Elde	Elde Toplam
	↓ ↓
$1 + 0 + 0 =$	01
$1 + 0 + 1 =$	10
$1 + 1 + 0 =$	10
$1 + 1 + 1 =$	11

3 bitlik iki sayının toplamı;

$$\begin{array}{r} 101_2 \\ + 001_2 \\ \hline 110_2 \end{array}$$

İkili Sistemde Aritmetik İşlemler

İki sistemde çıkarma:

İkili sayıların çıkarma işleminde yine 4 temel kural vardır;

$0 - 0 = 0$, $1 - 0 = 1$, $1 - 1 = 0$ ve $10 - 1 = 1$ (borç biti sayesinde olur)

Örnek: 3 bitlik iki sayının farkının hesabı;

$$\begin{array}{r} 110 \\ - 001 \\ \hline 101 \end{array}$$

İkili sistemde çarpma:

İkili sayıların çarpma işleminde yine 4 temel kural vardır;

$0 \times 0 = 0$, $0 \times 1 = 0$, $1 \times 0 = 0$ ve $1 \times 1 = 1$

Onluk sistemdeki çarpma işlemi gibidir: kısmi çarpımların oluşturulması ve arda arda gelen kısmi çarpımların bir sola kaydırılıp toplanması esasına dayanır.

İkili Sistemde Aritmetik İşlemler

İkili sistemde çarpma:

Örnek: 2 bitlik iki sayının çarpımının hesabı;

$$\begin{array}{r} 10 \\ \times 11 \\ \hline 10 \\ +10 \\ \hline 110 \end{array} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \begin{array}{l} \text{Ara} \\ \text{çarpımlar} \end{array}$$

İkili sistemde bölme:

Bölme işlemi de onluk sistemdeki bölmeye aynı yapıya sahiptir.

$$\begin{array}{r} \overline{1001} \overline{) 11} \\ - \underline{11} \\ 011 \\ - \underline{11} \\ 000 \end{array}$$

İkili Sayıların 1'e ve 2'ye Tümleyeninin Bulunması

İkili sayıların 1'e ve 2'ye tümleyen gösterimi, özellikle negatif sayıların temsil edilmesi ve negatif sayıları ihtiva eden aritmetik işlemler için gereklidir.

1'e tümleyen:

Pratik olarak ikili sayıların 1'e tümleyenini bulmak için 1 ihtiva eden bitlerin 0 ve 0 ihtiva eden bitlerin de 1 yapılması gerekir.

Örnek:

İkili sayı	110110
	↓ ↓
1'e tümleyeni	001001

Ya da $2^n - N - 1$ formülüyle bulunabilir. Burada n bit sayısını, N ise sayının kendisini göstermektedir.

$$\begin{aligned} 110110_2 \text{ sayısının } 1' \text{e tümleyeni;} & 2^6 - 110110 - 1 = 1000000 - 110110 - 1 \\ & = 1000000 - (110111) \\ & = 001001 \end{aligned}$$

İkili Sayıların 1'e ve 2'ye Tümleyeninin Bulunması

2'ye tümleyen:

İkili sayıların 2'ye tümleyenini bulmak için ise 1'e tümleyenine 1 eklenmesi gerekir.

$$\begin{array}{rcl} \text{İkili sayı} & \longrightarrow & 110110 \\ \text{1'e tümleyeni} & \longrightarrow & 001001 \\ & & + \quad 1 \\ \hline \text{2'ye tümleyeni} & \longrightarrow & 001010 \end{array}$$

Ya da $2^n - N$ formülüyle bulunabilir. Burada n bit sayısını, N ise sayının kendisini göstermektedir.

$$\begin{aligned} 110110_2 \text{ sayısının 2'ye tümleyeni;} \quad 2^6 - 110110 &= 1000000 - 110110 \\ &= 001010 \end{aligned}$$

İşaretli Sayılar

İşaretli sayılar hem işaret hem de büyüklük bilgisi içerirler. İşaret, sayının pozitif veya negatif olduğunu, büyüklük ise sayının değerini gösterir. İkilik sistemde işaretli sayıların gösterimi üç şekilde olabilir; işaret-büyüklük, 1'e tümleyen ve 2'ye tümleyen.

Tüm bu gösterim metotlarının ortak özelliği, MSB bitinin 1 olması durumunda sayının negatif, 0 olması durumunda ise pozitif olmasıdır.

İşaret-büyüklük gösterimi:

İkili sayıların bu metot ile gösterimi kullanıldığında en anlamlı bit işareti, geri kalan bitler de büyüklüğü gösterir. Büyüklük kısmı hem pozitif hem de negatif sayılar için sayının ikilik sistemdeki karşılığıdır.

Örnek: -19 sayısını 8 bit ile ifade etmek istersek;

10010011
↓
İşaret biti Büyüklük bitleri

İşaretili Sayılar

1'e tümleyen gösterimi:

Pozitif sayıların gösterimi, işaret-büyüklik gösterimi ile aynıdır. Negatif sayılar ise ikilik sistemdeki karşılığının 1'e tümleyeni formundadır.

Örnek: 8 bitlik gösterimde -19 sayısı; +19 $(00010011)_2$ sayısının 1'e tümleyeni $(11101100)_2$ ile ifade edilir.

2'ye tümleyen gösterimi:

Pozitif sayılar diğer iki metoda benzer şekilde gösterilir. Negatif sayılar ise ikilik sistemdeki karşılığının 2'ye tümleyeni formundadır.

Örnek: -19 sayısının 2'ye tümleyen gösterimi 11101101_2 'dir.

İşaretli Sayıların Onluk Sistemdeki Karşılığı

İşaret-büyüklik gösteriminde hem pozitif hem de negatif sayıların onluk sistemdeki karşılığı, büyüklik bitlerindeki 1'lerin ağırlıklarının toplamıyla elde edilir. İşaret bitine göre de sonucun pozitif veya negatif olduğuna karar verilir.

Örnek: 10011000_2 işaretli ikili sayısının onluk sistemdeki karşılığı;
 $-(2^3+2^4) = -24$ 'tür.

1'e tümleyen formundaki pozitif sayıların onluk sistemdeki karşılığı, ikilik sistemdeki sayının sahip olduğu 1'lerin ağırlık değerlerinin toplanmasıyla elde edilir. Negatif sayılarda ise işaret bitinin negatif ağırlığı ile diğer bitlerin ağırlıkları toplamına 1 ilave edilir.

Örnek: 00011000_2 sayısı işaret biti 0 olduğundan pozitif bir sayıdır.
Değeri $2^4+2^3=24$ 'tür.

11100111_2 sayısı ise işaret biti 1 olduğundan negatif bir sayıdır. Bu sayı 1'e tümleyen gösterimi olduğundan onluk sistemdeki karşılığı,
 $-2^7+2^6+2^5+2^2+2^1+2^0 + 1 = -24$ 'tür.

İşaretili Sayıların Onluk Sistemdeki Karşılığı

2'ye tümleyen formundaki pozitif ve negatif sayıların onluk sistemdeki karşılığı, işaret bitinin negatif ağırlık değeriyle diğer bitlerin ağırlık değerlerinin toplanması neticesinde elde edilir. N bitlik işaretili 2'ye tümleyen formuyla -2^{N-1} ile $(2^{N-1} - 1)$ arasındaki sayılar gösterilebilir.

Örnek: 2'ye tümleyen formundaki 01000101_2 sayısının onluk sistemdeki karşılığı, $2^6 + 2^2 + 2^0 = 69$ 'tur.

10111011_2 sayısının karşılığı ise $-2^7 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 = -69$ 'tur.

2'ye tümleyen gösteriminin diğer gösterim şekillerine nazaran tercih edilmesinin sebebi hem pozitif hem de negatif sayılar için aynı prosedürün takip edilmesindendir. Oysa 1'e tümleyen gösteriminde farklı prosedürler işletilir ve sıfır sayısının iki gösterimi (00000000_2 ve 11111111_2) vardır.

Sayı Sistemleri

Geçen Hafta

- Analog ve Sayısal Büyüklük Kavramı
- Sayı Sistemleri
- İkilik Sistemde Aritmetik İşlemler
- İkili Sayıların 1'e ve 2'ye Tümleyeninin Bulunması
- İşaretli Sayılar

Bu Hafta

- Kayan Noktalı Sayılar
- İşaretli Sayılarda Aritmetik İşlemler
 - Toplama ve Çıkarma İşlemleri
 - Çarpma ve Bölme İşlemleri

Kayan Noktalı Sayılar

Diğer gösterim şekilleri kullanılarak tüm tamsayı değerlerini göstermek oldukça fazla sayıda bit gerektirecektir. Bunun yanı sıra kesirli sayıların da gösterimi gereklidir. Kayan nokta gösterimiyle, hem çok küçük hem de çok büyük sayıları bilimsel formda göstermek mümkündür. Örneğin IEEE-754'e göre kayan noktalı sayılar 32 bit ile gösterilir; bir bit işareti, 8 bit üst kısmını, 23 bit de kesirli kısmı gösterir. 64 bitlik ve 80 bitlik gösterimler de mevcuttur.

İşaret	Üst kısmı	Kesir kısmı
--------	-----------	-------------

1 bit

8 bit

23 bit

32 bitlik gösterim

Normalizasyon

Herhangi ikili bir sayı bilimsel notasyonda farklı şekillerde gösterilebilir. Bir standart getirmek gayesiyle en soldaki bit değeri 1 olacak şekilde düzenlemeye gidilir. Buna normalizasyon denir.

Örneğin 10011001011_2 sayısı $1.0011001011 \times 2^{10}$ olarak gösterilir.

Normalize edilmiş bu formatın kayan nokta gösteriminde, üst kısmı biaslı olarak kullanılır yani gerçek üst değerine 127 ($2^8 - 1$) sayısı eklenir.

Bunun sebebi ise çok büyük ve çok küçük sayıların gösterimine olanak sağlamaktır. Gerçek üst değeri -126 ile 128 arasındadır.

Kayan nokta gösteriminde tüm bitlerin sıfır olması 0 olarak, üst kısmındaki bitlerin hepsinin 1, kesir kısmındaki tüm bitlerin 0 olması da sonsuz olarak yorumlanır.

Kayan Noktalı Sayılar

Örnek: $1.0011001011 \times 2^{10}$ sayısının kayan nokta gösterimi;

Sayı pozitif olduğundan işaret biti 0'dır. Üst kısmı $10+127$ (bias)=137 ve kesirli kısım da 0011001011_2 dir.

0	10001001	00110010110...0
1 bit	8 bit	23 bit

Yine aynı sayının negatifi gösterilmek istenirse, işaret bitinin 1 yapılması yeterlidir.

32 bitlik kayan nokta gösterimi ile gösterilebilecek en büyük sayıyı ikilik sistemde gösterebilmek için 129 bite ihtiyaç duyulacaktı.

Kayan Noktalı Sayılar

Örnek: Sayıların 4 bit ile gösterildiği bir sistemde, işaret kısmına 1 bit, üst kısmına 2 bit ve kesir kısmına da 1 bit ayırdığımızı düşünelim. Normalize edilmiş formata göre gösterebileceğimiz sayı aralığını inceleyelim.

İşaret	Üst kısım	Kesir kısım
1 bit	2 bit	1 bit

Bias değerimiz $2^{2-1}-1=1$ 'dir.

Yazılabilecek sayılar;

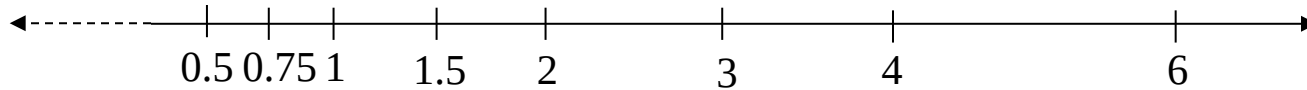
$$1.0 \times 2^{-1} = 0.5 \quad 1.1 \times 2^{-1} = 0.75$$

$$1.0 \times 2^0 = 1 \quad 1.1 \times 2^0 = 1.5$$

$$1.0 \times 2^1 = 2 \quad 1.1 \times 2^1 = 3$$

$$1.0 \times 2^2 = 4 \quad 1.1 \times 2^2 = 6$$

Örnek: (Devamı)



- Sayı ekseninde sayıların eksi değerleri de vardır. 4 bit ile gösterilebilecek sayı adedi 16 olduğundan kayan nokta formatıyla da 16 adet sayı gösterilebilmektedir. Ancak hassasiyet artmıştır ve sayılar ekseninde homojen olarak dağılmamıştır.
- Dikkat edilecek olursa 0.5 ile 1 arasında 3 sayı gösterebilmekteyiz. Aynı kural 1 ile 2 arasındaki, 2 ile 4 arasındaki sayılar için de geçerlidir.
- Şayet 2 biti kesir kısmı, 1 biti üst kısmı için ayırmış olsaydık, daha da hassas olarak sayıları gösterebilirdik (1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5). Ancak, üst kısmına daha az bit ayırdığımızdan gösterebileceğimiz sayının büyüklüğü azalmıştır.

İşaretili Sayılarda Aritmetik İşlemler

Bu kısımda daha sıklıkla kullanılan 2'ye tümleyen sistemi esas alınacaktır.

Toplama ve çıkarma işlemleri:

- İki pozitif sayının toplamı pozitiftir.

Örnek: 00001001 (9)
+ 00000100 (4)

00001101 (13)

- Pozitif bir sayıyla daha küçük negatif bir sayının toplamı pozitiftir. Elde biti oluşursa göz ardı edilir.

Örnek: 00001001 (9)
+ 11111010 (-6)

100000011 (3)

İşaretili Sayılarda Aritmetik İşlemler

- Pozitif sayının büyüklüğünden daha büyük negatif bir sayı olursa, oluşan toplam 2'ye tümleyen formunda negatif bir sayıdır.

Örnek: 00001000 (8)

+ 11110100 (-12)

11111100 (2'te tümleyen formunda -4'e karşılık gelir)

- İki negatif sayının toplamı negatiftir, elde biti oluşursa göz ardı edilir.

Örnek: 11111101 (-3)

+ 11111110 (-2)

111111011 (-5)

İşaretili Sayılarda Aritmetik İşlemler

- İşaretili sayılar toplandığında elde biti oluşursa göz ardı edilir. İki istisnai durum vardır; ilki, iki sayı da pozitif ancak işaret biti 1 ise, diğeri ise iki sayı da negatif ancak işaret biti 0 ise durumlarıdır. Her iki durumda da bir taşma meydana gelmiştir yani toplamı ifade etmek için 1 bite daha gereksinim olduğu söylenebilir.

Örnek:
$$\begin{array}{r} 01101110 \quad (110) \\ + 00011001 \quad (25) \\ \hline 10000111 \quad (\text{taşma durumu}) \end{array}$$

İşaretili sayılarda çıkartma işlemi aslında toplama işleminin değişik bir formudur. Yani çıkartılan sayının 2'ye tümleyeni ile çıkan sayının toplamından sonuç elde edilir.

Örnek: 8-3 işlemi $8+(-3)$ olarak da yazılabilir.

$$\begin{array}{r} 00001000 \quad (8) \\ + 11111101 \quad (-3) \\ \hline \underline{100000101} \quad (5) \end{array}$$

Çarpma ve Bölme İşlemleri

Çarpma, toplama işlemiyle gerçekleştirilebilir. Çarpılan sayının, çarpanın değeri kadar kendi üzerine toplanması esasına dayanır. Yani 4×2 işlemi $4+4$ şeklinde düşünülür. Çarpan değerinin büyük olması durumunda zahmetli bir hal alır bu yüzden kısmi çarpım metodu kullanılır;

- İlk olarak çarpılan ve çarpanın aynı işarete sahip olup olmadığına bakılır ve çarpım sonucunun işareti elde edilir.
- Negatif sayının pozitif değeri alınır.
- Çarpanın en az anlamlı bitinden başlanır ve kısmi çarpımlar elde edilir. Şayet çarpan 1 ise çarpılanın kendisi, 0 ise 0 değerinden kısmi çarpım elde edilir. Her kısmi çarpım bir sola kaydırılır.
- Ard arda gelen kısmi çarpımlar toplanır ve ara toplamalar elde edilir.
- İlk adımda işaret biti negatifse, sonucun ikiye tümleyenini alınır. Pozitifse olduğu gibi bırakılır. Elde edilen çarpımın önüne işaret biti eklenir.

Çarpma ve Bölme İşlemleri

Örnek: $25 \times (-4)$ işlemini yapalım;

11001	
$\times 100$	-4'ün pozitif değeri
00000	1.kısmi çarpım
$+ 00000$	2.kısmi çarpım
000000	ara toplam
$+ 11001$	3. kısmi çarpım
1100100	sonuç

Çarpımın sonucu negatif olacağından sonucun ikiye tümleyeni alınır;

1100100_2 'ın ikiye tümleyeni 0011100_2 ve işaret biti eklendiğinde 10011100_2 elde edilir.

Bölme İşlemi

Bölme, çıkarma işlemiyle gerçekleştirilebilir. Çıkarma, toplama işlemi ile yapılabildiğinden bölme işlemi de toplama işlemiyle yapılabilir. Bölenin bölünenden çıkarılma sayısı bölümü verecektir. Bölme işleminde şu adımlar takip edilir;

- İlk olarak bölünen ve bölenin aynı işarete sahip olup olmadığına bakılır ve bölme sonucunun işareti elde edilir.
- Her iki sayının da pozitif formu elde edilir.
- Bölüme 0 değeri atanır.
- Bölünenden bölen çıkartılır (2'ye tümleyen ile toplama işlemine dönüştürülebilir), kısmi kalan bulunur ve bölüme 1 eklenir. Kısmi kalan pozitifse bir sonraki adıma gidilir, 0 veya negatifse bölme işlemi tamamlanmıştır.
- Kısmi kalandan bölen çıkartılır ve bölüme 1 eklenir. Sonuç pozitifse sonraki kısmi kalan için işlemler tekrar edilir. Sonuç 0 veya negatifse bölme işlemi tamamlanmıştır.

Bölme İşlemi

Örnek: 25/6 işlemini yapalım.

Her iki sayı da pozitif olduğundan sonuç pozitifdir.

$$25 = 00011001_2 \text{ ve } 6 = 00000110_2$$

Bölüme 0 değeri atanır. Bölünenden bölen çıkartılır;

$$\begin{array}{r} 00011001 \\ + 11111010 \text{ (6'nın 2'ye tümleyeni)} \\ \hline 100010011 \text{ (elde biti atılır) kısmi kalan pozitif olduğundan} \end{array}$$

bölüm=0+1=1

Kısmi kalandan bölen çıkartılır;

$$\begin{array}{r} 00010011 \\ + 11111010 \\ \hline 100001101 \text{ (elde biti atılır) kısmi kalan pozitif olduğundan} \end{array}$$

bölüm=1+1=2

Kısmi kalandan bölen çıkartılır;

$$\begin{array}{r} 00001101 \\ + 11111010 \\ \hline 100000111 \text{ (elde biti atılır) kısmi kalan pozitif olduğundan} \end{array}$$

bölüm=2+1=3

Örnek: (Devamı)

Kısmi kalandan bölen çıkartılır;

$$\begin{array}{r} 00000111 \\ + 11111010 \\ \hline \end{array}$$

100000001 (elde biti atılır) kısmi kalan pozitif olduğundan

bölüm=3+1=4

Kısmi kalandan bölen çıkartılır;

$$\begin{array}{r} 00000001 \\ + 11111010 \\ \hline \end{array}$$

11111011 elde oluşmadığından sonuç negatiftir, bölme işlemi tamamlanmıştır. Bölme işleminin sonucu bölümün aldığı değerdir,

4'ü ikilik sistemde ifade edersek 00000100_2 dır. Kalan da bir önceki işlemde 1 olarak bulunur.

Sayı Sistemleri

Geçen Hafta

- Kayan Noktalı Sayılar
- İşaretili Sayılarda Aritmetik İşlemler
 - Toplama ve Çıkarma İşlemleri
 - Çarpma ve Bölme İşlemleri

Bu Hafta

- Onaltılık Sayı Sistemi
- Sekizlik (Octal) Sayı Sistemi
- Dönüşüm İşlemleri

Onaltılık (Hexadecimal) Sayı Sistemi

Onaltılık sayı sisteminde 16 rakam vardır. İlk on rakam için 0 ile 9 arasındaki sayılar, sonraki rakamlar için ise A ile F arasındaki harfler kullanılır. Çok büyük ikili sayıları okumak ve yazmak oldukça zordur. Bilgisayarın anladığı dil ise 1 ve 0'lardır. Bir bilgisayar komutunun 32 bit olduğu düşünülürse, makine dilinde bir program yazmaya kalkışmak oldukça zor olacaktır. Yani onaltılık sistem ikili sisteme göre daha okunaklıdır diyebiliriz. Bu sayı sisteminde her hexadecimal dijital, 4 bitlik ikili sayıya karşılık gelir.

Onluk	İkili	Onaltılık
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Onaltılık (Hexadecimal) Sayı Sistemi

Onaltılık sayı sisteminde F_{16} 'ten sonra gelen sayı 10_{16} dır ve $11_{16}, 12_{16}, \dots, 19_{16}, 1A_{16}, \dots, 1F_{16}$ şeklinde devam eder.

2 dijitlek bir hexadecimal sayı ile 0 (00_{16}) ile 255 (FF_{16}) arasındaki sayılar gösterilebilir.

➤ İkilik sistemden onaltılık sisteme dönüşüm için gereken şey, ikilik sistemde verilmiş olan sayının dörderli bit gruplarına ayrılması ve karşılık gelen hexadecimal sayının yazılmasıdır. Bu ayırma işlemi sayının tam kısmı için sağdan sola doğru, kesirli kısmı için ise soldan sağa doğrudur. Şayet dörderli bit gruplarına ayırma işleminde, 4 bitten daha az sayıda bit grubu bulunursa eksik kalan yerlere 0 konulur.

Örnek: Dönüşüm İşlemleri

❖ 010001111110001_2 ikili sayısının onaltılık sistemdeki karşılığını bulmak için bu sayıyı dörderli bit gruplarına ayırmak gerekir.

$\underbrace{0010}_2 \underbrace{0011}_3 \underbrace{1111}_F \underbrace{0001}_1$

O halde $(010001111110001)_2 = (23F1)_{16}$

❖ 1011001101.110010_2 sayısının 16'lık sistemdeki karşılığını bulalım.

1011001101.110010_2 sayısını dörderli bit gruplarına ayırırsak;

$0010 \ 1100 \ 1101 \ . \ 1100 \ 1000_2 = 2CD.C8_{16}$ olarak bulunur.

Dönüşüm İşlemleri

➤ Onaltılık sistemdeki bir sayıyı da ikilik sisteme çevirmek için, onaltılık sistemdeki sayının her dijiti için, 4 bitlik ikili karşılığı yazılır.

Örnek: $4A3F_{16}$ sayısının ikilik sistemdeki karşılığını bulmak için her dijitin dört bit ile ikili olarak ifade edilmesi gerekir.

4	A	3	F
0100	1010	0011	1111

O halde $4A3F_{16} = 0100101000111111_2$

Örnek: $ABC.DE_{16}$ sayısının 2'lik sistemdeki karşılığını bulalım.

Her bir hexadecimal dijiti 4 bit olarak ifade edersek;

$ABC.DE_{16} = 1010\mathbf{1011}1100.\mathbf{1101}1110_2$

Dönüşüm İşlemleri

➤ Onaltılık sistemden onluk sisteme dönüşüm için iki yol düşünülebilir; onluk sistemdeki sayılar önce ikilik sisteme sonra da ikilik sistemden onluk sisteme çevrilir.

Örnek: $2A_{16} = 00101010_2 = 2^5 + 2^3 + 2^1 = 42_{10}$

➤ Diğer yöntem ise onaltılık sistemdeki her basamağın onluk sistemdeki karşılığını, basamak değeriyle çarpıp, bu çarpımları toplamaktır.

Örnek: $(1B.1A)_{16}$ sayısının 10'luk sistemdeki karşılığını bulalım.

Bir önceki örnekteki gibi her bir hexadecimal dijital, basamak ağırlıklarıyla çarpılıp toplanır.

$$(1B.1A)_{16} = 1 \times 16^1 + 11 \times 16^0 + 1 \times 16^{-1} + 10 \times 16^{-2} = 16 + 11 + 1/16 + 10/256 \cong 27.1$$

Dönüşüm İşlemleri

➤ Onluk sistemden onaltılık sisteme dönüşüm için yapılması gereken, sayıyı sürekli 16'ya bölüp, kalan kısımlardan onaltılık sayıyı oluşturmaktır.

Örnek: 2577_{10} sayısının onaltılık sistemdeki karşılığı,

$$\begin{array}{r|l} 2577 & 16 \\ \hline -16 & 161 \\ \hline 97 & -160 \\ \hline & 10 \rightarrow A_{16} \\ -96 & 1 \\ \hline & 17 \\ -16 & \\ \hline & 1 \end{array}$$

O halde $2577_{10} = A11_{16}$

Örnek:

Örnek: 123.256 sayısının 16'lık sistemdeki karşılığını bulalım.

Tamsayı kısmını ve ondalıklı kısmı ayrı ayrı düşünmek gerekir.

Tamsayı Kısmı		
	Bölüm	Kalan
123/16	7	11 (B)
7/16	0	7

Ondalıklı Kısım	
0.256×16	4.096
0.096×16	1.536
0.536×16	8.576

$$123.536 \cong 7B.418_{16}$$

Onaltılık Sistemde Toplama

İki hexadecimal sayının toplama işleminde, her basamağın onluk sistemdeki karşılıkları toplanır. Şayet toplam 15 ve altındaysa karşılık gelen hexadecimal değeri yazılır. Toplam 15'ten büyükse bu değerin 16'yı geçen kısmı, elde olarak bir sonraki basamağa kaydırılır, 16'yı geçmeyen kısmı ise sonuca yazılır.

Örnek: 29_{16}
+ $1A_{16}$

 43_{16}

$$9_{16} + A_{16} = 9_{10} + 10_{10} = 19_{10}$$
$$19 - 16 = 3 \text{ elde } 1$$
$$2 + 1 + 1(\text{elde}) = 4$$

Örnek: EF_{16}
+ $9B_{16}$

 $18A_{16}$

$$F_{16} + B_{16} = 15_{10} + 11_{10} = 26_{10}$$
$$26 - 16 = 10 = A_{16} \text{ elde } 1$$

$$E_{16} + 9_{16} + 1(\text{elde}) = 14_{10} + 9_{10} + 1 = 24_{10}$$
$$24 - 16 = 8 \text{ elde } 1$$

Onaltılık Sistemde Çıkartma

İkilik sistemde çıkartma işlemi 2'ye tümleyen ile toplama işlemine dönüşüyordu. Bu sistemde çıkartma yapmak için yine aynı yöntem kullanılabilir; önce çıkarılacak olan sayı ikilik sisteme çevrilir daha sonra 2'ye tümleyeni alınır ve onaltılık sisteme dönüştürülür. Böylece çıkartma işlemi toplama işlemine dönüşmüş olur.

Örnek: $25_{16} - 1B_{16}$ işlemini yapmak için;

$1B_{16} = 00011011_2$ sayısının 2'ye tümleyeni 11100101_2 ve onaltılık sistemdeki karşılığı $E5_{16}$ 'tir.

$$\begin{array}{r} 25_{16} \\ + E5_{16} \\ \hline 10A_{16} \end{array}$$

oluşan elde biti 2'ye tümleyenindeki gibi göz ardı edilir.

Sekizlik (Octal) Sayı Sistemi

Onaltılık sayı sisteminde olduğu gibi, ikili sayıları veya kodları daha kolay anlaşılır olarak ifade etmeye yarar. Onaltılık sayı sistemine göre daha az sıklıkla kullanılır. Sekizlik sayı sisteminde 0 ile 7 arasındaki rakamlar vardır. Bu sistemde 7'den sonraki sayılar 10,11,12,...,17,20 şeklinde ifade edilir.

➤ Sekizlik sistemdeki sayıları onluk sisteme çevirmek için her basamak, ağırlık değeriyle çarpılır ve bu çarpımlar toplanır.

Örnek: 1234_8 sayısının onluk sistemdeki karşılığı;

$$1234_8 = 1 \times 8^3 + 2 \times 8^2 + 3 \times 8^1 + 4 \times 8^0 = 668_{10}$$

Örnek: 12.34_8 sayısının onluk sistemdeki karşılığı;

$$12.34_8 = 1 \times 8^1 + 2 \times 8^0 + 3 \times 8^{-1} + 4 \times 8^{-2} = 8 + 2 + 3/8 + 4/64 \cong 10.44$$

Dönüşüm İşlemleri

➤ Onluk sistemden sekizlik sisteme dönüşüm işleminde ise sayı, sürekli 8'e bölünür, bölme neticesinde kalan kısımlar sayının sekizlik sistemdeki karşılığını verir.

Örnek: 123_{10} sayısının sekizlik sistemdeki karşılığı;

$$\begin{array}{r|l} 123 & 8 \\ - 8 & 15 \quad 8 \\ \hline 43 & - 8 \quad 1 \\ - 40 & 7 \\ \hline & 3 \end{array}$$

O halde $123_{10} = 173_8$

Dönüşüm İşlemleri

Örnek: 34.45 sayısının sekizlik sistemdeki karşılığı;

Tamsayı Kısmı		
	Bölüm	Kalan
34/8	4	2
4/8	0	4

Ondalık Kısım	
0.45×8	3.6
0.6×8	4.8
0.8×8	6.4

$$34.45 \cong 42.346_8$$

➤ Sekizlik sistemden ikilik sisteme dönüşüm için, her basamak üç bitle temsil edilecek şekilde düzenleme yapılır.

Örnek: 456_8 sayısının ikilik sistemdeki karşılığı;

$$456_8 = \underline{100101110}_2$$

Örnek: 56.34_8 sayısının ikilik sistemdeki karşılığı;

8'lik sistemdeki her bir rakam, 3 bit ile temsil edilir.

$$56.34_8 = \mathbf{101110.011100}_2$$

Dönüşüm İşlemleri

➤ İkilik sistemden sekizlik sisteme dönüşüm için, sekizlikten ikiliğe dönüşüm işleminin tersi yapılır. Yani ikilik sistemdeki sayının bitleri, sağdan sola doğru üçerli olarak gruplanır. Sayı, kesirli kısım içeriyorsa, bu kısım soldan sağa doğru üçerli gruplara ayrılır ve sekizli sistemdeki karşılığı yazılır (son grupta eksik bit olabilir, eksik kısım 0 ile doldurulur).

Örnek: 101110001_2 ikili sayısının sekizlik sistemdeki karşılığı,
 $\underline{101}\underline{110}\underline{001}_2 = 561_8$

Örnek: 10011.11010_2 ikili sayısının sekizlik sistemdeki karşılığı,
010 011 . 110 100 (Üçerli gruplama işlemi)

O halde $10011.11010_2 = 23.64_8$

Dönüşüm İşlemleri

➤ Sekizlik sistemde verilen bir sayının onaltılık sistemdeki karşılığını bulmak istersek, ilk olarak sekizlik sistemde verilen sayının ikilik sistemdeki karşılığını buluruz daha sonra da dörderli bit gruplarına ayırırız. Benzer şekilde onaltılık sistemde verilen bir sayının sekizlik sistemdeki karşılığını bulmak istersek, ilk olarak onaltılık sistemde verilen sayının ikilik sistemdeki karşılığını buluruz daha sonra da üçerli bit gruplarına ayırırız.

Örnek: 45.63_8 sayısının onaltılık sistemdeki karşılığı,

$$45.63_8 = 100101.110011_2$$

0010 0101 . 1100 11**00** (Dörderli bit gruplarına bölme işlemi)

O halde $45.63_8 = 25.CC_{16}$ olarak bulunur.

Örnek: $A2.B3_{16}$ sayısının sekizlik sistemdeki karşılığı,

$$A2.B3_{16} = 10100010.10110011_2$$

010 100 010 . 101 100 11**0** (Üçerli bit gruplarına bölme işlemi)

O halde $A2.B3_{16} = 242.546_8$ olarak bulunur.

KODLAMA

❖ BCD (Binary Coded Decimal)

BCD'de Toplama İşlemi

❖ Gray Kodu

❖ Parity Kodu

❖ 5'te 2 Kodu

❖ Aiken Kodu

❖ Alfasayısal Kodlar

Kodlama

Genel olarak verileri göstermek için rakamları, harfleri ve sembolleri kullanırız. Ancak sayısal sistemler 1 ve 0 mantığı üzerine kurulmuştur. Kullandığımız veri yapısını sayısal sistemlere dönüştürme işlemine kodlama denir.

Kodlama kendi içerisinde sayısal (numeric) kodlar ve alfasayısal (alphanumeric) kodlar olmak üzere ikiye ayrılır.

Sayısal kodlar da basamak ağırlığı olan ve basamak ağırlığı olmayan olarak ikiye ayrılır. Örnek: BCD, Gray, Parity, 5'te 2 kodu ve Aiken

En çok kullanılan alfasayısal kodlar ise ASCII, Genişletilmiş ASCII, EBCDIC ve UNICODE'dur.

BCD (Binary Coded Decimal)

İkili kodlanmış onluk sistem manası taşımaktadır. Her onluk basamağın ikili kodla gösterildiği bir kodlama türüdür. Bu sistemde 10 kod grubu vardır.

Bu sistemde her onluk basamak 4 bit ikili sayı ile ifade edilir. Basamak ağırlıkları 2^3 2^2 2^1 2^0 dır. Bu kodlamaya 8421 kodu da denilmektedir.

Örnek: 25_{10} sayısının BCD'ye çevirmek için her basamağı 4 bit ile ikili olarak ifade etmek gerekir. Ters işlem de mümkündür.

2	5	
↓	↓	
0010	0101	O halde $25_{10} = 00100101_{\text{BCD}}$

BCD'de Toplama İşlemi

BCD, basamak ağırlığı olan sayısal kodlar grubuna girdiğinden aritmetik işlemlerde kullanılabilir. İki BCD sayıyı toplarken üç kural gözetilir;

1. İkilik sistemdeki toplama kuralları geçerlidir.
2. Şayet dörder bitlik grupların toplamı 9 veya daha küçükse sonuç geçerli BCD sayıdır.
3. Eğer toplam 9'dan büyükse veya elde oluşmuşsa sonuç geçerli bir BCD sayı değildir. 6 geçersiz durumdan kurtulmak için toplama 6 (0110_2) sayısı eklenir. Şayet bu toplam sonucunda elde oluşursa, bu elde diğer dörder bit grubuna dahil edilir.

BCD'de Toplama İşlemi

Örnek: $8_{10} + 4_{10}$ işlemini BCD sisteminde yapalım.

$$\begin{array}{r} 8_{10} = 1000_{\text{BCD}} \\ 4_{10} = 0100_{\text{BCD}} \\ \hline 1100_{\text{BCD}} \rightarrow \text{Bu sayı 9'dan büyük olduğundan} \\ + 0110_{\text{BCD}} \quad \text{6 ilave edilir.} \\ \hline 0001\ 0010_{\text{BCD}} \end{array}$$

Örnek: $39_{10} + 97_{10}$ işlemini BCD sisteminde yapalım.

$$\begin{array}{r} 39_{10} = 0011\ 1001_{\text{BCD}} \\ 97_{10} = 1001\ 0111_{\text{BCD}} \\ \hline 1101\ 0000 \rightarrow \text{Elde olduğu için 6 eklemek gerekiyor.} \\ + 0110\ 0110 \\ \hline 0001\ 0011\ 0110_{\text{BCD}} \end{array}$$

9'dan büyük olduğu için 6 eklemek gerekiyor.

Gray Kodu

Bitlerin pozisyonlarına atanmış ağırlıklar olmadığından aritmetik kod grubuna girmez. Gray kodunun özelliği bir kod kelimesinden diğerine geçişte sadece bir bitin değişim göstermesidir. Genellikle hata kontrolü için kullanılır.

Onluk sistem	İkilik Sistem	Gray Kodu
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101



Frank Gray

Gray Kodu

İkilik sistemden Gray koduna dönüşüm işleminde MSB bitinin değeri olduğu gibi alınır, soldan sağa doğru ard arda gelen bitler toplanır ve sonra gelen Gray kod bitleri elde edilir. Toplam sonucunda elde biti oluşursa göz ardı edilir.

Örnek: 101101_2 ikili sayısını Gray koduna dönüştürelim;

$$\begin{array}{cccccc} 1 & + & 0 & + & 1 & + & 1 & + & 0 & + & 1 \\ \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow \\ 1 & & 1 & & 1 & & 0 & & 1 & & 1 \end{array} \quad \text{O halde } 101101_2 = 111011_{\text{Gray}}$$

Gray kodundan ikilik sisteme dönüşüm işleminde yine MSB biti aynen alınır. Bu MSB biti Gray koddaki bir sonraki bitle toplanır ve ikili sayının ikinci biti elde edilir. Oluşan ikinci bitle de bir sonraki Gray kod bitini toplanır ve bu şekilde devam eder. Toplam sonucunda elde biti oluşursa göz ardı edilir.

Örnek: 101101_{Gray} kodunun ikilik sistemdeki karşılığı,

$$\begin{array}{cccccc} 1 & 0 & 1 & 1 & 0 & 1 \\ \downarrow + \nearrow & \downarrow + \nearrow & \downarrow + \nearrow & \downarrow + \nearrow & \downarrow + \nearrow & \downarrow \\ 1 & 1 & 0 & 1 & 1 & 0 \end{array} \quad \text{O halde } 101101_{\text{Gray}} = 110110_2$$

Parity Kodu

Parity kodu özellikle bilgilerin taşınması sırasında oluşabilecek bit hatalarını tespit etmek için kullanılır. Tek veya çift parity kodu kullanılmaktadır. İletilecek bilginin 7 bit olduğu düşünülürse ve tek parity kullanılırsa, sekizinci bit parity biti olacaktır.

Bu bilgi tek sayıda 1 içeriyorsa parity biti 0, çift sayıda 1 içeriyorsa parity biti 1 olur. Benzer şeyleri çift parity için de söylemek mümkündür; iletilecek bilgidaki 1'lerin sayısı çift sayıdaysa parity biti 0, tek sayıda 1 içeriyorsa parity biti 1 olur.

Örnek: $(1010001)_2$ sayısını tek eşlik biti yöntemini göre kodlayalım.

$(1010001)_2$ sayısı tek sayıda 1 içerdiğinden parity biti 0 olacaktır. Yani kodlanmış bilgi 01010001_2 olacaktır.

Şayet çift eşlik biti yöntemini kullanarak kodlarsak;

$(1010001)_2$ sayısı tek sayıda 1 içerdiğinden parity biti 1 olacaktır. Yani kodlanmış bilgi 11010001_2 olacaktır.

5'te 2 Kodu

5'te 2 kodlama sisteminde, onluk sistemdeki her sayı, içinde iki tane 1 bulunan 5 bitlik ikili sayı şeklinde kodlanır. Böylelikle hatalar kolaylıkla tespit edilebilir. 5 bitlik ikili sayının basamak ağırlıkları '7-4-2-1-0' dır.

Sayılar	5'te 2 Kodu
	7 4 2 1 0
0	1 1 0 0 0
1	0 0 0 1 1
2	0 0 1 0 1
3	0 0 1 1 0
4	0 1 0 0 1
5	0 1 0 1 0
6	0 1 1 0 0
7	1 0 0 0 1
8	1 0 0 1 0
9	1 0 1 0 0

Aiken Kodu

Aiken kodunda sayılar 4 bit ile ifade edilirler ve basamak ağırlıkları '2-4-2-1' şeklindedir. 5'e kadar olan sayıları kodlamak için sağ taraftaki bitler kullanılırken, 5'ten büyük sayıları kodlamak için sol taraftaki bitler kullanılır. Simetrik bir kodlama türüdür.

Sayı	Aiken Kodu
	2 4 2 1
0	0 0 0 0 ←
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0 ←
5	1 0 1 1 ←
6	1 1 0 0
7	1 1 0 1
8	1 1 1 0
9	1 1 1 1 ←



Howard Aiken

Alfasayısal (Alphanumeric) Kodlar

- İletişim için sadece sayıları kullanmak yetmez bunun yanında harflere ve diğer sembollere de ihtiyaç duyulur. Ayrıca iletişim için birçok uygulamada sayıların, harflerin, sembollerin ve noktalama işaretlerinin haricinde, alıcı sistemin bu bilgiyi ne yapacağı ile ilgili komutlara da ihtiyaç vardır. En popüler alfasayısal kodlama türü ASCII'dir (American Standart Code for Information Interchange).
- ASCII, bilgisayar ve diğer birçok sistem için kabul görmüş evrensel bir alfasayısal kod türüdür. Örneğin klavyeden tuşlara basıldığında, basılan tuşun ASCII kod karşılığı bilgisayara gönderilir. Bu kodun içerdiği bit sayısı 7'dir. Sekizinci bit hata kontrolü için kullanılabilir. 7 bit ile 128 karakter sembolize edilebilir. Bu karakterlerin ilk 32'si kontrol amaçlıdır ve ekrana basılamazlar, diğerleri ise ekrana basılabilirler. Diğer bir uygulama da bilgisayar ile yazıcı arasındaki bilgi transferidir.

ASCII

Karakter	7-Bit ASCII	Onaltılık	Karakter	7-Bit ASCII	Onaltılık
A	100 0001	41	0	011 0000	30
B	100 0010	42	1	011 0001	31
C	100 0011	43	2	011 0010	32
D	100 0100	44	3	011 0011	33
E	100 0101	45	4	011 0100	34
F	100 0110	46	5	011 0101	35
G	100 0111	47	6	011 0110	36
H	100 1000	48	7	011 0111	37
I	100 1001	49	8	011 1000	38
J	100 1010	4A	9	011 1001	39
K	100 1011	4B	boşluk	010 0000	20
L	100 1100	4C	.	010 0001	2E
M	100 1101	4D	(	010 1000	28
N	100 1110	4E	+	010 1011	2B
O	100 1111	4F	\$	010 0100	24
P	101 0000	50	*	010 1010	2A
Q	101 0001	51	)	010 1001	29
R	101 0010	52	-	010 1101	2D
S	101 0011	53	/	010 1111	2F
T	101 0100	54	:	010 1100	2C
U	101 0101	55	=	011 1101	3D
V	101 0110	56	RETURN	000 1101	0D
W	101 0111	57	LINEFEED	000 1010	0A
X	101 1000	58			
Y	101 1001	59			
Z	101 1010	5A			

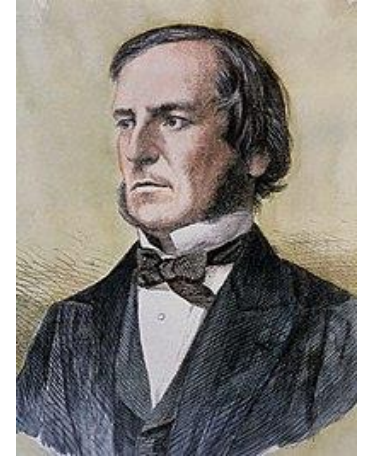
BOOLE CEBRİ

- ❖ Boole Cebrinin Temel Kanunları
- ❖ Boole Cebrinin Kuralları
- ❖ De Morgan Teoremleri
- ❖ Doğruluk Tablosu
- ❖ Venn Diyagramı
- ❖ Lojik İfadelerin Sadeleştirilmesi

BOOLE CEBRİ

1854 yılında George Boole tarafından özellikle lojik devrelerde kullanılmak üzere ortaya konulmuş bir matematiksel sistemdir. 1938'li yıllarda da ilk defa Claude Shannon tarafından Boole'un çalışması, lojik devrelerin tasarımı ve analizinde kullanılmıştır. Boole cebri AND, OR ve NOT temel mantıksal işlemlerinden oluşan sembolik bir sistem olarak düşünülebilir.

AND işlemi ikilik sistemdeki çarpma işlemine denk düşerken, OR işlemi de ikilik sistemdeki toplama işlemine karşılık gelmektedir. Mantıksal büyüklükleri göstermek için kullanılan lojik değişkenlerin iki değeri olabileceğinden, NOT işlemi değişkenin tümleyenini göstermektedir.



George Bool

Boole Cebrinin Temel Kanunları

Toplama ve çarpma işlemlerindeki kullanılan yer değiştirme (commutativity), birleşme (associativity) ve dağılma (distributivity) özellikleri Boole cebrinin temel kanunlarını oluşturur.

OR ve AND işlemleri için yer değiştirme kanunları;

- $A + B = B + A$
- $A \cdot B = B \cdot A$

OR ve AND işlemleri için birleşme kanunları;

- $(A + B) + C = A + (B + C) = A + B + C$
- $A \cdot B \cdot C = A \cdot (B \cdot C) = (A \cdot B) \cdot C$

Dağılma kanunu;

- $A \cdot (B + C) = A \cdot B + A \cdot C$

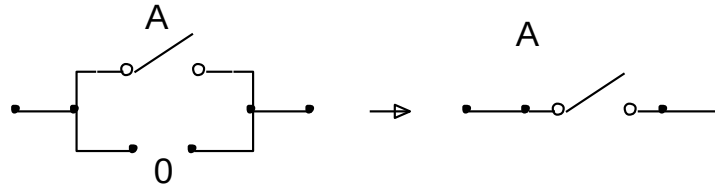
Boole Cebrinin Kuralları

Bu kurallar özellikle lojik ifadelerin sadeleştirilmesi için kullanılırlar.

- $A + 0 = A$

$$1 + 0 = 1$$

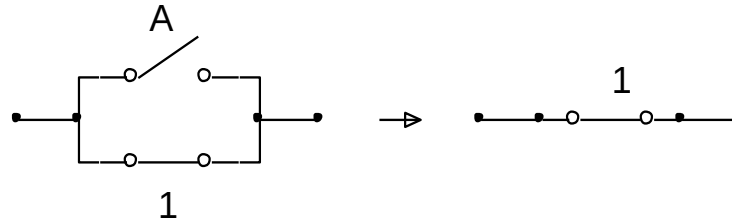
$$0 + 0 = 0$$



- $A + 1 = 1$

$$1 + 1 = 1$$

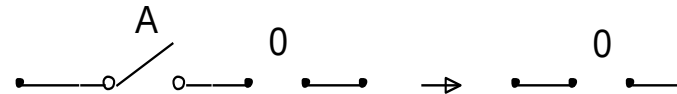
$$0 + 1 = 1$$



- $A \cdot 0 = 0$

$$1 \cdot 0 = 0$$

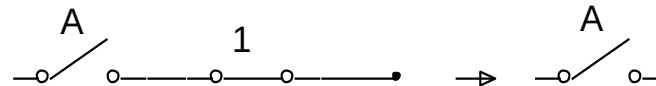
$$0 \cdot 0 = 0$$



- $A \cdot 1 = A$

$$1 \cdot 1 = 1$$

$$0 \cdot 1 = 0$$

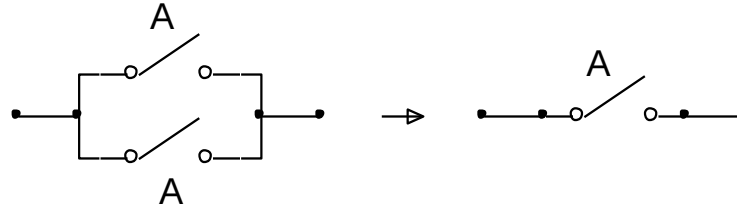


Boole Cebrinin Kuralları

- $A + A = A$

$$1 + 1 = 1$$

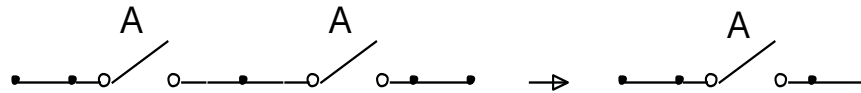
$$0 + 0 = 0$$



- $A \cdot A = A$

$$1 \cdot 1 = 1$$

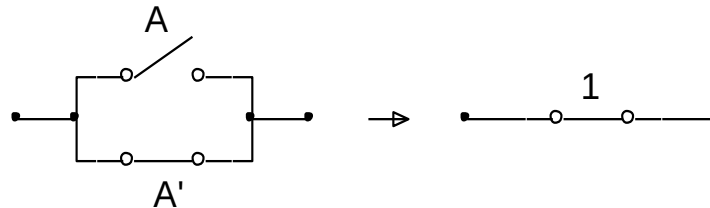
$$0 \cdot 0 = 0$$



- $A + A' = 1$

$$1 + 0 = 1$$

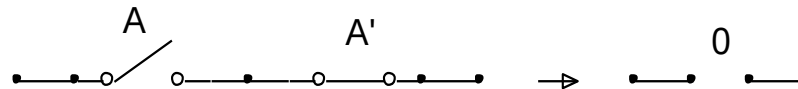
$$0 + 1 = 1$$



- $A \cdot A' = 0$

$$1 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$



- $(A')' = A$

Boole Cebrinin Kuralları

Bu kurallardan faydalanılarak çok sık kullanılan sadeleştirme kuralları da şunlardır;

- $A + A.B = A$

$$A.(1+B) = A$$

- $A + B.C = (A+B).(A+C)$

$$\begin{aligned}(A+B).(A+C) &= A.A + A.C + A.B + B.C = \underline{A + A.C} + A.B + B.C \\ &= \underline{A + A.B} + B.C = A + B.C\end{aligned}$$

- $A + A'.B = A + B$

$$(A + A')(A + B) = A + B$$

De Morgan Teoremleri

Birinci teoremi; değişkenlerin çarpımlarının tümleyeni, tümleyenlerinin toplamına eşittir.

$$(A \cdot B)' = A' + B'$$

İkinci teoremi; değişkenlerin toplamalarının tümleyeni, tümleyenlerinin çarpımına eşittir.

$$(A + B)' = A' \cdot B'$$

De Morgan teoremleri ikiden fazla değişken için ve lojik ifadeler için genişletilebilir:

- $(A \cdot B \cdot C)' = A' + B' + C'$
- $(A + B + C)' = A' \cdot B' \cdot C'$

$(A + B + C)' = A' \cdot B' \cdot C'$ olduğunu gösterelim,

$$[A + (B + C)]' = A' \cdot (B + C)' = A' \cdot B' \cdot C'$$



Augustus De Morgan

Doğruluk Tablosu

Doğruluk tablosu, bir lojik devredeki giriş değişkenlerinin alabilecekleri tüm değerlere karşılık gelen çıkışları gösterir. Doğruluk tablosundaki durum sayısı, n giriş değişkeni için 2^n dir.

Giriş değişkenlerinin kombinasyonlarını elde etmek için sistematik bir yaklaşım vardır; en düşük anlamlı değişken için bir 0 bir 1 yazılarak, ikinci değişken için iki 0 iki 1 yazılarak, üçüncü değişken için de dört 0 dört 1 yazılarak sütunlar oluşturulur. Değişken sayısı arttıkça yazılacak 1 ve 0'ların sayısı ikinin kuvveti olarak artacaktır.

<i>A</i>	<i>B</i>	<i>Çıkış</i>
0	0	
0	1	
1	0	
1	1	

<i>A</i>	<i>B</i>	<i>C</i>	<i>Çıkış</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Doğruluk Tablosu

Örnek: $F(A,B,C) = A+B'C$ ifadesinin doğruluk tablosunu oluşturalım.

Giriş değişkenlerinin tüm kombinasyonları, lojik ifadede yerine konularak tablo oluşturulur. Diğer bir yöntem de şöyle olabilir; bu lojik ifadenin 1 olabilmesi için ya A nın 1 olması ya da $B'C$ nin 1 olması gereklidir. $B'C$ nin de 1 olabilmesi için $B = 0$ ve $C = 1$ olmalıdır.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

B nin 0 ve C nin 1 olduğu durum

} A nın 1 olduğu durumlar

Doğruluk Tablosu

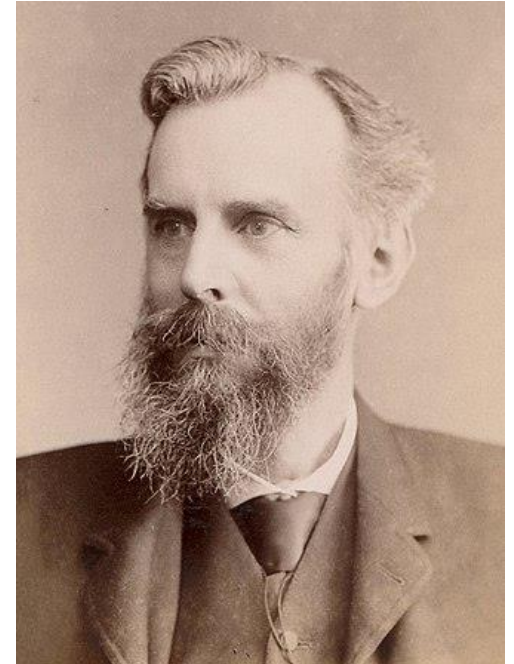
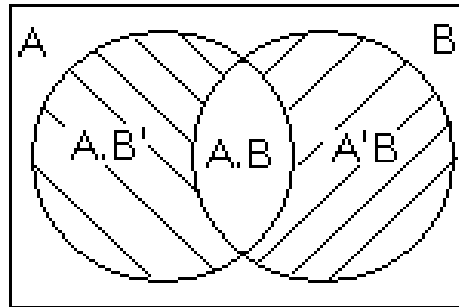
Örnek: $(A+B)' = A' \cdot B'$ DeMorgan teoreminin ispatını doğruluk tablosu ile gösterelim.

Eşitliğin sağ ve sol tarafındaki ifadelerden doğruluk tablosu oluşturulduğunda, ilgili sütunların aynı değerlere sahip olması, bu eşitliğin doğruluğunu ispatlayacaktır.

A	B	A'	B'	$A+B$	$(A+B)'$	$A' \cdot B'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

Venn Diyagramı

Venn diyagramı, lojik ifadeleri görsel olarak ifade etmeye yarayan bir araçtır. Bu yöntemde her bir küme, bir değişkeni temsil eder. Küme içerisindeki tüm noktalar değişkenin kendisini gösterirken, kümenin dışındakiler ise ‘değişkenin değil’ olarak ifade edilir.

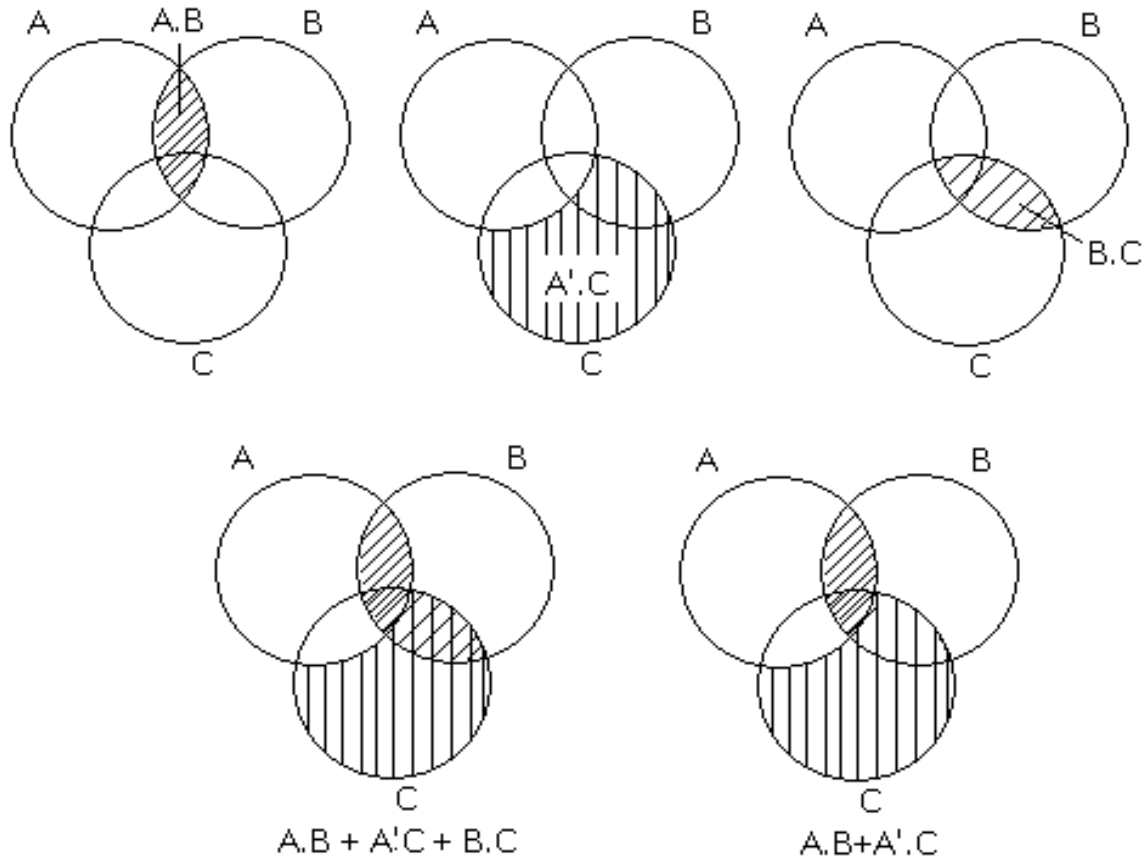


John Venn

Venn diyagramı yardımıyla da lojik ifadelerdeki eşitlikleri gösterebiliriz.

Venn Diyagramı

Örnek: $F(A,B,C) = A.B + A'.C + B.C = A.B + A'.C$ eşitliğinin doğruluğunu Venn diyagramı yardımıyla gösterelim.



Lojik İfadelerin Sadeleştirilmesi

Karmaşık yapıdaki lojik ifadeler, Boole cebri kuralları kullanılarak sadeleştirilebilirler. Sadeleşmiş lojik ifadelerden oluşturulacak devreler, hem daha basit hem de daha ucuz olarak elde edilmiş olacaktır. Sadeleştirmede birkaç yol takip edilebilir; ortak paranteze alma, terimleri genişletme ya da terim ilave etme gibi.

Örnek: $F(A,B,C) = ABC' + \underline{A'B'C} + A'BC + \underline{A'B'C'}$ ifadesini sadeleştirelim.

İfadedeki 2. ve 4. terimler $A'B'$ parantezine alınır;

$$\begin{aligned} F(A,B,C) &= ABC' + A'B'(C+C') + A'BC && A+A'=1 \text{ kuralından} \\ &= ABC' + \underline{A'B'} + \underline{A'}BC \\ &= ABC' + A'(\underline{B'+BC}) && A+A'B = A+B \text{ kuralından} \\ &= ABC' + A'(B'+C) \\ &= ABC' + A'B' + A'C \end{aligned}$$

Lojik İfadelerin Sadeleştirilmesi

Örnek: $F(A,B,C) = A.B + A'.C + B.C$ ifadesini sadeleştirelim

$B.C$ terimini $(A+A')$ ile genişletebiliriz;

$$\begin{aligned} F(A,B,C) &= A.B + A'.C + B.C.(A+A') = \underline{A.B} + \underline{A'.C} + \underline{A.B.C} + \underline{A'.B.C} \\ &= A.B.(1+C) + A'.C.(1+B) = A.B + A'.C \end{aligned}$$

Örnek: $F(A,B,C) = AB' + A(B+C)' + B(B+C)'$ ifadesini sadeleştirelim.

İfadedeki 2. ve 3. terim için DeMorgan kuralını uygularsak;

$$\begin{aligned} F(A,B,C) &= AB' + A(B'C') + \underline{B(B'C')} & B.B' &= 0' \text{ dır.} \\ &= \underline{AB'} + \underline{AB'}C' & AB' \text{ parantezine alınırsa;} \\ &= AB'(1+C') & 1+C' &= 1 \text{ olduğundan;} \\ &= AB' \end{aligned}$$

BOOL CEBRİ (Devamı)

❖ Standart Formlar (Canonical Forms)

- Çarpımlar Toplamı Formu

Standart Çarpımlar Toplamı

Minterm Kavramı

- Toplamlar Çarpımı Formu

Standart Toplamlar Çarpımı

Maxterm Kavramı

- Çarpımlar Toplamı Formu ile Toplamlar Çarpımı Formu Arasındaki Dönüşüm İşlemleri

Standart Formlar (Canonical Forms)

Lojik ifadeler, çarpımlar toplamı ya da toplamlar çarpımı formunda ifade edilebilir ve aralarında dönüşümleri yapılabilir. Bu formlar sayesinde hem lojik ifadeler standardize edilmiş olur hem de lojik ifadelerin değerlendirilmesi, sadeleştirilmesi ve gerçekleştirilmesi daha kolay ve sistematik hale gelir.

Çarpımlar Toplamı Formu: Çarpımlar toplamı formu, değişkenlerin çarpımlarından ve bu çarpımların toplamlarından oluşur. Değişkenlerin ise ya normal ya da tümleyen formları olur. Çarpım terimleri, değişkenlerin tamamını içermek zorunda değildir.

$ABC + A'B + BC'D'$ veya $A' + AB + BC'$ ifadeleri çarpımlar toplamı formundadır. Oysa birden fazla değişkenin tümleyenini içeren terimler varsa çarpımlar toplamı formundadır diyemeyiz; örneğin lojik ifade $(ABC)'$ terimini içeriyorsa...

Verilen lojik bir ifadeyi çarpımlar toplamı formuna dönüştürmek için Boole cebrinin kuralları kullanılır.

Standart Formlar

Örnek: $A'B(C+BD')$ lojik ifadesini çarpımlar toplamı formuna dönüştürebilmek için dağılma kuralını uygulayabiliriz;

$$A'B(C+BD') = A'BC + A'BD'$$

Standart Çarpımlar Toplamı: Çarpımlar toplamı formunda ifade edilen bir lojik ifadenin içerdiği terimler, tüm değişkenleri içermeyebilir. Çarpımlar toplamı olarak elde edilmiş bir lojik ifadeyi standart çarpımlar toplamı biçiminde yazmak istiyorsak, lojik ifadedeki her terimin, tüm değişkenleri içerecek şekilde düzenlenmesi gerekir.

Örnek: $F(A,B,C) = A.B' + A.C$ lojik ifadesini standart çarpımlar toplamı biçiminde elde etmek istersek, ifadedeki birinci terimi $(C+C')$ ile ikinci terimi de $(B+B')$ ile genişletmemiz gerekir.

$$\begin{aligned} AB' + AC &= AB'(C+C') + A(B+B')C = \underline{AB'C} + AB'C' + ABC + \underline{AB'C} \\ &= AB'C + AB'C' + ABC \end{aligned}$$

Standart Formlar

Örnek: $F(A,B,C) = A+BC+A'C$ lojik ifadesini pratik olarak standart forma getirmek için aşağıdaki gibi bir tablo hazırlanabilir.

A	BC	A'C
<u>ABC</u>	<u>ABC</u>	<u>ABC</u>
100	011	001
101	111	011
110		
111		

$$\begin{aligned} F(A,B,C) &= AB'C' + AB'C + ABC' + \underline{ABC} + \underline{A'BC} + \underline{ABC} + A'B'C + \underline{A'BC} \\ &= AB'C' + AB'C + ABC' + ABC + A'BC + A'B'C \end{aligned}$$

- Standart çarpımlar toplamı formu özellikle doğruluk tablosu oluşturmada ve ileride göreceğimiz Karnaugh haritası yardımıyla sadeleştirme işlemlerinde gerekli olacaktır.

Minterm Kavramı

Standart çarpımlar toplamı şeklinde ifade edilmiş bir lojik ifadedeki her terim *minterm* ismiyle anılır. Üç değişken içeren bir lojik ifade, 8 adet minterm (2^3) içerir. Bu mintermler, girişlerin sadece belli bir kombinasyonunda 1 değerini alırlar. Mintermler m_{indis} ile ifade edilirler. İndis, mintermin değerini 1 yapan değişken kombinasyonunun decimal değeridir.

Değişkenler			Mintermler	
A	B	C	Terim	Simges
0	0	0	$A'B'C'$	m_0
0	0	1	$A'B'C$	m_1
0	1	0	$A'BC'$	m_2
0	1	1	$A'BC$	m_3
1	0	0	$AB'C'$	m_4
1	0	1	$AB'C$	m_5
1	1	0	ABC'	m_6
1	1	1	ABC	m_7

Minterm Kavramı

Örnek: $F=A'B'C+A'BC+ABC$ standart formda verilmiş ifadenin doğruluk tablosunu oluşturmak istediğimizde devrenin çıkışı, bu çarpım terimlerinin değerlerini 1 yapan kombinasyonlarda 1 olacaktır.

Girişler			Çıkış(F)	Mintermler
A	B	C		
0	0	0	0	
0	0	1	1	$A'B'C$
0	1	0	0	
0	1	1	1	$A'BC$
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	1	ABC

$F(A,B,C)$ lojik ifadesini mintermler cinsinden ifade etmek istersek ' Σ ' sembolünü kullanırız:

$$F(A,B,C) = \Sigma (1,3,7) = m_1 + m_3 + m_7$$

Standart Formlar

Toplamlar Çarpımı Formu : Toplamlar çarpımı formu, değişkenlerin toplamlarından ve bu toplamların çarpımından oluşur.

$(A+B')(A'+B+C)$ veya $A(A'+B)(B+C')$ ifadeleri toplamlar çarpımı formundadır. Oysa birden fazla değişkenin tümleyenini içeren terimler varsa toplamlar çarpımı formundadır diyemeyiz; $(A+B+C)'$

Standart Toplamlar Çarpımı : Toplamlar çarpımı olarak elde edilmiş bir lojik ifadeyi, standart toplamlar çarpımı biçiminde yazmak istiyorsak, lojik ifadedeki her terimin, tüm değişkenleri içerecek şekilde düzenlenmesi gerekir.

Örnek: $F(A,B,C)=(A+B')(B+C)$ lojik ifadesini standart toplamlar çarpımı biçiminde elde etmek istersek,

$$\begin{aligned}(A+B')(B+C) &= (\underline{A+B'}+C.C')(A.A'+\underline{B+C}) \\ &= (A+B'+C).(A+B'+C').(A+B+C).(A'+B+C)\end{aligned}$$

Maxterm Kavramı

Standart toplamlar çarpımı şeklinde ifade edilmiş bir lojik ifadedeki her terim *maxterm* ismiyle anılır. Üç değişkene sahip bir lojik ifade 8 adet maxterm (2^3) içerir. Bu maxtermler girişlerin sadece belli bir kombinasyonunda 0 değerini alır. Maxtermler M_{indis} ile ifade edilirler. İndis, maxtermin değerini 0 yapan değişken kombinasyonunun decimal değeridir. Ayrıca maxtermler, mintermlerin değili olarak da düşünülebilir.

Değişkenler			Maxtermler		Mintermler	
A	B	C	Terim	Simge	Terim	Simge
0	0	0	$A+B+C$	M_0	$A'B'C'$	m_0
0	0	1	$A+B+C'$	M_1	$A'B'C$	m_1
0	1	0	$A+B'+C$	M_2	$A'BC'$	m_2
0	1	1	$A+B'+C'$	M_3	$A'BC$	m_3
1	0	0	$A'+B+C$	M_4	$AB'C'$	m_4
1	0	1	$A'+B+C'$	M_5	$AB'C$	m_5
1	1	0	$A'+B'+C$	M_6	ABC'	m_6
1	1	1	$A'+B'+C'$	M_7	ABC	m_7

Maxterm Kavramı

Örnek: $F(A,B,C)=(A+B+C).(A+B'+C).(A'+B+C).(A'+B+C').(A'+B'+C)$
standart formda verilmiş ifadenin doğruluk tablosunu oluşturmak istediğimizde $F(A,B,C)$ çıkışı, bu toplam terimlerinin değerlerini 0 yapan kombinasyonlarda 0 olacaktır. Diğer durumlar 1'dir.

Girişler			Çıkış(F)	Maxtermler
A	B	C		
0	0	0	0	$A+B+C$
0	0	1	1	
0	1	0	0	$A+B'+C$
0	1	1	1	
1	0	0	0	$A'+B+C$
1	0	1	0	$A'+B+C'$
1	1	0	0	$A'+B'+C$
1	1	1	1	

$F(A,B,C)$ lojik ifadesini maxtermler cinsinden ifade etmek istersek 'Π' sembolünü kullanırız:

$$F = \prod (0,2,4,5,6) = M_0.M_2.M_4.M_5.M_6$$

Çarpımlar Toplamı Formu ile Topamlar Çarpımı Formu Arasındaki Dönüşüm İşlemleri

Çarpım terimlerinin değerlerini 1 yapan kombinasyonlar bulunur, daha sonra bu kombinasyonların dışındaki kombinasyonlar tespit edilir ve elde edilen kombinasyonların değerini 0 yapan toplam terimleri bulunur.

Örnek: $F(A,B,C) = \sum(4,5,6,7)$ lojik ifadesini maxtermler cinsinden bulalım.

Girişler			F	F'
A	B	C		
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

F fonksiyonunun tümleyeni ,

$$F'(A,B,C) = \sum (0,1,2,3) = m_0 + m_1 + m_2 + m_3$$

F' fonksiyonunun bir daha tümleyeni alınır,

$$F(A,B,C) = (m_0 + m_1 + m_2 + m_3)' = m_0' \cdot m_1' \cdot m_2' \cdot m_3'$$

$$= M_0 \cdot M_1 \cdot M_2 \cdot M_3$$

$$= \prod (0,1,2,3) \text{ olarak bulunur.}$$

Özetle, çarpımlar toplamı formunun içerdiği terimlerin dışında kalan kombinasyonlardan, toplamlar çarpımı formu elde edilir. Bu ifadenin tersi de geçerlidir.

Dönüşüm İşlemleri

Örnek: $F(A,B,C) = AB'C + A'BC + AB'C' + ABC + A'B'C$ standart çarpımlar toplamı biçiminde verilmiş lojik ifadeyi standart toplamlar çarpımı formuna dönüştürelim.

Çarpım terimlerini 1 yapan kombinasyonlar; 101, 011, 100, 111, 001 dir. Bu kombinasyonların dışındakiler; 000, 010, 110 dir. O halde toplamaları 0 yapan terimler $(A+B+C)$, $(A+B'+C)$ ve $(A'+B'+C)$ olur. Bu üç terim çarpıldığında toplamaların çarpımı formu elde edilmiş olur;

$$F(A,B,C) = (A+B+C).(A+B'+C).(A'+B'+C)$$

Ayrıca $F(A,B,C)$ fonksiyonu mintermler cinsinden $\Sigma(1,3,4,5,7)$ şeklinde de ifade edilebilir. Bu mintermler dışındaki kombinasyonlar, maxtermleri oluşturur.

$$F(A,B,C) = \Sigma (1,3,4,5,7) = \Pi (0,2,6) \text{ yazılabilir.}$$

Sadeleştirme İşlemleri

Örnek: Aşağıda verilen doğruluk tablosuna göre F fonksiyonunu minterm ve maxterm yöntemlerini kullanarak sadeleştirelim.

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

İlk olarak F fonksiyonunu mintermler cinsinden yazarak sadeleştirelim;

$$\begin{aligned} F(A,B,C) &= \sum(0,2,3,5,7) = \underline{A'B'C'} + \underline{A'BC'} + A'BC + \underline{AB'C} + \underline{ABC} \\ &= A'C'(B'+B) + A'BC + AC(B'+B) = A'C' + \underline{A'BC} + \underline{AC} \\ &= A'C' + C(A'B+A) = A'C' + C(A+B) = A'C' + AC + BC \end{aligned}$$

Ya da $A'C' + A'BC + AC$ terimini C değil de A' parantezine alırsak,
 $A'(C' + BC) + AC = A'(B + C') + AC = A'C' + AC + A'B$ elde edilir.

Her iki ifade de birbirine lojik olarak eşdeğerdir.

Örnek: (Devamı)

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Şimdi de F fonksiyonunu maxtermler cinsinden yazarak sadeleştirelim;

$$\begin{aligned} F(A,B,C) &= \prod (1,4,6) = (A+B+C').(A'+B+C).(A'+B'+C) \\ &= (AA'+AB+AC+A'B+BB+BC+A'C'+BC'+C'C)(A'+B'+C) \\ &= (\underline{AB}+AC+\underline{A'B}+\underline{B}+\underline{BC}+A'C'+\underline{BC}')(A'+B'+C) \\ &= (B+AC+A'C')(A'+B'+C) \\ &= A'B+\underline{BB'}+BC+\underline{A'AC}+AB'C+ACC+A'A'C'+A'B'C'+\underline{A'C'C} \\ &= A'B+BC+\underline{AB'C}+AC+A'C'+\underline{A'B'C'} \\ &= A'B+B'(\underline{AC+A'C'})+\underline{AC+A'C'} \\ &= A'B+AC+A'C' \end{aligned}$$

Toplamlar çarpımı formunda farklı tip indirgemeler yapılarak diğer çözüm de bulunabilirdi.

Sonuç olarak, doğruluk tablosundaki 1'lerden yola çıkarak mintermler cinsinden ya da 0'lardan yola çıkarak maxtermler cinsinden F fonksiyonunu ifade edebilmekteyiz. Her iki form için sadeleştirme yapıldığında ise elde edilen lojik ifadeler birbirine denk olmaktadır.

LOJİK KAPILAR (GATES)

- ❖ ‘Değil’ veya ‘Tümleme’ Kapısı (NOT Gate)
- ❖ ‘Ve’ Kapısı (AND Gate)
- ❖ ‘Veya’ Kapısı (OR Gate)
- ❖ ‘Vedeğil’ Kapısı (NAND Gate)
- ❖ ‘Veyadeğil’ Kapısı (NOR Gate)
- ❖ ‘Özelveya’ Kapısı (EXOR Gate)
- ❖ ‘Özelveyadeğil’ Kapısı (EXNOR Gate)
- ❖ Lojik İfadelerden Lojik Devrelerin Elde Edilmesi

LOJİK KAPILAR

Lojik devrelerin en temel elemanı, lojik kapılardır. Kapılar, lojik değişkenlerin değerlerini (1 veya 0) giriş olarak alırlar, bu değerler üzerinde işlem yaparlar ve lojik çıkış üretirler.

Kapılar transistör, diyot, direnç, kondansatör gibi devre elemanlarından oluşur. Düşük enerji tüketimi, az yer kaplaması, ek bağlantı içermemesi, ekonomik olması gibi nedenlerle entegre devre olarak piyasadan temin edilebilirler.

Temel olarak 7 kapı vardır bunlar;

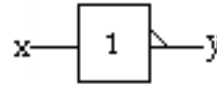
‘ve’ (AND), ‘veya’ (OR), ‘değil’ (NOT),

‘vedeğil’ (NAND), ‘veyadeğil’ (NOR),

‘özelveya’ (EXOR) ve ‘özelveyadeğil’ (EXNOR) kapılarıdır.

‘Değil’ veya ‘Tümleme’ Kapısı (NOT Gate)

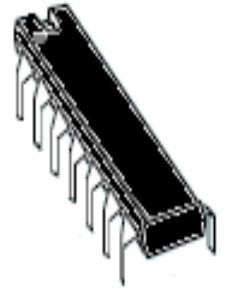
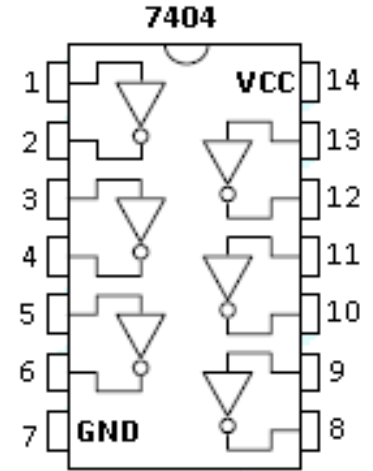
Girişine gelen lojik sinyalleri tersler. Yani, girişine lojik 1 değeri geldiyse çıkışı 0, 0 değeri geldiyse çıkışı 1 olur. Aşağıdaki sembollerle gösterilir;



Değil veya tümleyen kapısının doğruluk tablosu;

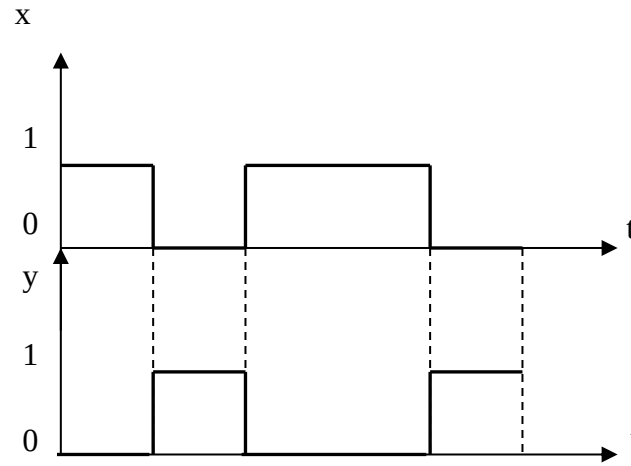
x	y
0	1
1	0

lojik ifadesi $y = x'$ veya $y = \bar{x}$ olarak ifade edilir.



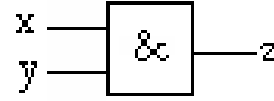
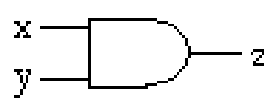
Örnek:

Değil kapısının girişine x sinyali uygulandığında, çıkışındaki sinyal (y) aşağıdaki gibidir;



‘Ve’ Kapısı (AND Gate)

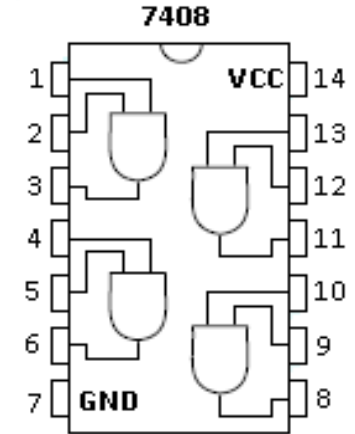
Mantıksal çarpma olarak bilinir. Bir AND kapısı iki veya daha fazla girişe sahip olabilir, tek çıkışı vardır. Aşağıdaki sembollerle gösterilir;



AND kapısının doğruluk tablosu;

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

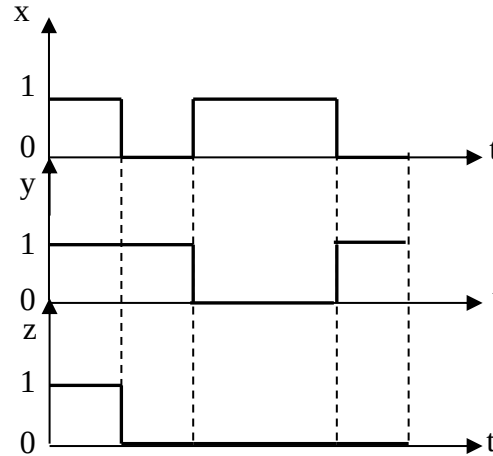
lojik ifadesi $z = x.y$ veya $z = xy$



AND kapısının bütün girişleri 1 iken çıkışı lojik 1 seviyesindedir, herhangi bir giriş lojik 0 ise çıkış 0'dır.

Örnek:

2 girişli bir AND kapısının girişlerine x ve y sinyalleri uygulandığında çıkışındaki sinyal (z) aşağıdaki gibidir;

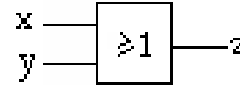


Not: AND işlemi, ikili çarpım ile aynı işleve sahiptir.

Yani, $0.0 = 0$, $0.1 = 0$, $1.0 = 0$, $1.1 = 1$

‘Veya’ Kapısı (OR Gate)

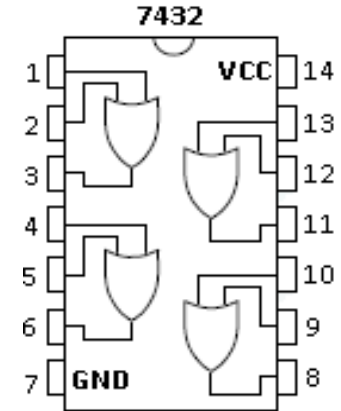
OR kapısı mantıksal toplama olarak da bilinir. Aşağıdaki sembollerle gösterilir;



OR kapısının doğruluk tablosu;

x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

lojik ifadesi $z = x + y$

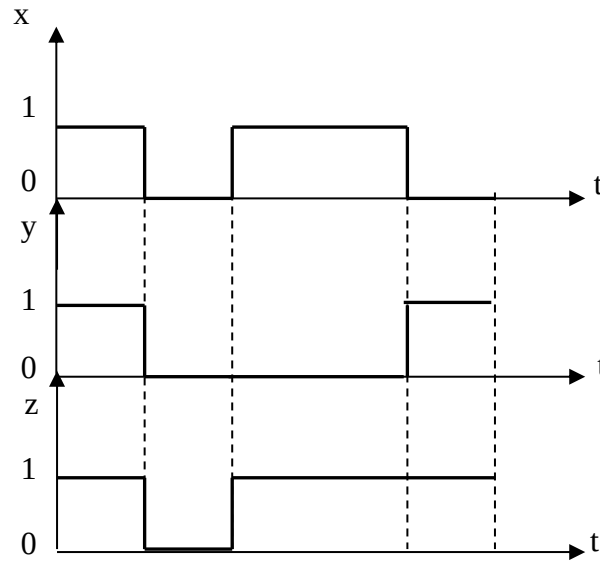


İkili toplama işlemine benzerdir ancak her iki giriş de 1 olduğunda sonuç 1'dir, oysa ikili toplamada toplam 0, elde 1'dir.

OR kapısının girişlerinden en azından biri lojik 1 seviyesine sahipse çıkış 1'dir.

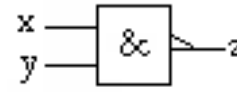
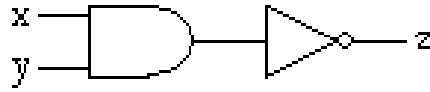
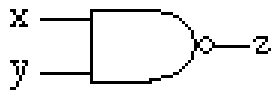
Örnek:

OR kapısının girişlerine x ve y sinyalleri uygulandığında çıkışı (z) aşağıdaki gibidir;



‘Vedeğil’ Kapısı (NAND Gate)

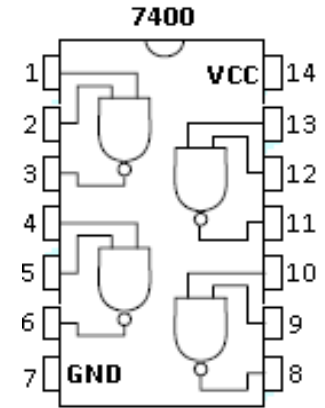
Üniversal kapı olarak kullanılırlar. Yani NAND kapısı kullanılarak AND, OR veya NOT kapıları elde edilebilir. AND kapısının çıkışına NOT kapısının bağlanmış hali olarak da düşünülebilir. Aşağıdaki sembollerle gösterilir;



NAND kapısının doğruluk tablosu;

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

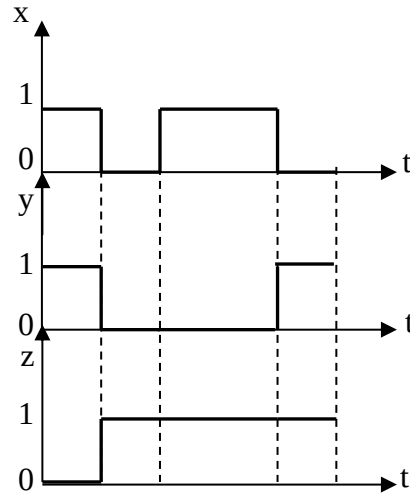
Lojik ifadesi $z = (x.y)'$ veya $z = \overline{x.y}$



NAND kapısının tüm girişleri 1 iken çıkışı 0, herhangi bir girişi 0 değerini alırsa çıkışı 1 olur.

Örnek:

NAND kapısının girişlerine x ve y sinyalleri uygulandığında çıkışındaki sinyal (z) aşağıdaki gibidir;

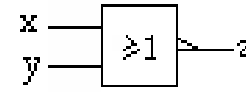
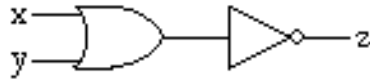


OR işlemiyle NAND işlemi arasında bir benzerlik olduğu söylenebilir. OR kapısında bir ya da daha fazla sayıda giriş 1 iken çıkış 1'dir, NAND'de ise bir veya daha fazla sayıda giriş 0 iken çıkış 1'dir.



‘Veyadeğil’ Kapısı (NOR Gate)

NAND kapısı gibi üniversal bir kapıdır. OR kapısının çıkışına NOT kapısının bağlanmış hali olarak da düşünülebilir. Aşağıdaki sembollerle gösterilir;



NOR kapısının doğruluk tablosu;

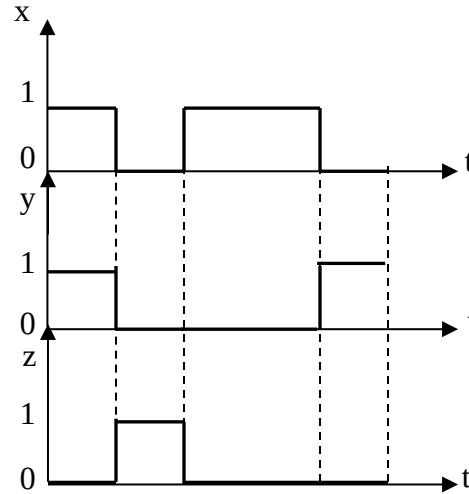
x	y	z
0	0	1
0	1	0
1	0	0
1	1	0

Lojik ifadesi $z = (x + y)'$ veya $z = \overline{x + y}$

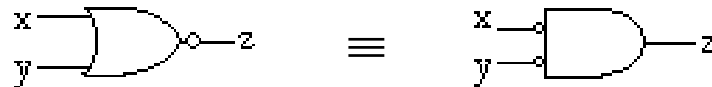
NOR kapısının tüm girişleri 0 ise çıkışı 1, aksi halde çıkışı 0'dır.

Örnek:

NOR kapısının girişlerine x ve y sinyalleri uygulandığında çıkışındaki sinyal (z) aşağıdaki gibidir;



AND işlemiyle NOR işlemi arasında bir benzerlik olduğu söylenebilir. AND'te tüm girişler 1 değerini aldığı anda çıkışı 1'dir, NOR'da ise tüm girişler 0 değerini aldığı anda çıkış 1'dir.



‘Özelveya’ Kapısı (EXOR Gate)

EXOR kapısı, AND ve OR kapıları kullanılarak elde edilebilir. Farklılık kapısı olarak da anılır. Yerine getirdiği işlevin öneminden dolayı temel lojik kapılar gibi kullanılırlar. Aşağıdaki sembollerle gösterilir;



EXOR kapısının doğruluk tablosu;

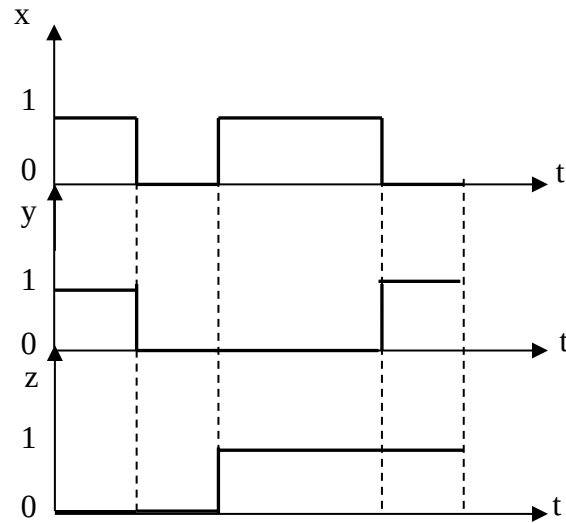
x	y	z
0	0	0
0	1	1
1	0	1
1	1	0

lojik ifadesi $z = x \oplus y$ veya $z = x'y + xy'$

- ❖ Çok sayıda girişe sahip EXOR kapısının çıkışı, girişlerindeki 1'lerin tek sayıda olması durumunda lojik 1 seviyesinde olacaktır.
- ❖ Birleşme özelliği vardır: $a \oplus (b \oplus c) = (a \oplus b) \oplus c = a \oplus b \oplus c$

Örnek:

EXOR kapısının girişlerine x ve y sinyalleri uygulandığında çıkışındaki sinyal (z) aşağıdaki gibidir;



‘Özelveyadeğil’ Kapısı (EXNOR Gate)

EXNOR kapısı da EXOR kapısı gibi, AND ve OR kapıları kullanılarak elde edilebilir. Eşitlik kapısı olarak da anılır. Yerine getirdiği işlevin öneminden dolayı temel lojik kapılar gibi kullanılırlar. Aşağıdaki sembollerle gösterilir;



EXNOR kapısının doğruluk tablosu;

x	y	z
0	0	1
0	1	0
1	0	0
1	1	1

lojik ifadesi $z = x \otimes y$ veya $z = x.y + x'.y'$

- ❖ EXNOR kapısı, EXOR kapısının değilidir. Yani $z = (x \oplus y)'$
- ❖ Çok sayıda girişe sahip EXNOR kapısının çıkışı girişlerindeki 0'ların çift sayıda olması durumunda 1'dir.
- ❖ Birleşme özelliği vardır: $a \otimes (b \otimes c) = (a \otimes b) \otimes c = a \otimes b \otimes c$

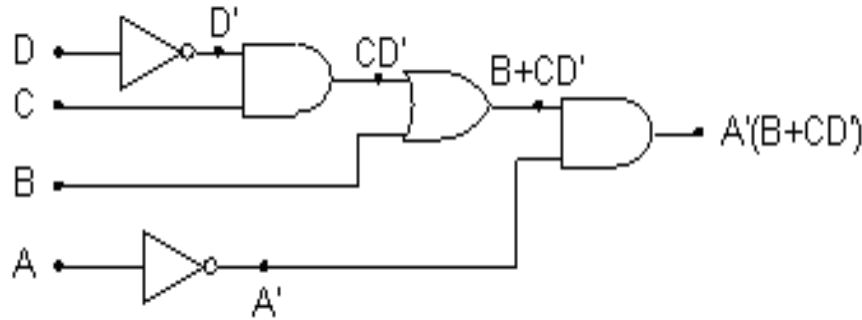
Lojik İfadelerden Lojik Devrelerin Elde Edilmesi

Lojik ifadelerden lojik devrelerin elde edilmesi işleminde, önce lojik ifadedeki değişkenlerin deęilleri alınması gerekiyorsa NOT kapıları kullanılarak bu işlem yapılır. Daha sonra çarpım ifadelerine bakılır ve bu çarpımlar AND kapıları kullanılarak gerçekleştirilir. Lojik ifadedeki toplanlar için de OR kapıları kullanılır. AND ve OR işlemlerinden sonra tümleyen işlemi gerekiyorsa yine NOT kapısı kullanılır ve elde edilen devre parçaları bir araya getirilir. Önemli olan şey, lojik ifadedeki parantezler ve çarpım işlemlerinin toplam işlemlerine göre öncelikli olduğudur.

Örnek:

$F = A' \cdot (B + C.D')$ lojik ifadesini temel kapılar kullanarak gerçekleştirelim.

İlk olarak parantez içindeki terimlere bakılır; bir çarpma ve bir de toplama işlemi mevcuttur. Önce çarpma işlemi AND kapısı kullanılarak gerçekleştirilir (D nin tümleyenini aldıktan sonra) daha sonra da toplama işlemi OR kapısı kullanılarak gerçekleştirilir. Parantezin içindeki ifade elde edildikten sonra parantezin dışındaki ifadelere bakılır. Parantezin dışındaki A'nın tümleyeni, NOT kapısı kullanılarak elde edilir. Daha sonra da elde edilen devre parçaları AND kapısı kullanılarak bir araya getirilir.



Verilen bir lojik devrenin lojik ifadesini elde etmek için de, devrenin çıkışını oluşturan kapıya kadar olan tüm kapıların çıkışları bulunur ve bu çıkışlar bir araya getirilir.

Örnekler

- ❖ 2'ye Tümlleyen ile Aritmetik İşlemler
- ❖ BCD Sisteminde Aritmetik İşlemler
- ❖ Bool Cebri Kurallarının Kullanımı
- ❖ Devrelerden Lojik Denklemlerin Elde Edilmesi
- ❖ Lojik Denklemlerden Yola Çıkararak Devre Gerçekleştirimi
- ❖ Devrelerin Giriş-Çıkış İlişkilerinin Zaman Domeninde İncelenmesi
- ❖ Sözel Olarak İfade Edilmiş Problemlerin Doğruluk Tablolarının Oluşturulması

Örnek:

Aşağıdaki işlemleri ikiye tümleyen formunda yapalım ve taşma durumu olup olmadığına bakalım.

$$\begin{array}{r} -117 \qquad 117 \\ + 89 \qquad - (-89) \end{array}$$

$\begin{array}{r} -117 \\ + 89 \\ \hline -28 \end{array}$	$\begin{array}{r} 10001011 \\ + 01011001 \\ \hline 11100100 \\ \textbf{Overflow yoktur} \end{array}$
$\begin{array}{r} 117 \\ - (-89) \\ \hline 206 \end{array}$	$\begin{array}{r} 01110101 \\ + 01011001 \\ \hline 11001110 \\ \textbf{Overflow vardır} \end{array}$

Örnek:

$(387)_{10} + (439)_{10}$ toplamını BCD sisteminde yapalım.

$$\begin{array}{r} \overset{1}{0}011 \overset{1}{1}000 0111 \\ + 0100 0011 1001 \\ \hline 1000 1100 \overset{1}{1}0000 \\ + 0110 0110 \\ \hline 1000 \overset{1}{1}0010 0110 \end{array}$$

$$(1000 \ 0010 \ 0110)_{\text{BCD}} = 826$$

Örnek:

32 bitlik formatla verilen kayan noktalı 01000000101101100000000000000000 sayısının ikilik sistemdeki karşılığını bulalım.

İşaret	Üst kısmı	Kesir Kısmı
0	10000001	011011000000000000000000

- İşaret biti 0 olduğundan sayımız pozitiftir.
- Üst kısmından bias değerini çıkarırsak ; $129 - 127 = 2$.
- O halde sayımızın 2'lik sistemdeki karşılığı $+ 1.011011 \times 2^2 = 101.1011$

Örnek:

$ab' + bc' + a'c = a'b + b'c + ac'$ eşitliğinin doğruluğunu cebirsel olarak gösterelim.

Eşitliğin sol yanındaki terimleri genişletirsek;

$$\begin{aligned} ab'(c+c') + (a+a')bc' + a'(b+b')c &= ab'c + \underline{ab'c'} + \underline{abc'} + \underline{a'bc'} + \underline{a'bc} + a'b'c \\ &= b'c + ac' + a'b \text{ bulunur.} \end{aligned}$$

Örnek: $ab + a'c + bcd = ab + a'c$ eşitliğinin doğruluğunu cebirsel olarak gösterelim.

$$ab + a'c + (a+a')bcd = \underline{ab} + a'c + \underline{abcd} + a'bcd = ab(1+cd) + a'c(1+bd) = ab + a'c$$

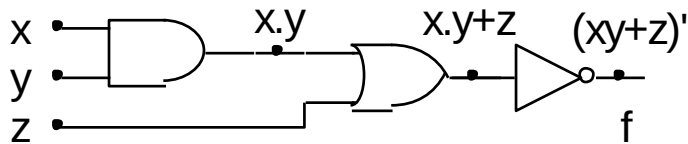
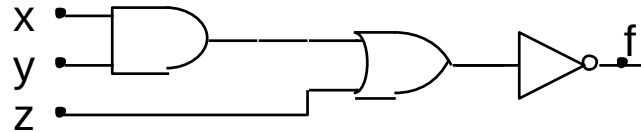
$ab + a'c + bc = ab + a'c$ olduğunu daha önceden ispat etmiştik.

❖ Şöyle bir kural geliştirilebilir: 3 terim olduğunda, 2 terimde olan değişkenlerden birer tanesi 3. terimde varsa ve 2 terim içerisinde de başka bir değişkenin kendisi ve tümleyeni bulunuyorsa, 3. terim sadeleşir.

$$\underline{a'b} + b'\underline{d} + \underline{a'c'd} = a'b + b'd \text{ dir.}$$

Örnek:

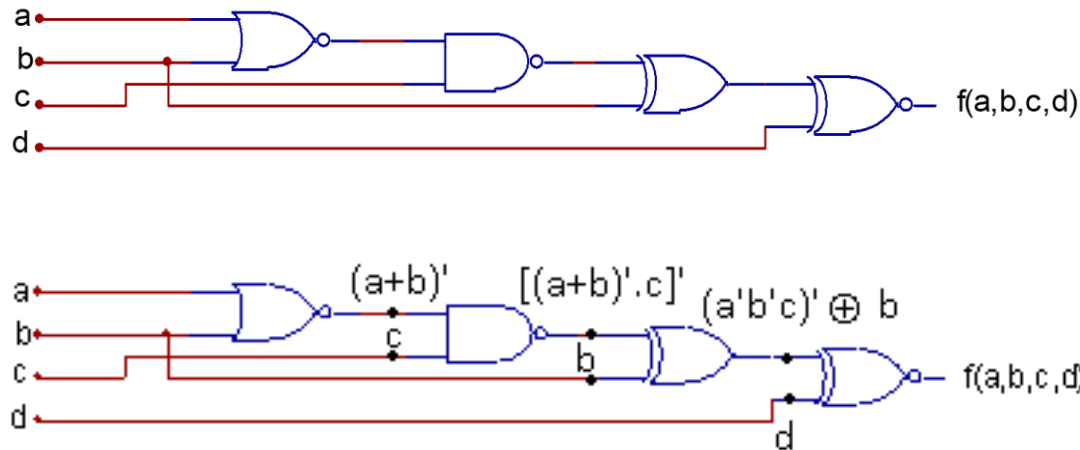
Aşağıda verilen devrenin doğruluk tablosunu oluşturalım.



x	y	z	x.y	x.y+z	(x.y+z)'
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	0	1	0
1	0	0	0	0	1
1	0	1	0	1	0
1	1	0	1	1	0
1	1	1	1	1	0

Örnek:

Aşağıdaki devrenin çıkışını mintermler cinsinden bulalım.



Exor kapısının girişlerinden biri $((a+b)'.c)' = (a'b'c)'$, diğeri b

Exnor kapısının girişlerinden biri $(a'b'c).b+(a'b'c)'b' = (a+b+c').b'$, diğeri d

Bu işlem yapıldığında $f = (ab'+b'c')d+(ab'+b'c')'d'$

$$\begin{aligned} f &= ab'd+b'c'd+(a'+b)(b+c)d' = ab'd+b'c'd+\underline{a'bd'}+a'cd'+\underline{bd'}+\underline{bcd'} \\ &= ab'd+b'c'd+bd'+a'cd' \end{aligned}$$

Örnek: (Devamı)

$$f = ab'd + b'c'd + bd' + a'cd'$$

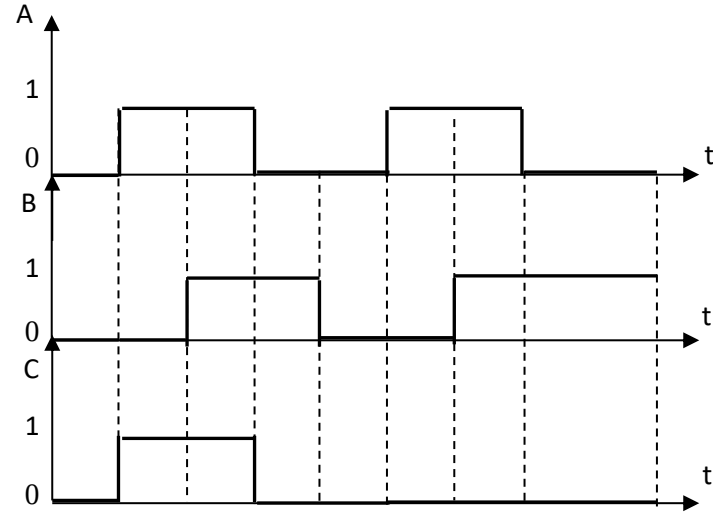
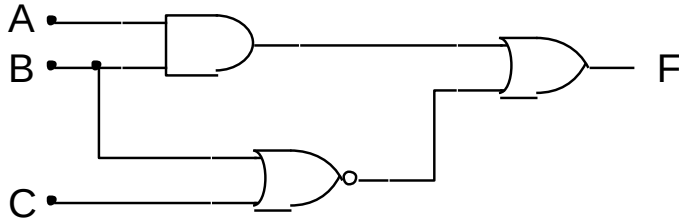
Mintermler hesaplandığında;

<u>ab'd</u>	<u>b'c'd</u>	<u>bd'</u>	<u>a'cd'</u>
abcd	abcd	abcd	abcd
1001	0001	0100	0010
1011	1001	0110	0110
		1100	
		1110	

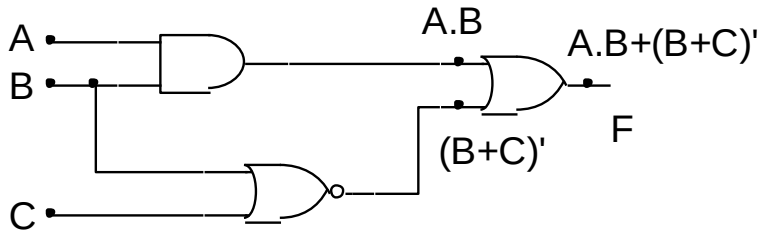
$$f(a,b,c,d) = \sum(1,2,4,6,9,11,12,14)$$

Örnek:

Aşağıda, 3 girişli ve 1 çıkışlı bir kombinasyonel devrenin devre şeması ve girişlerine uygulanan sinyaller zamana bağlı olarak verildiğine göre, devrenin çıkışının dalga şeklini çizelim.



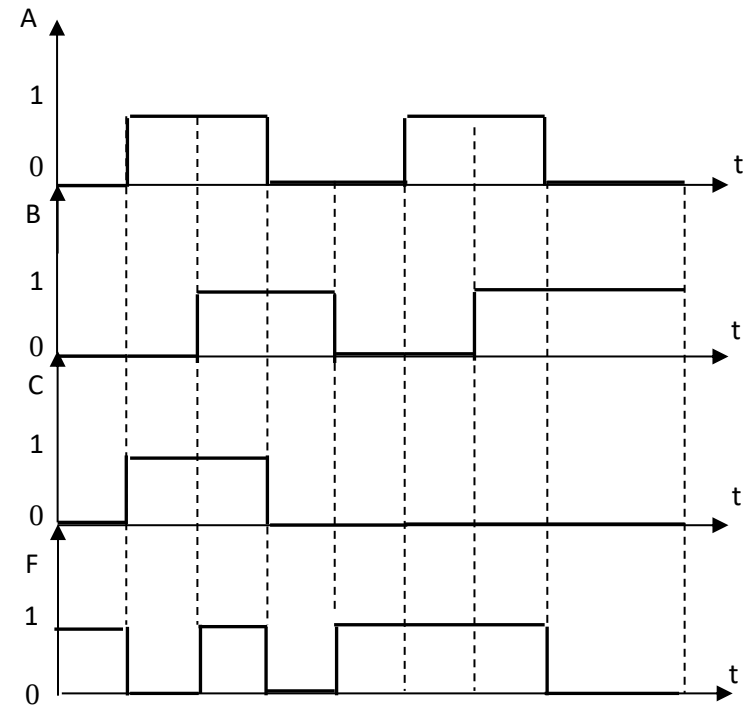
Örnek: (Devamı)



$$F = AB + (B+C)' = AB + B'C'$$

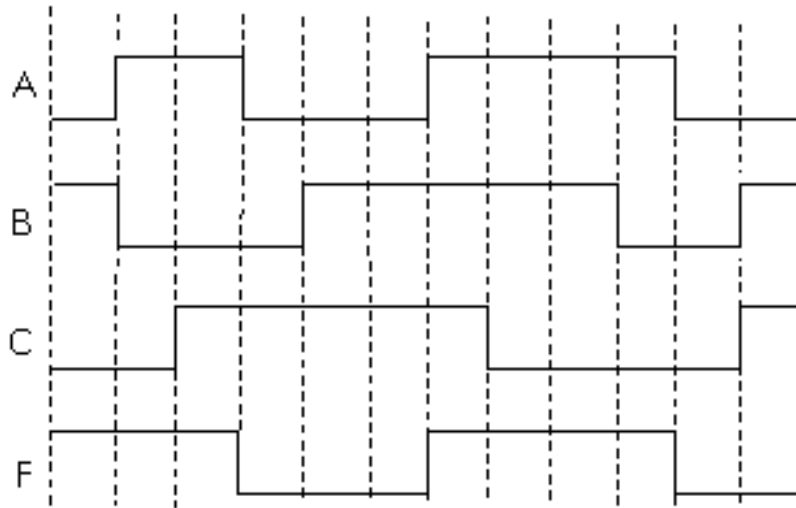
A ve B nin 1 olduğu ya da

B ve C nin 0 olduğu yerlerde çıkış 1 dir.



Örnek:

Lojik bir devrenin çıkışı olan F fonksiyonu A, B ve C girişlerine bağlıdır. Devrenin giriş-çıkış ilişkisi aşağıdaki gibidir. İndirgenmiş F fonksiyonunu çarpımların toplamı şeklinde bulalım.



A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = A + B.C'$$

Örnek:

3 girişe (a,b,c) ve 1 çıkışa (z) sahip bir kombinasyonel devrenin, girişlerinin ikili değeri 3'ün altındaysa çıkışının 1 olması isteniyor. Buna göre doğruluk tablosunu ve çıkışın en sade halini bulalım.

a	b	c	z
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

$$\begin{aligned}z &= a'b'c' + a'b'c + a'bc' = a'b'(c' + c) + a'bc' \\ &= a'b' + a'bc' = a'(b' + bc') = a'(b' + c') = a'b' + a'c'\end{aligned}$$

Örnek:

3 girişe (a,b,c) sahip bir kombinasyonel devrenin girişlerindeki 1'lerin 0'lardan fazla olması durumunda çıkışının (z) 1 olması isteniyor (**Çoğunluk fonksiyonu**). Buna göre doğruluk tablosunu ve çıkışın en sade halini bulalım.

a	b	c	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$\begin{aligned}z &= a'bc + ab'c + abc' + abc = a'bc + ab'c + ab(c + c') \\&= a'bc + ab'c + ab = a'bc + a(b'c + b) = a'bc + a(b + c) \\&= a'bc + ab + ac = b(a'c + a) + ac = b(a + c) + ac = ab + bc + ac\end{aligned}$$

Örnek:

2 girişe (A ve B) sahip bir kombinasyonel devrenin, girişlerin ikili değerinin (AB_2) 4 katının 3 fazlasını bulması isteniyor. Buna göre doğruluk tablosunu oluşturalım ve çıkışların lojik ifadesini bulalım.

Girişlerden gelen bilgi maksimum 11_2 olacağından, bu sayının 4 katının 3 fazlası 1111_2 olacaktır. O halde 4 çıkışa ihtiyacımız vardır.

A	B	F3	F2	F1	F0
0	0	0	0	1	1
0	1	0	1	1	1
1	0	1	0	1	1
1	1	1	1	1	1

$$F3 = AB' + AB = A, F2 = A'B + AB = B, F1 = 1, F0 = 1$$

Örnek:

Bir bilgisayar ağında kullanılan yönlendirici (router), çok sayıda bilgisayarı birbirine bağlar ve bu bilgisayarlar arasındaki mesaj alışverişine olanak tanır. Fakat iki veya daha fazla bilgisayar eşzamanlı olarak mesaj göndermek istediğinde ağda bir çarpışma olur ve mesajın tekrardan gönderilmesini gerektirir. 3 bilgisayarı birbirine bağlayan router için çarpışma durumunu algılayan bir dedektör tasarlanması istenmektedir. Devrenin 3 girişi vardır (x,y,z) ve ilgili bilgisayar mesaj gönderdiğinde 1 olmaktadır.

x	y	z	d
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$d = \underline{x'yz} + xy'z + xyz' + \underline{xyz} = yz + x(y'z + yz') = yz + x(y \oplus z)$$

Örnek:

Bir sınavda 3 zorunlu 2 de seçmeli soru vardır. Öğrencinin sınavdan başarılı olabilmesi için 3 zorunlu soruya veya 2 zorunlu ve 2 seçmeli soruya yanıt vermesi gerekmektedir. Geçme durumunu ifade eden lojik ifadeyi (G) bulalım.

(Sorular: z_1, z_2, z_3, s_1, s_2 . Soruların yapılması durumunda bu değişkenler 1 değerini almaktadır.)

Direkt olarak lojik ifadeyi yazabiliriz:

$$G = z_1.z_2.z_3 + \textcolor{red}{z_1}.\textcolor{red}{z_2}.s_1.s_2 + \textcolor{red}{z_1}.\textcolor{red}{z_3}.s_1.s_2 + \textcolor{red}{z_2}.\textcolor{red}{z_3}.s_1.s_2$$

Örnek:

Bir arabanın sesli ikaz sisteminde 3 sensör vardır. D sensörü tüm kapılar kapalıysa 0, herhangi bir kapı açıksa 1 üretmektedir. G sensörü motor çalışmıyorken 0, çalışırken 1 üretmektedir. L sensörü ışıklar kapalıysa 0, açıksa 1 üretmektedir.

İkaz devresinin çıkışı (Y) aşağıdaki koşullarda 1 olmaktadır:

1. Motor çalışmıyorken ışıklar açıksa,
2. Motor çalışırken herhangi bir kapı açıksa

Doğruluk tablosunu oluşturalım ve çıkışı mintermler cinsinden ifade edelim.

D	G	L	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$Y = \sum (1, 5, 6, 7) = DG + G'L$$

Örnek:

Oda sıcaklığının kontrolü için 3 giriş (A,B,T) ve 2 çıkış (I,S) içeren bir sıcaklık kontrolörü tasarlanmak isteniyor.

A girişi, kontrolün otomatik (lojik 1 ise) ya da manuel (lojik 0 ise) olarak seçimi için,

B girişi, ısıtma mı (lojik 1) yoksa soğutma mı (lojik 0) yapılmak istendiğini seçmek için,

T girişi ise, oda sıcaklığı istenen değerin üstünde olduğunda 1, altında olduğunda ise 0 değerini üreten sensörden gelen bilgi için kullanılmaktadır.

I çıkışı, ısıtma sistemini devreye almak (lojik 1) veya devreden çıkarmak (lojik 0) için,

S çıkışı ise soğutma sistemini devreye almak (lojik 1) veya devreden çıkarmak (lojik 0) için kullanılmaktadır.

Örnek: (Devamı)

Sıcaklık kontrolörünün davranışının aşağıdaki gibi olması isteniyor:

Otomatik Kontrol (A=1 ise)	Manuel Kontrol (A=0 ise)
1. Kullanıcı B=1 yapmışsa ve oda sıcaklığı istenen değerin altındaysa ısıtıcı devreye alınacak. Oda ısınınca ısıtıcı devreden çıkarılacak.	1. Kullanıcı B=1 yapmışsa ısıtıcı devrede olacak
2. Kullanıcı B=0 yapmışsa ve oda sıcaklığı istenen değerin üstündeyse soğutucu devreye alınacak. Oda soğuyunca soğutucu devreden çıkarılacak.	2. Kullanıcı B=0 yapmışsa soğutucu devrede olacak.

Buna göre doğruluk tablosu;

A	B	T	S	I	Açıklama
0	0	0	1	0	Manuel 2.maddeden
0	0	1	1	0	Manuel 2.maddeden
0	1	0	0	1	Manuel 1.maddeden
0	1	1	0	1	Manuel 1.maddeden
1	0	0	0	0	Otomatik 2.maddeden
1	0	1	1	0	Otomatik 2.maddeden
1	1	0	0	1	Otomatik 1.maddeden
1	1	1	0	0	Otomatik 1.maddeden

$$S(A,B,T) = \sum(0,1,5)$$

$$I(A,B,T) = \sum(2,3,6)$$

KARNAUGH HARİTALARI (Karnaugh Maps)

- ❖ 2, 3, 4 ve 5 Değişkenli Karnaugh Haritaları
- ❖ Komşuluk Kavramı
- ❖ Standart Çarpımlar Toplamı Biçimindeki İfadelerin Haritalanması
- ❖ Standart Çarpımların Toplamı Biçimindeki İfadelerin Sadeleştirilmesi
- ❖ Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi
- ❖ Standart Toplamlar Çarpımı Biçimindeki İfadelerin Haritalanması
- ❖ Standart Toplamlar Çarpımı Biçimindeki İfadelerin Karnaugh ile Sadeleştirilmesi
- ❖ Önemsiz “Don’t Care” Durumlar

Karnaugh Haritaları

Çarpımlar toplamı ya da toplamlar çarpımı formundaki lojik ifadelerin sadeleştirilmesine yönelik sistematik bir yaklaşımdır.

Karnaugh haritası giriş değişkenlerinin tüm kombinasyonlarını ve çıkışları gösterdiğinden doğruluk tablosuna benzer yapısı vardır. Doğruluk tablosunda giriş değişkenlerinin ikili değerleri satır ve sütun yapısında gösterilirken harita yönteminde bir matris formundaki hücrelere yerleştirilir. Bu hücreler sayesinde grupta yapılarak sadeleştirme sağlanır.



Maurice Karnaugh

Karnaugh Haritaları

Karnaugh haritası özellikle az değişkene sahip lojik ifadeleri sadeleştirmek için uygundur. Çok değişkene sahip lojik ifadelerde Quine McCluskey tablo yöntemi kullanılır.

Karnaugh haritasındaki hücre sayısı, doğruluk tablosundaki gibi giriş değişkenlerinin mümkün olan kombinasyonları kadardır.

İki ve Üç Değişkenli Karnaugh Haritası

İki değişkenli Karnaugh haritası, $2^2 = 4$ hücreye sahiptir. Her bir hücre, bir mintermi ya da maxtermi gösterir.

A \ B	0	1
0	$A'B'$	$A'B$
1	AB'	AB

A \ B	0	1
0	m_0	m_1
1	m_2	m_3

B \ A	0	1
0	m_0	m_2
1	m_1	m_3

Üç değişkenli Karnaugh haritası, $2^3 = 8$ hücreye sahiptir. 2×4 ya da 4×2 matris yapısında hücreler ile gösterilebilir.

AB \ C	0	1
00	$A'B'C'$	$A'B'C$
01	$A'BC'$	$A'BC$
11	ABC'	ABC
10	$AB'C'$	$AB'C$

AB \ C	00	01	11	10
0	$A'B'C'$	$A'BC'$	ABC'	$AB'C'$
1	$A'B'C$	$A'BC$	ABC	$AB'C$

AB değişkenleri 00 – 01 – 11 – 10 sırasında değer almışlardır. Bunun sebebi, 1 bit değişim esas alınarak haritanın doldurulmasıdır.

Dört Değişkenli Karnaugh Haritası

$2^4 = 16$ hücreye sahiptir. 4×4 matris yapısında hücreler ile gösterilir.

CD \ AB	00	01	11	10
00	$A'B'C'D'$	$A'B'C'D$	$A'B'CD$	$A'B'CD'$
01	$A'BC'D'$	$A'BC'D$	$A'BCD$	$A'BCD'$
11	$ABC'D'$	$ABC'D$	$ABCD$	$ABCD'$
10	$AB'C'D'$	$AB'C'D$	$AB'CD$	$AB'CD'$

AB \ CD	00	01	11	10
00	$A'B'C'D'$	$A'BC'D'$	$ABC'D'$	$AB'C'D'$
01	$A'B'C'D$	$A'BC'D$	$ABC'D$	$AB'C'D$
11	$A'B'CD$	$A'BCD$	$ABCD$	$AB'CD$
10	$A'B'CD'$	$A'BCD'$	$ABCD'$	$AB'CD'$

Beş Değişkenli Karnaugh Haritası

$2^5=32$ hücreden oluşur. 5 değişkenli harita oluşturulurken iki adet 4 değişkenli harita olarak düşünülebilir, 5. değişken ayrıca ele alınır.

$F(A,B,C,D,E)$ için;

A=0									
BC \ DE		00	01	11	10				
00	0	4	12	8					
01	1	5	13	9					
11	3	7	15	11					
10	2	6	14	10					

A=1									
BC \ DE		00	01	11	10				
00	16	20	28	24					
01	17	21	29	25					
11	19	23	31	27					
10	18	22	30	26					

5 değişkenli Karnaugh haritası aşağıdaki gibi de düzenlenebilir:

CDE \ AB		000	001	011	010	110	111	101	100
00									
01									
11									
10									

Komşuluk Kavramı

Karnaugh haritasındaki hücrelerin komşu olabilmesi için sadece 1 değişkenin değerinin değişiklik göstermesi gerekir. Örneğin 3 değişkenli bir haritada, değişkenlerin 100 değerine karşılık gelen hücre, 101, 000 veya 110 hücresine komşu olur. Birden fazla değişkenin değeri değişiyorsa bu hücreler komşu değildir. Diagonal hücreler de komşu olmazlar.

AB \ C	0	1
00	↑ X	
01		
11	X ↑	
10	X →	X

Aslında her hücre, kendisinin dört tarafındaki hücrelere komşudur diyebiliriz. Ayrıca örneğimizdeki gibi en üst hücre ile en alt hücre komşu olabilmektedir.

❖ Genel olarak lojik ifademizde kaç değişken varsa, hücreler o sayıda komşuluğa sahiptir.

Standart Çarpımlar Toplamı Biçimindeki İfadelerin Haritalanması

Lojik ifadedeki her çarpım terimi için, haritada ilgili hücreye 1 yazılır. Diğer hücrelere bir şey yazılmaz. Normalde bu işlem sonrasında çok sayıda hücre 1 değerine sahip olur. Şayet hiç 1 yoksa lojik ifade 0'a, hepsi 1 ise lojik ifade 1'e eşit olacaktır. Ayrıca doğruluk tablosundan da harita oluşturulabilir.

Örnek: $F(A,B,C) = A'B'C + AB'C + AB'C'$ şeklindeki lojik ifadeyi Karnaugh haritasına taşıyalım.

Lojik ifadedeki her mintermi 1 yapan değişkenlerin ikili karşılıkları;

$A'B'C : 001$

$AB'C : 101$

$AB'C' : 100$

C AB	0	1
00		1
01		
11		
10	1	1

Not:

Standart olmayan formlardan haritalama yapmak istersek, her terimi eksik değişkenlerle genişletmemiz gerekir. 3 değişkenli bir ifadede $A'B$ şeklinde bir terim varsa, bunu $A'B(C+C') = A'BC + A'BC'$ olarak standart forma dönüştürmemiz gerekir. Bu genişletme işleminin pratik pratik yolunu daha önce göstermiştik.

Örnek: $F(A,B,C) = AB' + A'B'C + B$ şeklindeki ifadeyi haritanın hücrelerine yerleştirelim.

➤ AB' teriminde C eksik değişkendir o halde 2 kombinasyon vardır;

$$AB' = AB'C' + AB'C$$

➤ $A'B'C$ teriminde eksik değişken yoktur.

➤ B teriminde A ve C eksik değişkendir o halde 4 kombinasyon vardır;

$$A'BC' + A'BC + ABC' + ABC$$

$\begin{matrix} & C \\ AB & \end{matrix}$	0	1
00		1
01	1	1
11	1	1
10	1	1

Standart Çarpımların Toplamı Biçimindeki İfadelerin Sadeleştirilmesi

Üç aşamada gerçekleşir; 1'lerin gruplandırılması, her bir gruptaki çarpım terimlerinin oluşturulması ve çarpım terimlerinin toplanması.

1'ler gruplandırılırken;

- Komşu olan hücreler göz önünde bulundurulur.
 - Grubun mümkün olduğunca çok sayıda 1 içermesine dikkat edilir.
 - Grup içerisindeki terim sayısı, 2'nin üstel kuvveti kadar olmalıdır. Yani 3 değişkenli bir ifade 1'li, 2'li, 4'lü ya da 8'li grup içerebilir.
 - Gruptaki her bir hücre aynı gruptaki bir veya daha fazla hücreye komşu olmalıdır. Ancak gruptaki tüm hücrelerin birbirlerine komşu olmaları gerekmez.
 - Haritadaki her 1, en az bir grup içerisinde yer almalıdır.
- ❖ Temel prensip, az sayıda grup oluşturmak ve grupların mümkün olduğunca çok sayıda terim içermesidir.

Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi

Gruplama işleminden sonra, her bir grup için ayrı ayrı bir çarpım terimi oluşturulur; şayet grup, bir değişkenin hem kendisini hem de tümleyenini içeriyorsa bu değişken göz ardı edilir, diğer değişkenler çarpım terimini oluştururlar.

A \ B	0	1
0	1	1
1	1	

$$B' + A'$$

AB \ C	0	1
00	1	1
01		
11		1
10	1	1

$$B' + AC$$

AB \ C	0	1
00	1	1
01	1	
11		
10		1

$$A'C' + B'C$$

Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi

CD \ AB	00	01	11	10
00			1	1
01	1	1	1	1
11				
10	1	1		

$$A'B + A'C + AB'C'$$

CD \ AB	00	01	11	10
00	1			1
01	1	1		1
11	1	1		1
10	1			1

$$D' + BC'$$

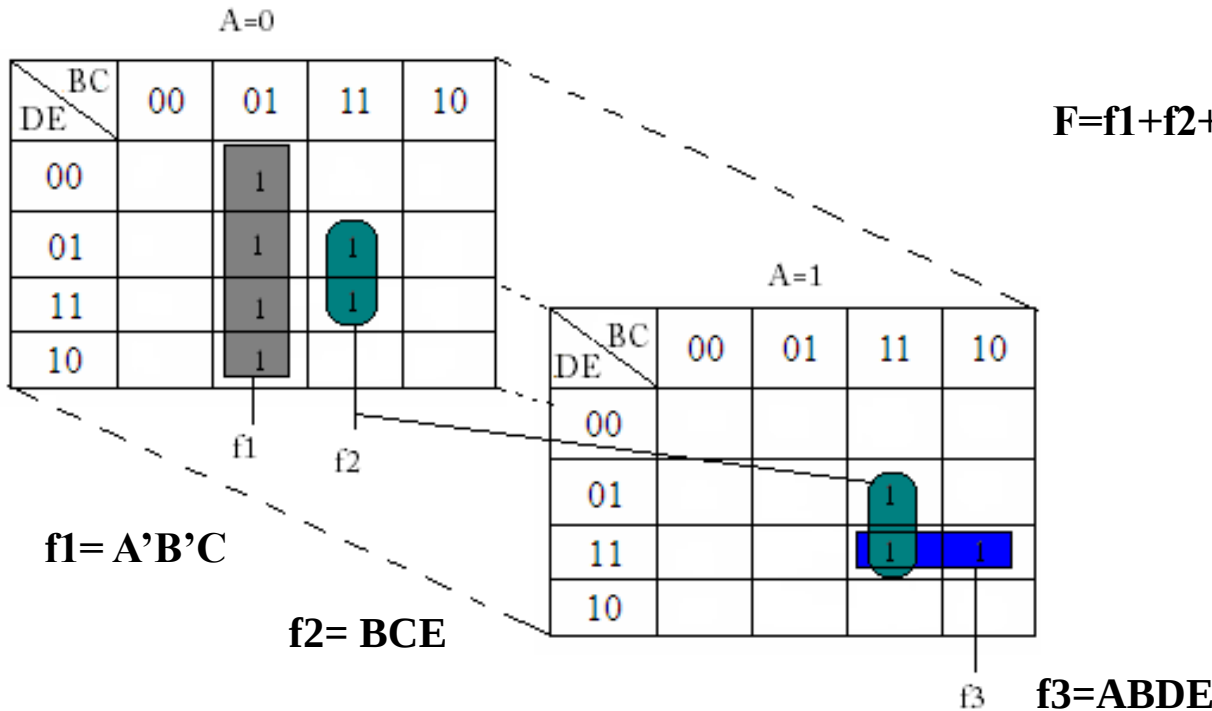
CD \ AB	00	01	11	10
00	1	1	1	
01	1	1	1	
11		1	1	
10				

$$A'C' + A'D + BD$$

Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi

Örnek: $F(A,B,C,D,E) = \sum (4,5,6,7,13,15,27,29,31)$ ifadesini Karnaugh ile sadeleştirelim.

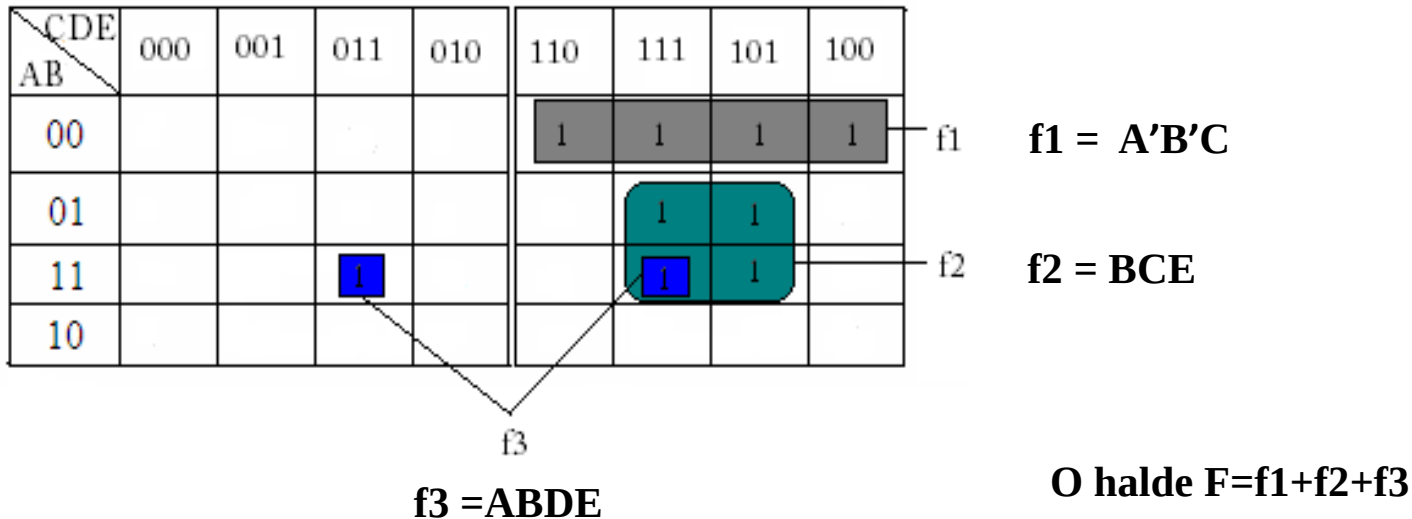
En anlamlı A değişkeni ayrıca ele alınır, diğer 4 değişken haritalara yerleştirilir. Mintermler haritalara taşındıktan sonra komşu olan hücreler belirlenmeye çalışılır. Üst üste gelen hücreler komşu olur.



Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi

Örnek: $F(A,B,C,D,E) = \sum(4,5,6,7,13,15,27,29,31)$ ifadesini Karnaugh ile sadeleştirelim.

Gruplama yapılırken, haritanın dikey olarak ortadan ikiye katlandığı düşünülür ve üst üste gelen terimler komşu olurlar. 6'lı Karnaugh haritasında ise hem dikey hem de yatay katlama işlemi yapılabilir, üst üste gelen hücreler komşudur.



Gruplama ve Sadeleşmiş İfadelerin Elde Edilmesi

Örnek: $F(A,B,C,D,E) = \sum(0,2,4,6,9,13,25,29)$ ifadesini Karnaugh ile sadeleştirelim.

CDE AB	000	001	011	010	110	111	101	100
00	1			1	1			1
01		1					1	
11		1					1	
10								

$f1 = A'B'E'$

$f2 = BD'E$

$$F = f1 + f2$$

Standart Toplamlar Çarpımı Biçimindeki İfadelerin Haritalanması

Lojik ifadedeki her toplam terimine karşılık gelen hücreye 0 yazılır. 0 olmayan hücreler 1'dir, yazılmayabilir. Örneğin 3 değişkenli bir ifadede $(A'+B+C')$ terimi varsa 101 hücresine 0 yazılır.

Örnek: $F(A,B,C) = (A'+B+C)(A+B'+C)(A+B'+C')$

AB \ C	0	1	
00			$A+B'+C$
01	0	0	$A+B'+C'$
11			
10	0		$A'+B+C$

Standart Toplamlar Çarpımı Biçimindeki İfadelerin Karnaugh ile Sadeleştirilmesi

Gruplama prosedürü, çarpımlar toplamı biçimindeki ifadelere benzer şekilde yapılır. Burada 0'lar gruplanarak toplam terimleri elde edilir. Daha sonra da toplam terimleri çarpılır.

Örnek: Şayet haritaya toplamlar çarpımı terimleri yerleştirildiyse geriye kalan hücreler çarpımlar toplamı terimlerini oluşturur. Sadeleştirmeler yapıldığında her iki sadeleşmiş ifadenin de eşit olduğu görülebilir.

BC \ A	00	01	11	10
0	0	0		
1	0	0	0	

$$B.(A'+C')$$

BC \ A	00	01	11	10
0			1	1
1				1

$$A'B+BC'$$

Önemsiz “Don’t Care” Durumlar

Değişkenlerin birbirinden tam olarak bağımsız olmadığı ya da bazı kombinasyonların oluşmasının mümkün olmadığı durumlar, önemsiz durumlar olarak tanımlanmaktadır. Örnek vermek gerekirse BCD kodunda 6 geçersiz durum, önemsiz durumları oluşturur. Önemsiz durumlar, lojik ifadeyi sadeleştirmede yardımcı oluyorsa değeri 1, diğer durumda 0 alınır.

Örnek: BCD kodunun kullanıldığı bir sistemde, sayının değeri 9 olduğunda çıkış veren bir devre tasarlayalım.

CD \ AB	00	01	11	10
00				
01				
11	x	x	x	x
10		1	x	x

Devrenin lojik eşdeğeri $f(A,B,C,D) = A.D$ olur.

Önemsiz “Don’t Care” Durumlar

Örnek: $F(A,B,C,D) = \sum(1,5,8,12)$

ve önemsiz durumlar $F_x(A,B,C,D) = \sum(2,3,6,7,10,11,14,15)$ olduğuna göre F fonksiyonunu Karnaugh yardımıyla indirgeyelim.

CD \ AB	00	01	11	10
00		1	x	x
01		1	x	x
11	1		x	x
10	1		x	x

f1 points to the group of 1s in the first two rows, columns 01 and 11.
f2 points to the group of 1s in the first column, rows 00 and 01.

$$f1 = A'.D$$

$$f2 = A.D'$$

$$F = f1 + f2 = A'D + AD' = A \oplus D$$

Bölüm 5. Karnaugh Haritaları (Devamı)

❖ Tablo Yöntemiyle (Tabulation Method) Lojik İfadelerin İndirgenmesi

- Asal Bileşenlerin Bulunması
- Asıl Asal Bileşenlerin Bulunması

❖ NAND veya NOR Kapılarının Üniversal Kapılar Olarak Kullanılması

- Üniversal Kapılar Kullanılarak Diğer Kapıların Elde Edilmesi
- Lojik İfadeleri Cebirsel Olarak NAND ya da NOR Formuna Getirme

Tablo Yöntemiyle (Tabulation Method) Lojik İfadelerin İndirgenmesi

Quine-McCluskey yöntemi olarak bilinen tablo yöntemi, özellikle değişken sayısı fazla olan lojik ifadelerin indirgenmesinde algoritmik bir yaklaşım sunar.

İlk olarak mintermler, içerdikleri 1 sayısına göre gruplanır (Önemsiz birleşimler de bu işleme dahil edilir).

Daha sonra birbirine komşu gruplar arasında sadece 1 değişken değişmişse, elde edilen mintermin o değişkenine önemsiz manasında ‘—’ işareti konur, diğerleri aynen alınır. Komşu gruptaki terimler arasında ilişki varsa, bu ilişkinin varlığını göstermek için ‘ $\sqrt{}$ ’ sembolü kullanılır.

Komşu gruplar arasında ilişki içermeyen terimler, asal bileşenleri oluşturur. Bu ilişkilendirme işlemleri, elde edilen terimler üzerinde de devam eder (Ta ki ilişkilendirme işlemleri kalmayıncaya kadar). Daha sonra da asıl asal bileşenlerin bulunması için bir tablo düzenlenir.

Quine-McCluskey Yöntemi

Örnek: $f(A,B,C,D)=\sum(4,8,10,11,12,15)$ ifadesini tablo yöntemiyle sadeleştirelim.

- İlk aşamada mintermler içerdikleri 1'lere göre gruplandırılır;

Terimlerdeki 1 sayısı	Mintermler	İkili gösterimi
1	m_4	0100
	m_8	1000
2	m_{10}	1010
	m_{12}	1100
3	m_{11}	1011
4	m_{15}	1111

Örnek: (devam)

Terimlerdeki 1 sayısı	Mintermler	İkili gösterimi	İlişkili terimler	Bileşenler
1	m_4	0100 ✓	$m(4,12)$	-100
	m_8	1000 ✓	$m(8,10)$	10-0
2	m_{10}	1010 ✓	$m(8,12)$	1-00
	m_{12}	1100 ✓	$m(10,11)$	101-
3	m_{11}	1011 ✓	$m(11,15)$	1-11
4	m_{15}	1111 ✓		



**Komşu gruplar arasında ilişki olmadığından,
buradaki tüm terimler asal bileşenleri oluşturur.**

Örnek: (devam)

Asal bileşenleri oluşturduktan sonra, asıl asal bileşenlerin bulunması süreci başlar. Asıl asal bileşen tablosu, satırlarda asal bileşenleri, sütunlarda ise mintermleri barındırır. Asal bileşenler hangi mintermleri içeriyorsa ilgili sütuna 'X' işareti konur. Daha sonra sütunlar taranır ve sadece tek 'X' işaretinin olduğu hücreler işaretlenir.

	Bileşenler	4	8	10	11	12	15
m(4,12)	-100 (BC'D')	X				X	
m(8,10)	10-0 (AB'D')		X	X			
m(8,12)	1-00 (AC'D')		X			X	
m(10,11)	101- (AB'C)			X	X		
m(11,15)	1-11 (ACD)				X		X

→
→
→
→

} Asıl asal bileşenler

$$F(A,B,C,D) = BC'D' + ACD + AB'D'$$

Örnek: (devam)

F fonksiyonunun içerdiği mintermleri Karnaugh haritasına taşıdığımızda, indirgemeye girebilecek olan terimlerin, tablo yöntemiyle bulduğumuz asal bileşenler olduğunu görmekteyiz. Asıl asal bileşenler ise, haritadaki en iyi indirgeme işlemine karşılık gelen asal bileşenlerdir.

AB \ CD	00	01	11	10
00		1	1	1
01				
11			1	1
10				1

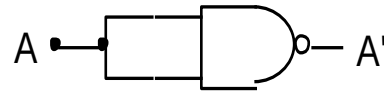
Gri ve kahverengi tonlamayla verilen gruplar, tablo yönteminde kesinlikle olması gereken asıl asal bileşenlere karşılık gelmektedir. Harita incelendiğinde m_4 ve m_{15} mintermlerini örten başka bir grup olmadığı görülmektedir. Diğer asıl asal bileşen de mavi tonlamayla verilen gruptur.

NAND veya NOR Kapılarının Üniversal Kapılar Olarak Kullanılması

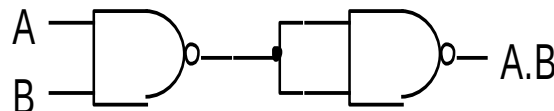
Şimdiye kadar ele alınan lojik ifadelerde NOT, AND ve OR kapıları kullanılarak kombinasyonel devrelerin tasarımı yapıldı. Aslında üniversal kapılar olarak bilinen NAND veya NOR kapılarıyla diğer kapı elemanlarını oluşturmak mümkündür. Böylece değişik tipte kapılar kullanmak yerine tek tipte kapılar kullanılarak, devrelerin oluşturulması daha ekonomik bir hale gelir.

NAND kapıları kullanılarak diğer kapıların oluşturulması:

- $(A.A)' = A'$ olduğundan, NOT kapısı oluşturmak için NAND kapısının girişlerini birleştirmek gerekir.

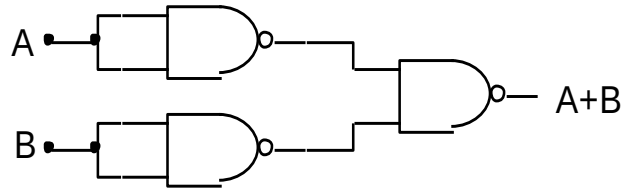


- $((A.B)')' = A.B$ olduğundan, NAND kapısının çıkışının değili alındığında AND kapısı elde edilir.

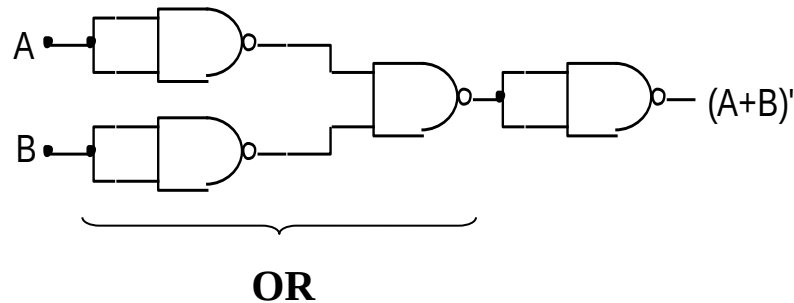


NAND veya NOR Kapılarının Üniversal Kapılar Olarak Kullanılması

- $(A'.B')' = A+B$ olduğundan, A ve B girişlerinin değilleri alındıktan sonra NAND kapısının girişlerine uygulanırsa OR kapısı elde edilmiş olur.



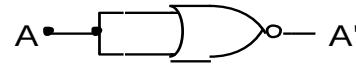
- NOR işlevini elde etmek için de NAND kapılarından oluşturulan OR kapısının çıkışı değillenir.



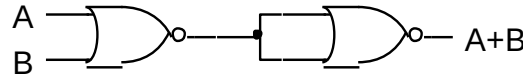
NAND veya NOR Kapılarının Üniversal Kapılar Olarak Kullanılması

NOR kapılarını kullanarak diğer kapıların oluşturulması:

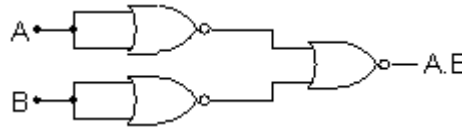
- $(A+A)' = A'$ olduğundan, NOT kapısı elde etmek için NAND kapısında olduğu gibi NOR kapısının girişlerinin birleştirilmesi gerekir.



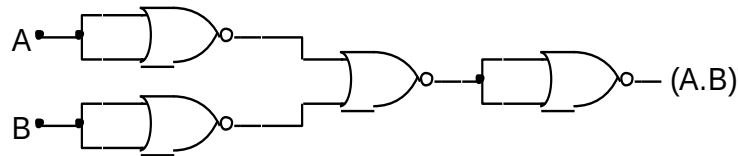
- $((A+B)')' = A+B$ olduğundan, OR kapısını elde etmek için NOR kapısının çıkışının değilinin alınması gerekir.



- $(A'+B')' = A.B$ olduğundan, A ve B girişlerinin değilleri alındıktan sonra NOR kapısının girişlerine uygulanırsa AND kapısı elde edilmiş olur.

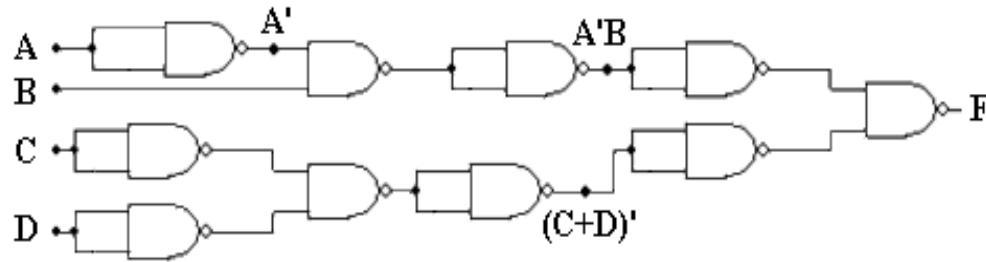


- NAND kapısı elde etmek için, NOR kapılarından oluşturulan AND kapısının çıkışının değilinin alınması gerekir.

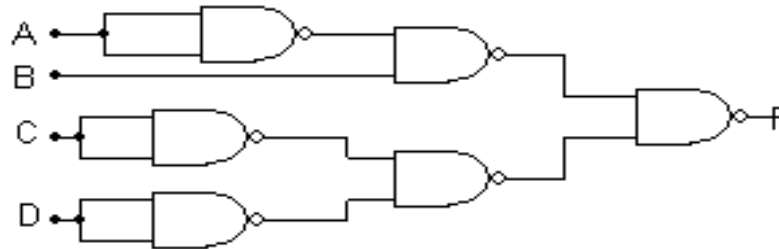


Lojik İfadelerin NAND ile Gerçeklenmesi

Örnek: $F(A,B,C,D) = A'B + (C+D)'$ lojik ifadesini NAND kapıları kullanarak gerçekleştirmek istersek NOT, AND, OR ve NOR işlemlerine karşılık gelen devreleri, yerlerine koymak gerekir.



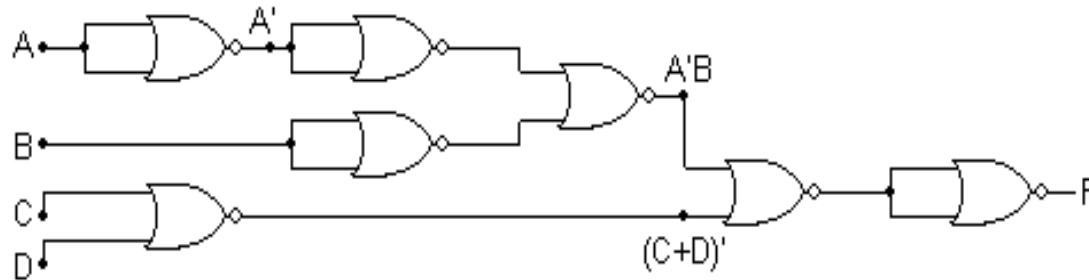
İki kere alınan değil işlemleri birbirini götürcektir;



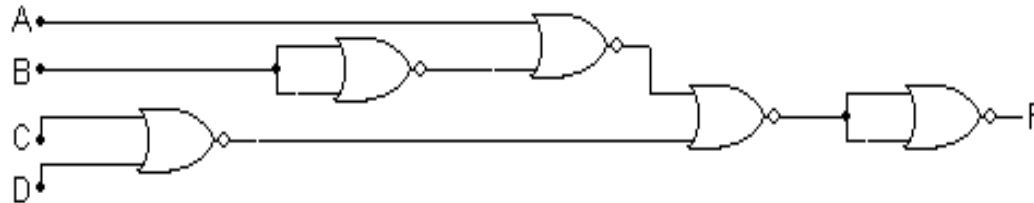
Lojik İfadelerin NOR ile Gerçeklenmesi

Örnek: Aynı işlemi NOR kapıları kullanarak gerçekleştirelim.

$$F(A,B,C,D) = A'B + (C+D)'$$



İki kere alınan değil işlemleri birbirini götürcektir;



Lojik İfadeleri Cebirsel Olarak NAND ya da NOR Formuna Getirme

Özellikle DeMorgan kuralları kullanılarak istenen form elde edilebilir.

- Çarpımlar toplamı şeklindeki ifadelerin iki kere değilleri alınmak suretiyle NAND formu elde edilebilir.
- Topamlar çarpımı şeklindeki ifadelerin de iki kere değilleri alınmak suretiyle NOR formu elde edilebilir.

Örnek: $f(a,b,c,d) = ab' + c'd$ ifadesinin NAND formunu elde etmek istersek;

$$(f')' = f \text{ olduğundan, } f = [(ab' + c'd)']' = [(ab')' \cdot (c'd)']'$$

Örnek: $f(a,b,c,d) = (a+b')(c'+d)$ ifadesinin NOR formunu elde etmek istersek;

$$(f')' = f \text{ olduğundan, } f = [(a+b')(c'+d)]' = [(a+b')' + (c'+d)']'$$

Lojik İfadeleri Cebirsel Olarak NAND ya da NOR Formuna Getirme

Örnek: $f(a,b,c,d) = ab' + c'd$ ifadesinin NOR formunu elde etmek istersek; ifadedeki tüm terimlerin ayrı ayrı iki kere değillerini almamız gerekir.

$$f(a,b,c,d) = [(ab')']' + [(c'd)']' = (a' + b)' + (c + d')'$$

Elde edilen ifadenin tamamının iki kere değilini alırsak;

$$f(a,b,c,d) = ([(a' + b)' + (c + d')']')'$$

Örnek: $f(a,b,c,d) = (a + b')(c' + d)$ ifadesinin NAND formunu elde etmek istersek, ifadedeki tüm terimlerin ayrı ayrı iki kere değillerini almamız gerekir.

$$f(a,b,c,d) = (a + b')(c' + d) = ([(a + b')]')' . ([(c' + d)]')' = (a'b)' . (cd')'$$

Elde edilen ifadenin tamamının iki kere değilini alırsak;

$$f(a,b,c,d) = ([(a'b)' . (cd')']')'$$

Mintermler Cinsinden Verilmiş Lojik İfadeleri, Karnaugh yardımıyla NAND ya da NOR Formuna Getirme

Örnek: $f(a,b,c) = \sum(0,1,3,5,6,7)$ lojik ifadesini NAND ve NOR kapılarıyla ayrı ayrı gerçekleyelim.

Lojik ifadenin içerdiği mintermleri Karnaugh haritasına taşırsak;

a \ bc	00	01	11	10
0	1	1	1	
1		1	1	1

$f(a,b,c) = c + a'b' + ab$ olur. İfadeyi NAND formuna dönüştürmek istersek iki kere deęilini almamız yeterlidir.

$$f(a,b,c) = [(c + a'b' + ab)']' = [c' \cdot (a'b')' \cdot (ab)']'$$

Devreler genellikle 2 giriře sahip NAND kapıları kullanılarak gereklenir:

$$f(a,b,c) = [c' \cdot (a'b')' \cdot (ab)']' = [c' \cdot ((a'b')' \cdot (ab)')]'$$

Örnek: (devam)

f fonksiyonunu NOR kapıları kullanarak gerçeklemek istersek f 'nin tümleyenini kullanmamız daha uygun olur. f 'nin tümleyenini (f') bulmak için Karnaugh haritasındaki boş hücrelere 1 koymamız gerekir.

f

$a \backslash bc$	00	01	11	10
0	1	1	1	
1		1	1	1

f'

$a \backslash bc$	00	01	11	10
0				1
1	1			

$$f'(a,b,c) = ab'c' + a'bc'$$

$$f = (f')' = (a' + b + c).(a + b' + c)$$

Toplamlar çarpımı biçimindeki ifadenin iki kere değilini almak suretiyle NOR formuna getirebiliriz:

$$f = [(a' + b + c)' + (a + b' + c)']'$$

Bu ifadeyi 2 girişli NOR kapılarıyla gerçeklemek istersek,

$$f = ([((a' + b)')' + c]' + [((a + b')')' + c]')'$$

Bölüm 6. KOMBİNASYONEL DEVRELER

❖ Toplayıcılar

- Yarım Toplayıcı
- Tam Toplayıcı
- Paralel Toplayıcılar

❖ Karşılaştırıcılar

❖ Kod Çözücüler

❖ Kodlayıcılar

Toplayıcılar

Yarım Toplayıcı (Half Adder - HA):

Daha önce ifade edildiği gibi ikili toplama işlemi şu şekildeydi;

$$0+0 = 0,$$

$$0+1 = 1,$$

$$1+0 = 1,$$

$$1+1 = 10 \text{ (buradaki 1, eldeyi temsil etmektedir)}$$

Tek bit üzerinde yerine getirilen bu işlemi icra eden devreler yarım toplayıcı olarak bilinir. Yani yarım toplayıcı, bir bitlik iki sayının toplamını bulan ve çıkışında toplam ve elde bitlerini veren kombinasyonel bir devredir.

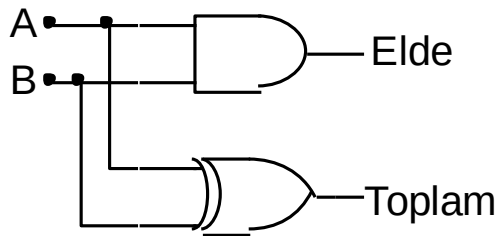
Toplayıcılar

Yarım toplayıcının doğruluk tablosu;

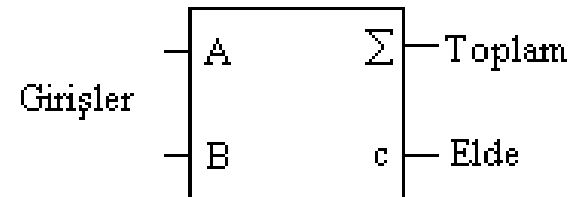
A B	Toplam	Elde
0 0	0	0
0 1	1	0
1 0	1	0
1 1	0	1

$$\text{Toplam} = A \oplus B$$

$$\text{Elde} = A.B$$



Yarım toplayıcının devre



Yarım toplayıcının grafiksel gösterimi

Toplayıcılar

Tam Toplayıcı (Full Adder - FA):

Tam toplayıcı bir devre, 1'er bitlik 2 girişi ve elde bitini alır, çıkışında toplam ve elde bitlerini oluşturur. Yarım toplayıcıya ek olarak elde girişi de vardır.

Tam toplayıcının doğruluk tablosu;

Girişler			Çıkışlar	
A	B	ci	Toplam	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

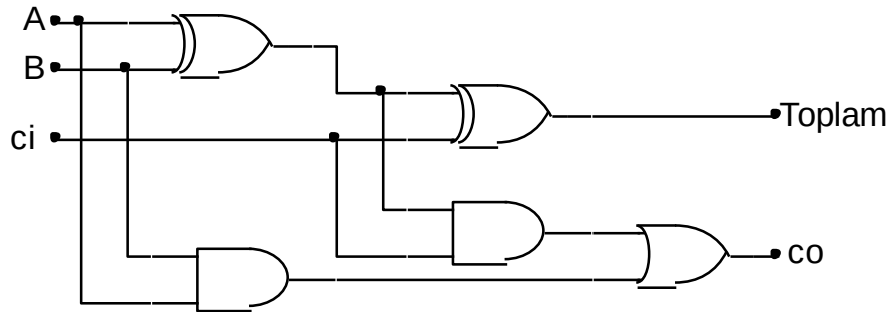
$$\begin{aligned}\text{Toplam} &= A'.B'.ci + A'.B.ci' + A.B'.ci' + A.B.ci \\ &= ci.(A'.B' + A.B) + ci'.(A'.B + A.B') \\ &= ci.(A \oplus B)' + ci'.(A \oplus B) \\ &= A \oplus B \oplus ci\end{aligned}$$

$$\begin{aligned}co &= A'.B.ci + A.B'.ci + A.B.ci' + A.B.ci \\ &= ci.(A \oplus B) + A.B\end{aligned}$$

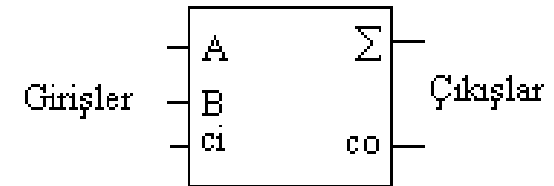
Toplayıcılar

$$\text{Toplam} = A \oplus B \oplus ci$$

$$co = ci.(A \oplus B) + A.B$$

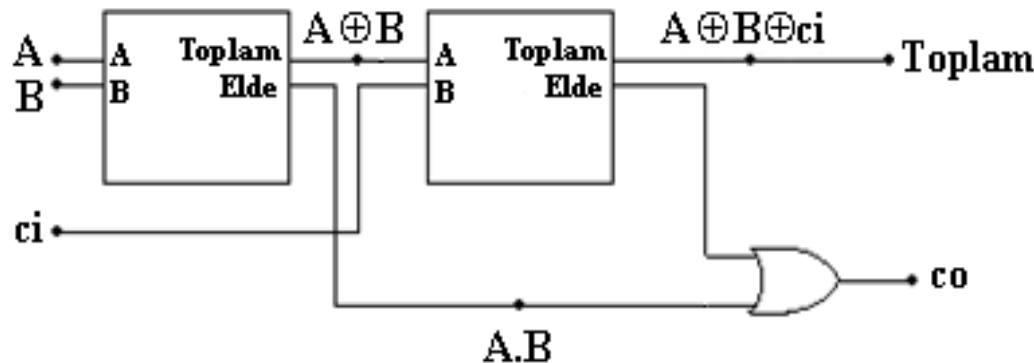


Tam toplayıcının devre şeması



Tam toplayıcının grafik gösterimi

2 yarım toplayıcı kullanarak tam toplayıcı elde etmek mümkündür:



Yarım toplayıcıda;

$$\text{Elde} = A.B$$

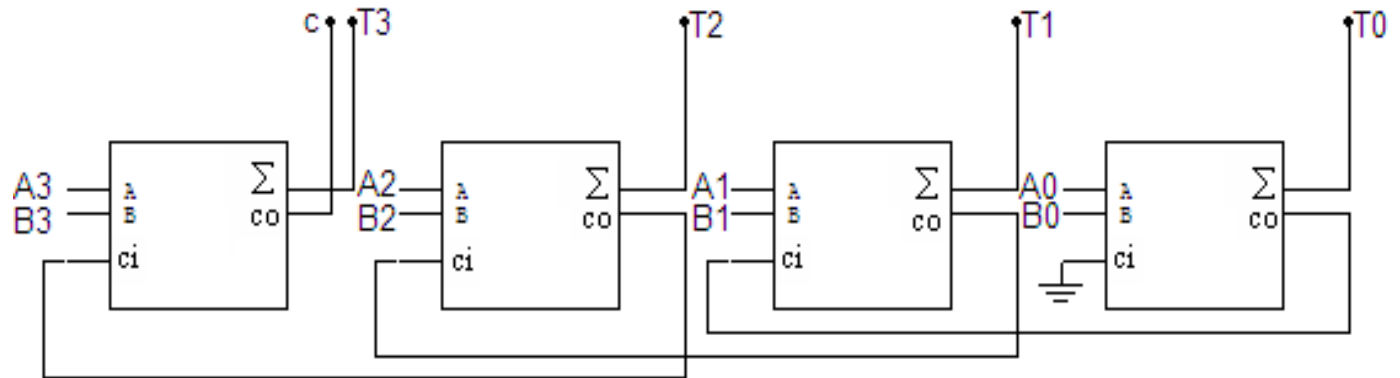
$$\text{Toplam} = A \oplus B$$

Toplayıcılar

Paralel İkili Toplayıcılar:

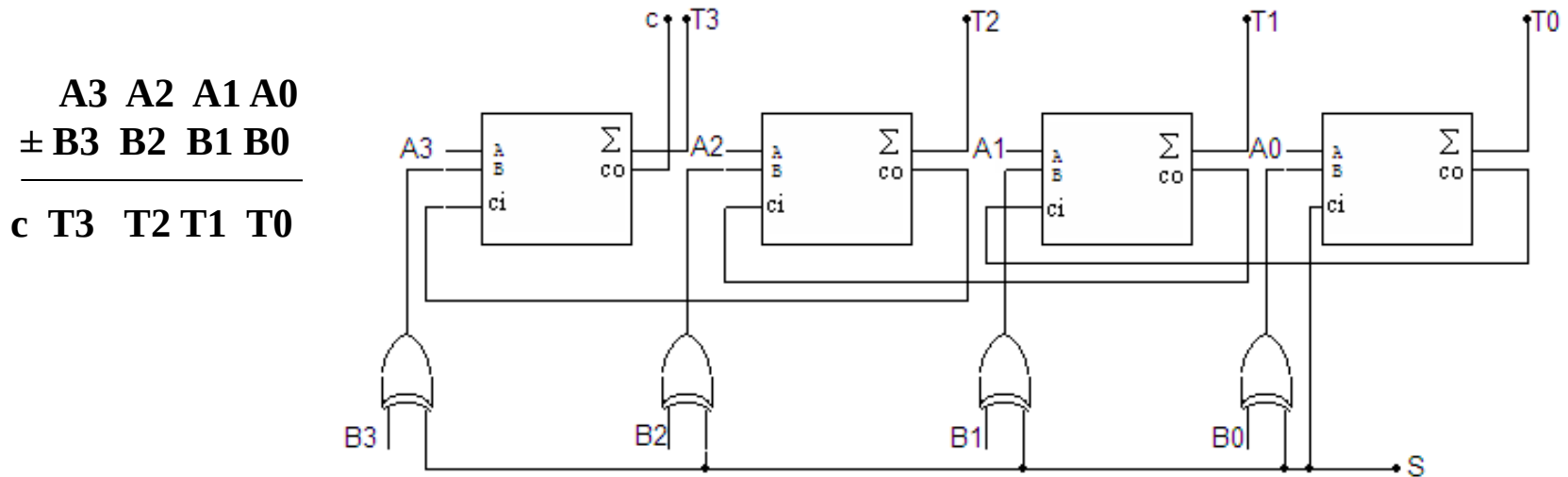
Yarım ve tam toplayıcı devreler, tek bitlik sayıların toplama işleminde kullanıldı. Çok sayıda bit içeren ikili sayıların toplanması için, tam toplayıcılar kaskat bağlanmak suretiyle paralel toplayıcılar geliştirilmiştir. Örneğin 4 bitlik iki sayının toplama işlemini 4 adet tam toplayıcıyla gerçekleştirebiliriz. Bu gerçekleştirimde, her bir tam toplayıcının elde çıkışı bir sonraki tam toplayıcının elde girişine verilir. En düşük değerli bitleri toplarken elde girişi 0 alınır. En yüksek değerli bitlerin toplamı neticesinde c elde biti oluşur.

$$\begin{array}{r} A3 \ A2 \ A1 \ A0 \\ + \ B3 \ B2 \ B1 \ B0 \\ \hline c \ T3 \ T2 \ T1 \ T0 \end{array}$$



Toplayıcılar

Çıkarma işlemi, 2'ye tümleyen ile toplama işlemine dönüşebildiğinden, aşağıdaki devre yardımıyla hem toplama hem de çıkarma işlemleri yapılabilir. 2'ye tümleyeni alınan sayı $(B3B2B1B0)_2$ ikili sayısıdır.

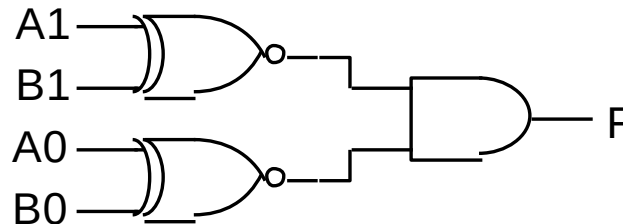


$S = 0$ için toplama işlemi, $S = 1$ için de çıkarma işlemi yapılır.

Karşılaştırıcılar

Karşılaştırıcılar, iki tane ikili sayının büyüklüklerini karşılaştırmak için kullanılırlar. Özellikle sayıların eşit veya eşit olmama durumlarını tespit etmek için EXOR veya EXNOR kapıları kullanılabilir.

Örneğin 2 bitlik iki sayının (A_1A_0 ve B_1B_0) karşılaştırması için aşağıdaki devre kullanılabilir.



A ve B ikili sayıları birbirine eşit olduğunda çıkış 1, diğer durumlarda ise çıkış 0 olur. N adet bit içeren sayılar için de bu devre genişletilebilir.

Karşılaştırıcılar

Örneğin 4 bitlik iki sayının ($A_3A_2A_1A_0$ ve $B_3B_2B_1B_0$) büyüklüklerini karşılaştırmak istersek o zaman şu ifadeleri yazabiliriz;

- Şayet $A_3 = 1$ ve $B_3 = 0$ ise $A > B$,
- Şayet $A_3 = 0$ ve $B_3 = 1$ ise $A < B$,
- Şayet $A_3 = B_3$ ise diğer düşük anlamlı bitlere bakmak gerekir.

Şu eşitlikler yazılabilir;

$$A > B \text{ çıkışı} = A_3.B_3' + (A_3 \otimes B_3).A_2.B_2' + (A_3 \otimes B_3).(A_2 \otimes B_2).A_1.B_1' + (A_3 \otimes B_3).(A_2 \otimes B_2).(A_1 \otimes B_1).A_0.B_0'$$

$$A < B \text{ çıkışı} = A_3'.B_3 + (A_3 \otimes B_3).A_2'.B_2 + (A_3 \otimes B_3).(A_2 \otimes B_2).A_1'.B_1 + (A_3 \otimes B_3).(A_2 \otimes B_2).(A_1 \otimes B_1).A_0'.B_0$$

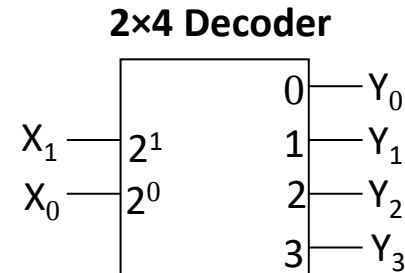
$$A = B \text{ çıkışı} = (A_3 \otimes B_3).(A_2 \otimes B_2).(A_1 \otimes B_1).(A_0 \otimes B_0)$$

Kod Çözücüler (Decoders)

En genel tanımıyla n adet girişi ve en fazla 2^n adet çıkışı bulunan kombinyasyonel devrelerdir. Girişindeki ikili sayının decimal değeri ne ise, o çıkışı aktif olur.

2×4 lük bir kod çözücünün doğruluk tablosu şu şekildedir;

$X_1 X_0$	Y_0	Y_1	Y_2	Y_3
0 0	1	0	0	0
0 1	0	1	0	0
1 0	0	0	1	0
1 1	0	0	0	1



$$Y_0 = X_1' \cdot X_0'$$

$$Y_1 = X_1' \cdot X_0$$

$$Y_2 = X_1 \cdot X_0'$$

$$Y_3 = X_1 \cdot X_0$$

2×4 decoderin grafiksel gösterimi

Kod Çözücüler (Decoders)

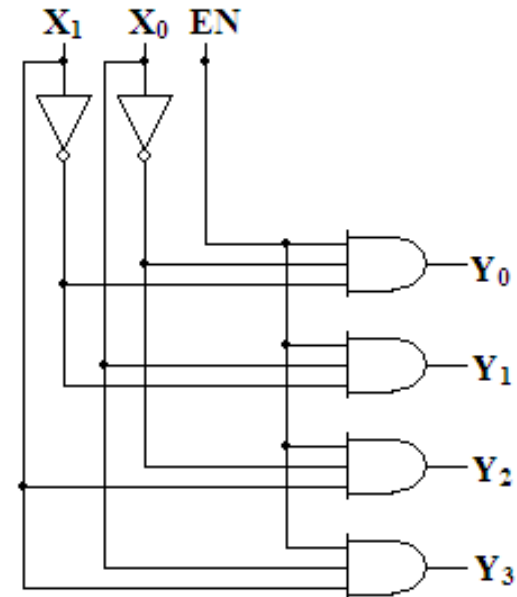
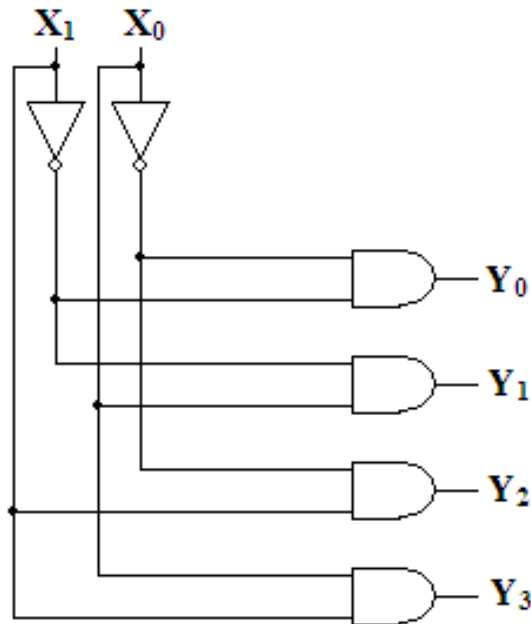
Kod çözücülerde yetki (Enable - EN) girişi de kullanılabilir. EN = 1 iken normal şekilde çalışırken, EN = 0 ise çıkışlar aktif değildir (tüm çıkışlar 0'dır).

$$Y_0 = X_1' \cdot X_0'$$

$$Y_1 = X_1' \cdot X_0$$

$$Y_2 = X_1 \cdot X_0'$$

$$Y_3 = X_1 \cdot X_0$$

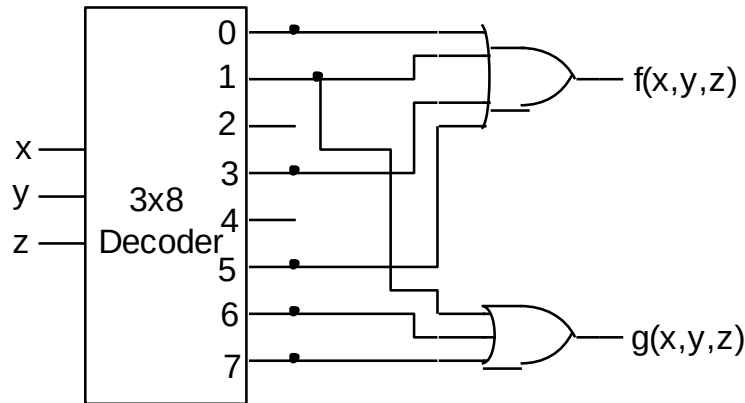


Kod Çözücüler (Decoders)

Decoder'ler, lojik fonksiyonların gerçekleştirilmesinde kullanılabilirler. Kod çözücülerin her bir çıkışı, aslında minterm ifadeleridir.

Genel olarak n adet girişe ve m adet çıkışa sahip bir lojik ifade, $n \times 2^n$ lik bir kod çözücü ve m adet de OR kapısı kullanılarak gerçekleştirilebilir.

Örnek: $f(x,y,z) = \Sigma(0,1,3,5)$ ve $g(x,y,z) = \Sigma(1,6,7)$ lojik ifadelerini bir kod çözücüyle gerçekleyelim.

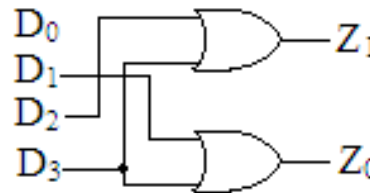


Kodlayıcılar (Encoders)

Kodlayıcılar, kod çözücülerin gerçekleştirdiğinin tersini yapan kombinasyonel devrelerdir. Genel olarak n adet çıkışı ve en çok 2^n adet de girişi vardır. Normal şartlarda girişlerinin sadece bir tanesinin 1 olması gerekir. Bu durumda çıkışında, hangi girişin 1 olduğunu gösteren ikili kod üretir. 4 girişli 2 çıkışlı bir kodlayıcının doğruluk tablosu aşağıdaki gibidir;

D_0	D_1	D_2	D_3	Z_1	Z_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

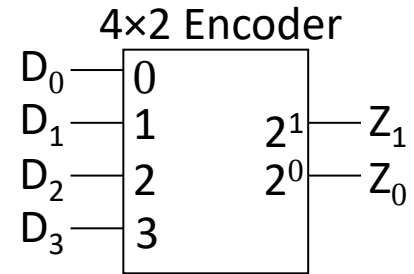
Aynı anda sadece bir tek girişin 1 olmasına müsaade edildiğinden, Z_1 ve Z_0 çıkışlarını Karnaugh ile hesap ederken, birden fazla girişin 1 olduğu durumları önemsiz durum (don't care) olarak alabiliriz.



$$Z_1 = D_2 + D_3$$

$$Z_0 = D_1 + D_3$$

$$NI = D_0' . D_1' . D_2' . D_3' \text{ (Giriş yoksa)}$$



Kodlayıcılar (Encoders)

Bazı durumlarda encoder'ın girişlerinden sadece bir tanesinin 1 olması mümkün olmayabilir. Böyle durumlar da düşünülerek öncelikli kodlayıcılar (Priority encoders) geliştirilmiştir. Örnek bir gerçekleştirim, 4 giriş ve 3 çıkış için aşağıda verilmiştir. Öncelik sırası D_3 'ten D_0 'a doğrudur.

D_0	D_1	D_2	D_3	Z_1	Z_0	NI
0	0	0	0	x	x	1
1	0	0	0	0	0	0
x	1	0	0	0	1	0
x	x	1	0	1	0	0
x	x	x	1	1	1	0

$$NI = D_0' \cdot D_1' \cdot D_2' \cdot D_3'$$

$D_3=1$ $D_2D_3=10$

D_2D_3 D_0D_1	00	01	11	10
00	x	1	1	1
01		1	1	1
11		1	1	1
10		1	1	1

$$Z_1 = D_2 + D_3$$

$D_2D_3=00$ $D_3=1$

D_2D_3 D_0D_1	00	01	11	10
00	x	1	1	
01		1	1	
11		1	1	
10		1	1	

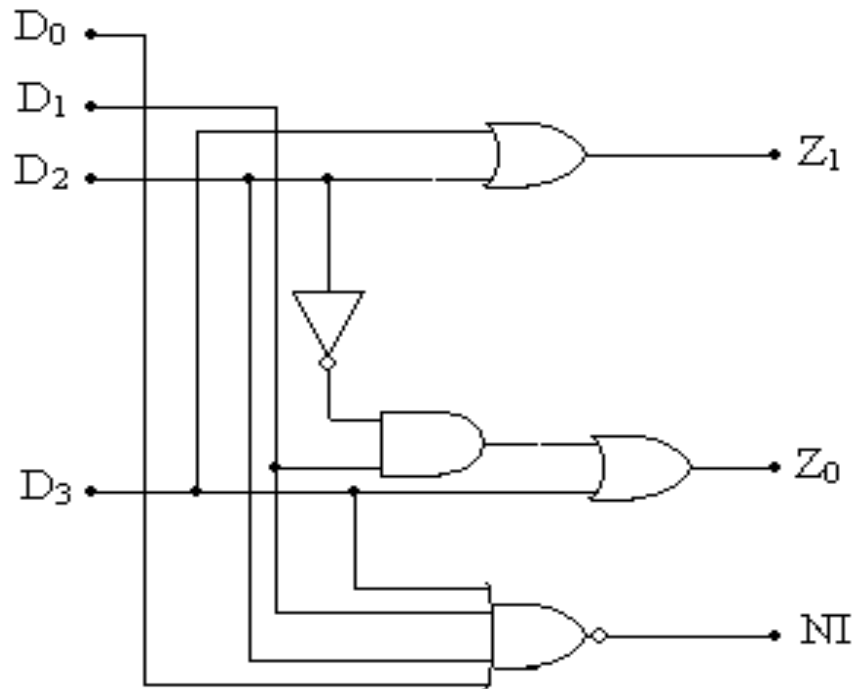
$$Z_0 = D_3 + D_2' \cdot D_1$$

Kodlayıcılar (Encoders)

$$Z_1 = D_2 + D_3$$

$$Z_0 = D_3 + D_2' \cdot D_1$$

$$NI = D_0' \cdot D_1' \cdot D_2' \cdot D_3' = (D_0 + D_1 + D_2 + D_3)'$$



4×2 öncelikli encoder devresi

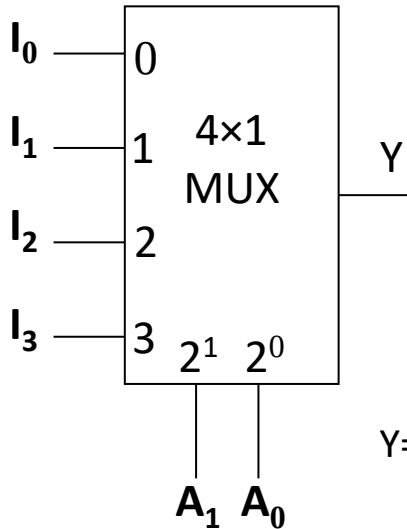
Bölüm 6. Kombinasyonel Devreler (Devamı)

- ❖ **Çoğullayıcılar (Veri Seçiciler-Multiplexers-MUX)**
- ❖ **Veri Dağıtıcılar (Demultiplexers-DEMUX)**
- ❖ **Yedi Segment Display ve Kod Çözücü Devresi**

Çoğullayıcılar (Veri Seçiciler-Multiplexers-MUX)

Çoğullayıcılar, n adet veri seçim ucuna, 2^n adet veri giriş ucuna ve 1 adet de çıkışa sahip kombinasyonel devrelerdir. Veri seçici uçların değerine göre, veri girişlerinden birini çıkışa aktarırlar.

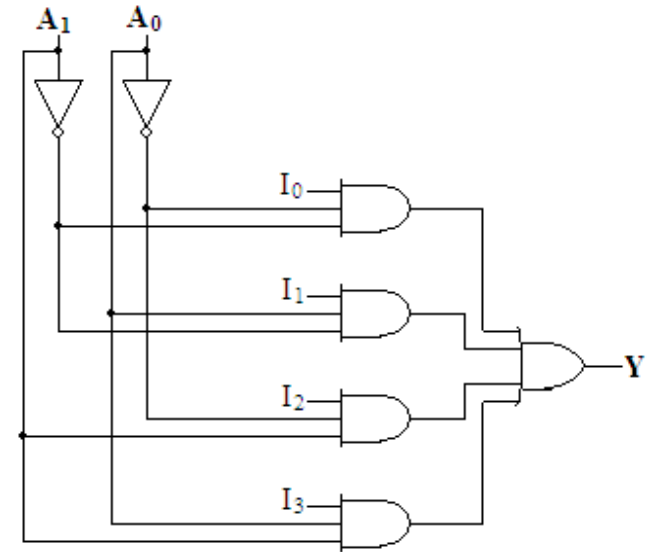
4×1'lik bir multiplexer'ın (MUX) gösterimi ve doğruluk tablosu aşağıdaki gibidir;



4×1 MUX'un gösterimi

A_1	A_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = I_0 \cdot A_1' \cdot A_0' + I_1 \cdot A_1' \cdot A_0 + I_2 \cdot A_1 \cdot A_0' + I_3 \cdot A_1 \cdot A_0$$



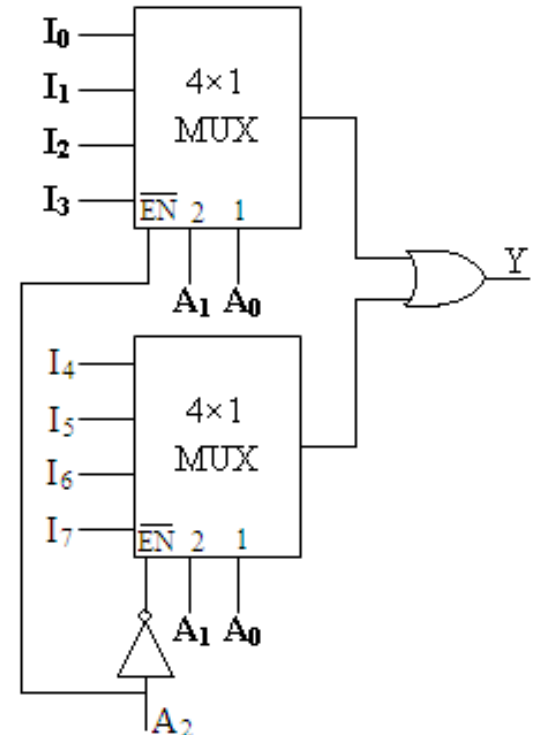
4×1 MUX'un lojik devresi

Multiplexers

➤ Yetki (Enable) girişli multiplexer'lar kullanılarak, daha fazla veri girişine sahip çoğullayıcı devreler oluşturulabilir.

Örnek: Yetki girişine sahip iki adet 4×1 'lik multiplexer kullanarak 8×1 'lik bir multiplexer oluşturalım.

	A_2	A_1	A_0	Y
Üstteki MUX aktif	0	0	0	I_0
	0	0	1	I_1
	0	1	0	I_2
	0	1	1	I_3
Alttaki MUX aktif	1	0	0	I_4
	1	0	1	I_5
	1	1	0	I_6
	1	1	1	I_7

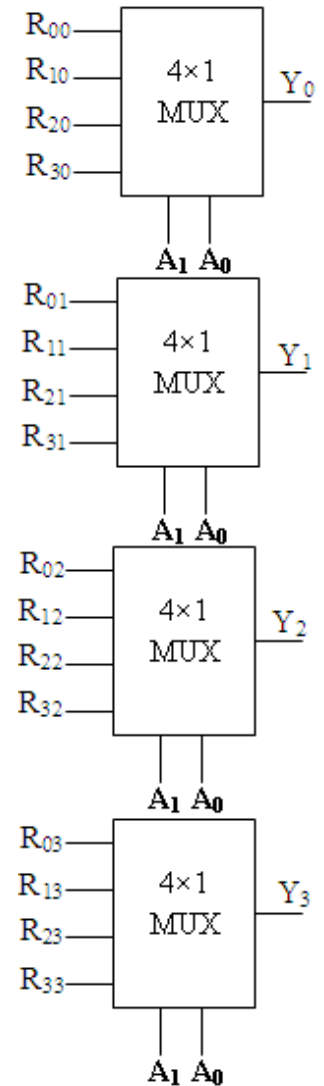


Multiplexers

➤ Multiplexer'lar ile ortak veri yolu oluşturulabilir.

Örnek: 4 bitlik bir veri yoluna, 4 kaydediciyi (register) bağlamak istersek, yandaki devre kullanılabilir. Burada R_0 , R_1 , R_2 ve R_3 dörder bitlik kaydedicilerdir.

Şayet $A_1A_0 = 00$ ise, R_0 (R_{00} , R_{01} , R_{02} , R_{03}) kaydedicisinin içeriği yola aktarılacaktır.



Multiplexers

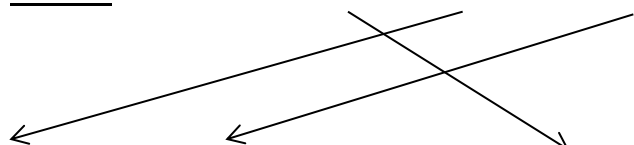
➤ Multiplexer'lar lojik ifadeleri gerçekleştirmek için de kullanılabilirler. n adet giriş ve 1 çıkışa sahip bir kombinasyonel devre, (n-1) adet veri seçici uca ve 2^{n-1} adet de veri giriş ucuna sahip $2^{n-1} \times 1$ multiplexer kullanılarak gerçekleştirilebilir.

Örnek: $f(a,b,c) = abc + a'b' + bc'$ fonksiyonunu, veri seçim uçlarını a ve b seçerek 4×1 MUX ile gerçekleştirelim.

Veri seçim uçları olarak a ve b değişkenleri seçildiğinden, tüm terimleri bu değişkenleri içerecek şekilde genişletmek gerekir;

$$f(a,b,c) = abc + a'b' + (a+a')bc' = \underline{abc} + a'b' + \underline{abc'} + a'bc' = ab + a'b' + a'bc'$$

Multiplexer'in tanım bağıntısından;

$$Y = I_0.a'.b' + I_1.a'.b + I_2.a.b' + I_3.a.b$$


O halde $I_0 = 1$, $I_1 = c'$, $I_2 = 0$, $I_3 = 1$ seçilmelidir.

Örnek: (devamı)

$$f(a,b,c) = ab + a'b' + a'bc'$$

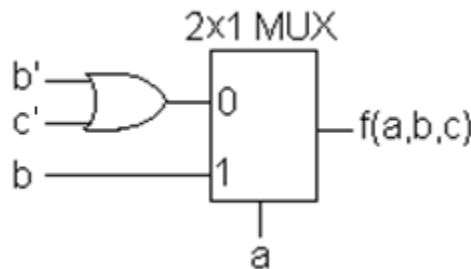
Aynı lojik ifadeyi veri seçim ucunu a seçerek 2×1 MUX ile gerçekleştirmek istersek, ekstra kapılar kullanmamız gerekecektir.

f fonksiyonunu a ve a' cinsinden yazarsak;

$$f(a,b,c) = ab + a'b' + a'bc' = ab + a'(b' + bc') = ab + a'(b' + c')$$

Multiplexer'in tanım bağıntısından; $Y = I_0.a' + I_1.a$

O halde $I_0 = b' + c'$ ve $I_1 = b$ seçersek lojik ifadeyi gerçekleştirmiş oluruz.



Dikkat edilecek olursa ekstradan VEYA kapısını kullanmamız gerekmiştir.

Multiplexers

➤ Standart çarpımlar biçiminde verilen bir lojik ifadeyi, uygun sayıda veri seçim ucuna sahip multiplexer kullanarak gerçekleştirmek için sistematik bir yaklaşım geliştirilmiştir.

Örnek: 3 girişe ve 1 çıkışa sahip bir kombinasyonel devrenin lojik ifadesi $f(a,b,c) = \Sigma(1,3,4,7)$ olsun. Bu lojik ifadeyi, seçim uçları b ve c olarak seçilen, 4×1 MUX ile gerçekleştirmek istediğimizi düşünelim.

$\begin{matrix} bc \\ a \end{matrix}$	00	01	10	11
0	m_0	m_1	m_2	m_3
1	m_4	m_5	m_6	m_7
Bağlantı	a	a'	0	1
Mux Veri Girişleri	I_0	I_1	I_2	I_3

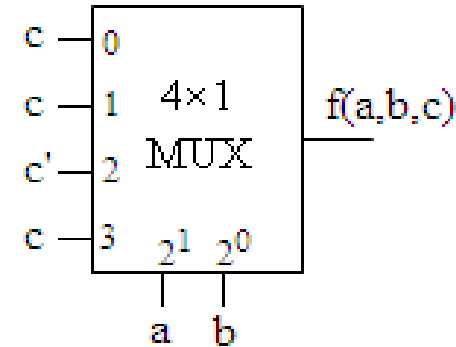
İşaretlenmiş olan minterler şayet a değişkeninin 1 olduğu satırdaysa bağlantı değeri a, 0 olduğu satırdaysa a', her ikisini de içeriyorsa 1 ve hiç birini içermiyorsa 0 seçilerek MUX'un I_0 , I_1 , I_2 , I_3 girişleri oluşturulmuştur.

Örnek: (devamı)

$$f(a,b,c)=\Sigma(1,3,4,7)$$

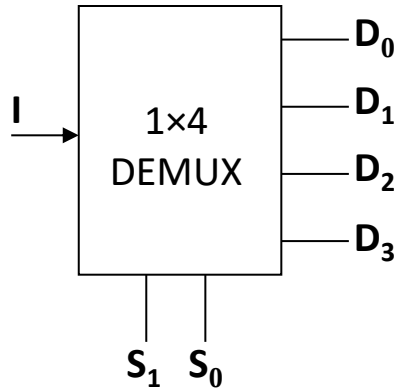
Aynı lojik ifadeyi veri seçim uçlarını a ve b seçerek gerçekleştirmek isteseydik;

ab c	00	01	10	11
0	m_0	m_2	m_4	m_6
1	m_1	m_3	m_5	m_7
Bağlantı	c	c	c'	c
Mux Veri Girişleri	I₀	I₁	I₂	I₃



Veri Dağıtıcılar (Demultiplexers-DEMUX)

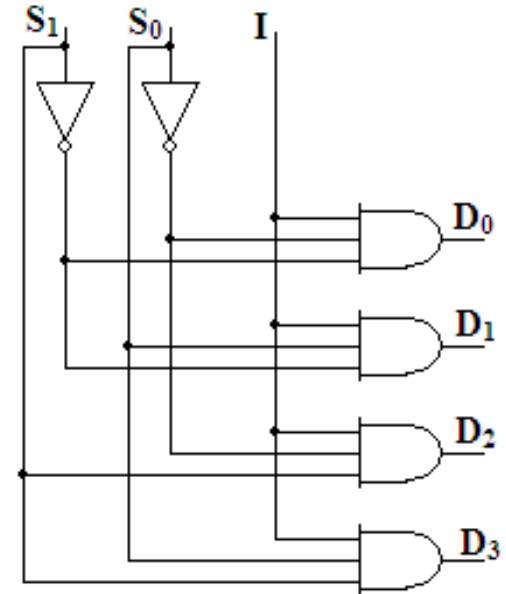
n adet seçici ucu, 1 tane girişi, 2^n adet de çıkışı bulunan kombinyasyonel devrelerdir. Seçici uçların oluşturduğu sayı, girişin hangi çıkışa yönlendirileceğini gösterir. Aşağıda 1×4 DEMUX'un gösterimi, doğruluk tablosu ve lojik devresi verilmiştir.



1×4 DEMUX'un gösterimi

S_1	S_0	D_0	D_1	D_2	D_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

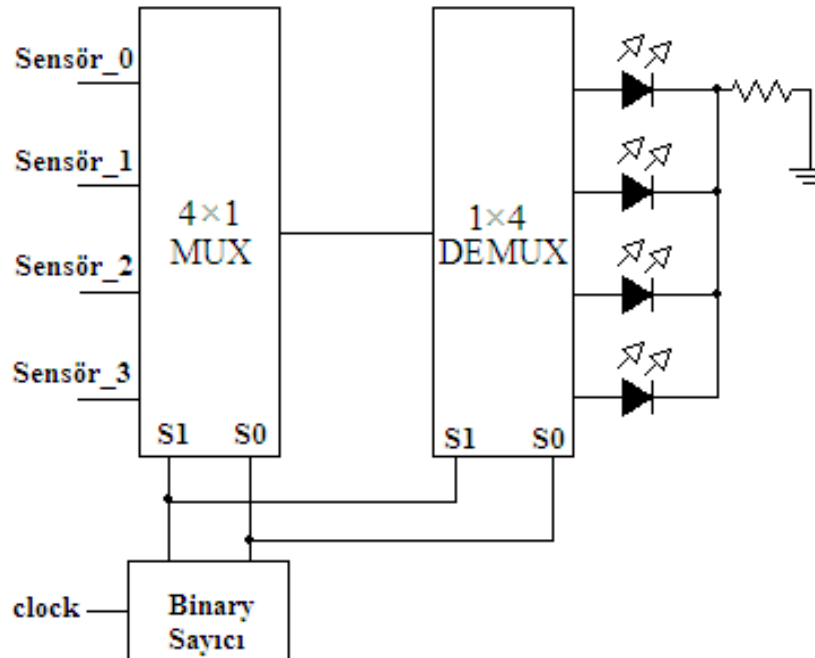
1×4 DEMUX'un
doğruluk tablosu



1×4 DEMUX'un lojik devresi

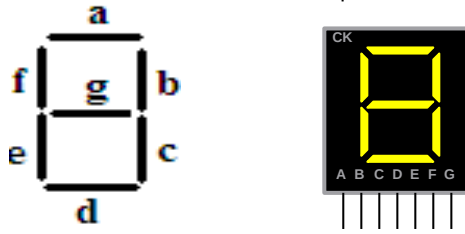
Demultiplexers

Örnek: Özellikle verilerin uzak yerlere taşınması için MUX ve DEMUX'lar birlikte kullanılabilir. Örneğin 4 sensörden gelen bilgi, uzakta bulunan bir gösterge panelinden izlenebilir. Şayet sensörlerden 1 bilgisi gelirse, DEMUX çıkışında bulunan LED'ler yanıp söner. Kullanılan sayıcı, MUX girişlerindeki bilgilerin sırayla iletilmesini sağlar. Sayıcının frekansı aynı zamanda LED'lerin yanıp sönme aralığını belirler.

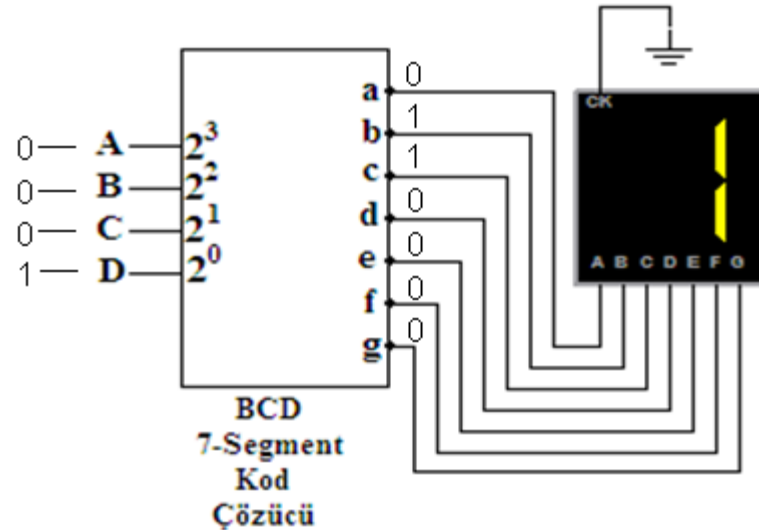


Yedi Segment Display ve Kod Çözücü Devresi

Yedi segment display'ler lojik devrelerde özellikle decimal sayıları göstermek için yoğun olarak kullanılırlar. 7 adet LED bir araya getirilerek oluşturulur. Ortak anot veya ortak katot olarak üretilirler. Bu display'i sürmek için bir kod çözücüye ihtiyaç vardır. Bu kısımda BCD'den 7-segment display'e bir kod çözücü tasarlanacaktır. Bu kod çözücü entegre devre (IC) olarak temin edilebilir.



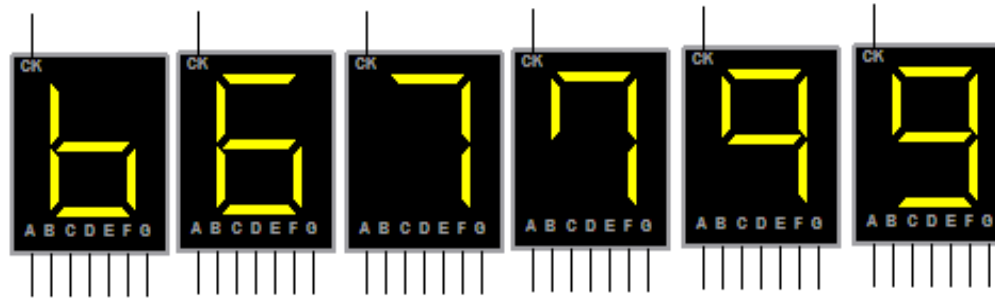
Yedi segment display



Yedi segment display için kod çözücü devre

Yedi Segment Display İçin Kod Çözücü Devre Tasarımı

Kod çözücünün tasarımını yapmak için, doğruluk tablosunu oluşturabilir ya da Karnaugh haritasına çıkışları direkt olarak taşıyabiliriz. 6,7 ve 9'un iki türlü gösteriminin olabileceğini düşünerek, çıkışları en sade haline getirebilmek için, uygun olan gösterim, tasarım aşamasında seçilecektir.

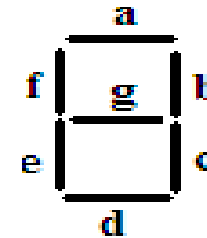


AB \ CD	00	01	11	10
00	1		x	1
01		1	x	1
11	1	1	x	x
10	1	x	x	x

$$a = A + C + BD + B'D'$$

AB \ CD	00	01	11	10
00	1	1	x	1
01	1		x	1
11	1	1	x	x
10	1		x	x

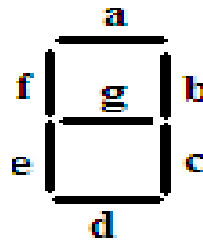
$$b = B' + CD + C'D'$$



Yedi Segment Display İçin Kod Çözücü Devre Tasarımı

c					d				
AB \ CD	00	01	11	10	AB \ CD	00	01	11	10
00	1	1	x	1	00	1		x	1
01	1	1	x	1	01		1	x	x
11	1	1	x	x	11	1		x	x
10		1	x	x	10	1	1	x	x

$c = B + C' + D$
 $d = B'D' + CD' + B'C + BC'D$



0 1 2 3 4 5 6 7 8 9

Yedi Segment Display İçin Kod Çözücü Devre Tasarımı

		e			
		AB			
CD \		00	01	11	10
00		1		x	1
01				x	
11				x	x
10		1	1	x	x

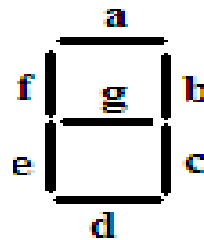
$$e = B'D' + CD'$$

		f			
		AB			
CD \		00	01	11	10
00		1	1	x	1
01			1	x	1
11			x	x	x
10			1	x	x

$$f = A + B + C'D'$$

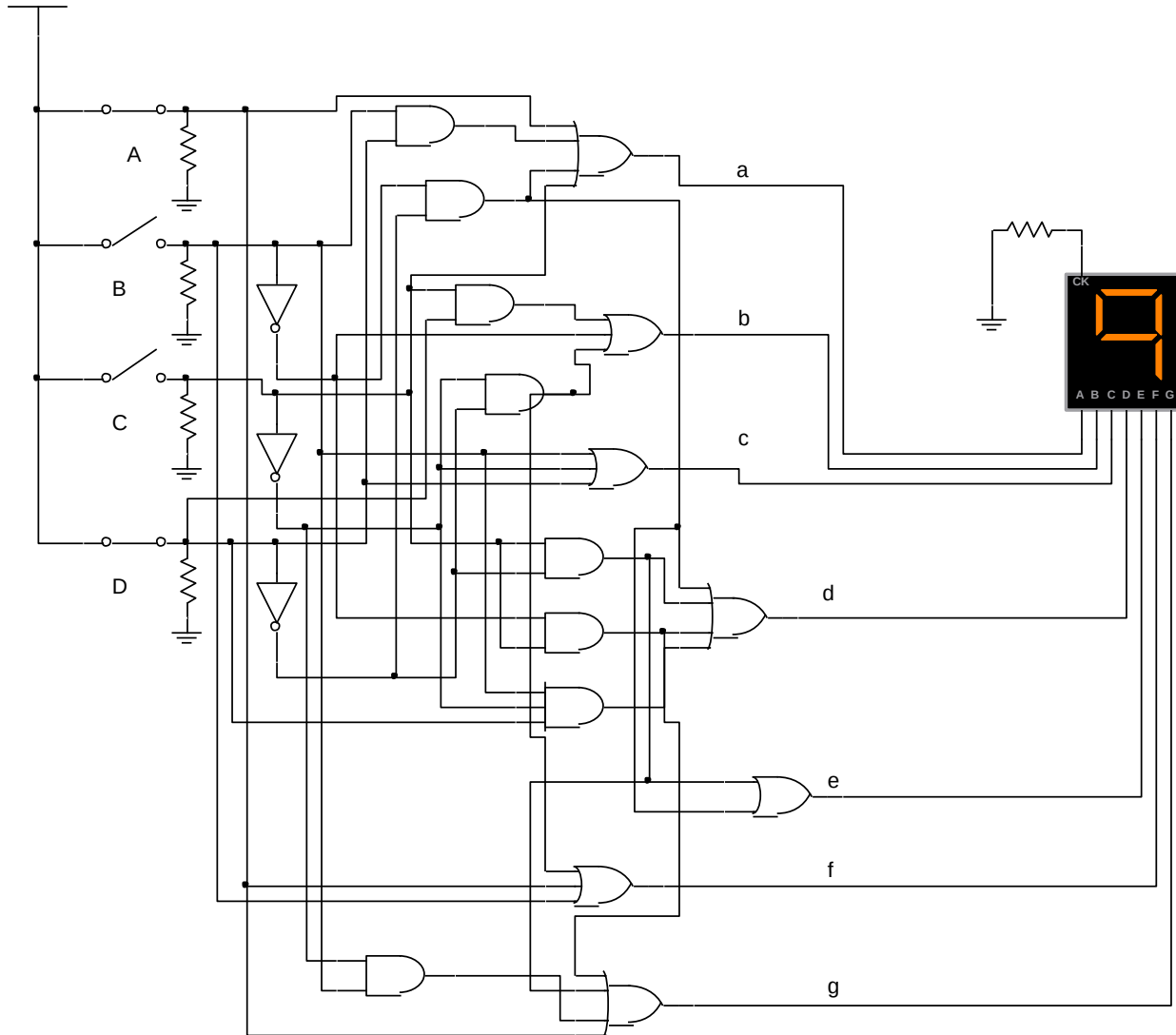
		g			
		AB			
CD \		00	01	11	10
00			1	x	1
01			1	x	1
11		1		x	x
10		1	1	x	x

$$g = A + B'C + CD' + BC'$$



0 1 2 3 4 5 6 7 8 9

Yedi Segment Display İçin Kod Çözücü Devre Tasarımı



Örnekler

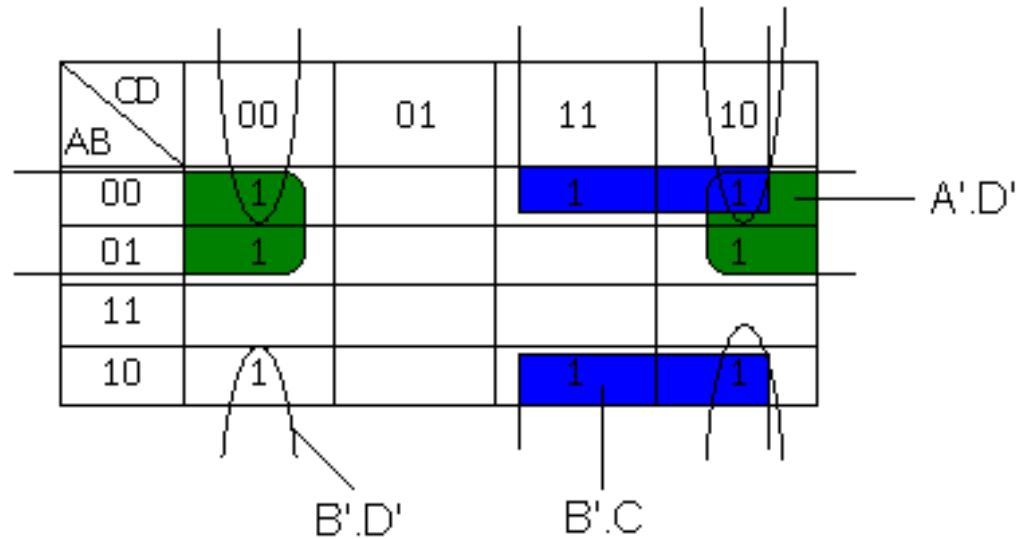
İÇERİK:

- ❖ Karnaugh Haritaları
- ❖ Önemsiz (Don't Care) Durumlar
- ❖ Tablo Yöntemiyle İndirgeme
- ❖ Devreleri Sadece NAND ya da NOR Kapılarıyla Gerçekleme
- ❖ Decoder'ler
- ❖ Multiplexer'lar

Örnek:

$F(A,B,C,D) = \sum (0,2,3,4,6,8,10,11)$ fonksiyonunu Karnaugh haritası yardımıyla indirgeyelim.

CD \ AB	00	01	11	10
00	1		1	1
01	1			1
11				
10	1		1	1



$$F(A,B,C,D) = A'D' + B'D' + B'C$$

Örnek:

Aşağıda verilen Karnaugh haritası için 2 farklı indirgeme yapılabilmektedir. Bu indirgemeleri bulalım.

CDE \ AB	000	001	011	010	110	111	101	100
00								
01	1		1	1	1	1		1
11		1	1			1	1	
10	1	1	1			1	1	

Minimum sayıda grup, maksimum sayıda minterm kuralına göre yandaki gruplar oluşturulabilir.

CDE \ AB	000	001	011	010	110	111	101	100
00								
01	1		1	1	1	1		1
11		1	1			1	1	
10	1	1	1			1	1	

CDE \ AB	000	001	011	010	110	111	101	100
00								
01	1		1	1	1	1		1
11		1	1			1	1	
10	1	1	1			1	1	

$AB'C'D'$ $A'BD$ AE $A'BE'$

CDE \ AB	000	001	011	010	110	111	101	100
00								
01	1		1	1	1	1		1
11		1	1			1	1	
10	1	1	1			1	1	

BDE

Örnek:

Bir alarm sistemi ile ilgili olarak kombinasyonel devre tasarımı yapılması isteniyor. Sistemin 3 girişi vardır. Bunlar sensörlerden gelen kapı kilitli (C), kapı açık (B) ve pencere açık (A) sinyalleridir. Kapı kilitli olmadığı müddetçe pencere ve kapı açılabilir. Alarm sisteminin pencere veya kapı açıksa alarm sinyali (S) üretmesi istenmektedir.

C: Kapı kilitli ise 1 olmaktadır.

A: Pencere açıksa 1 olmaktadır.

B: Kapı açıksa 1 olmaktadır.

Kapı kilitliyse pencere veya kapı açılmayacağı kısıtı olduğundan önemsiz durumlar vardır.

AB \ C	00	01	11	10
0	0	1	1	1
1	0	x	x	x

AB \ C	00	01	11	10
0	0	1	1	1
1	0	x	x	x

$$S = A + B$$

Örnek:

$$f(a,b,c,d) = \sum(1,4,6,7,8,9,10,11,15)$$

f fonksiyonunun Quine-McCluskey yöntemi (tablo yöntemi) kullanılarak, asal ve asıl asal bileşenlerinin bulunması isteniyor.

Asal bileşenlerin bulunması;

	a	b	c	d			a	b	c	d			a	b	c	d
1	0	0	0	1	√	1-9	-	0	0	1		8-9-10-11	1	0	-	-
4	0	1	0	0	√	4-6	0	1	-	0						
8	1	0	0	0	√	8-9	1	0	0	-	√					
6	0	1	1	0	√	8-10	1	0	-	0	√					
9	1	0	0	1	√	6-7	0	1	1	-						
10	1	0	1	0	√	9-11	1	0	-	1	√					
7	0	1	1	1	√	10-11	1	0	1	-	√					
11	1	0	1	1	√	7-15	-	1	1	1						
15	1	1	1	1	√	11-15	1	-	1	1						

Örnek: (devamı)

Asıl asal bileşenlerin bulunması;

Mintermler/ Asal içerikler	1	9	4	6	7	15	11	8	10	
	✓	✓	✓	✓			✓	✓	✓	
b'c'd m(1,9)	x	x								✓
a'bd' m(4,6)			x	x						✓
a'bc m(6,7)				x	x					
bcd m(7,15)					x	x				
acd m(11,15)						x	x			
ab' m(8,9,10,11)		x					x	x	x	✓

$$f(a,b,c,d) = b'c'd + a'bd' + ab' + bcd$$

Örnek:

$f(a,b,c) = a'c + bc + a'b$ fonksiyonunu sadece NOR kapıları kullanarak gerçekleyelim.

f fonksiyonu yapı itibarıyla çarpımlar toplamı biçimindedir. NAND kapıları kullanılarak gerçekleştirilmesi istenseydi ifadenin tamamının 2 kere değilini almak yeterli olacaktı. Fakat NOR kapıları ile gerçekleştirilmesi istendiği için her bir çarpım teriminin 2 kere değilini almak gerekir.

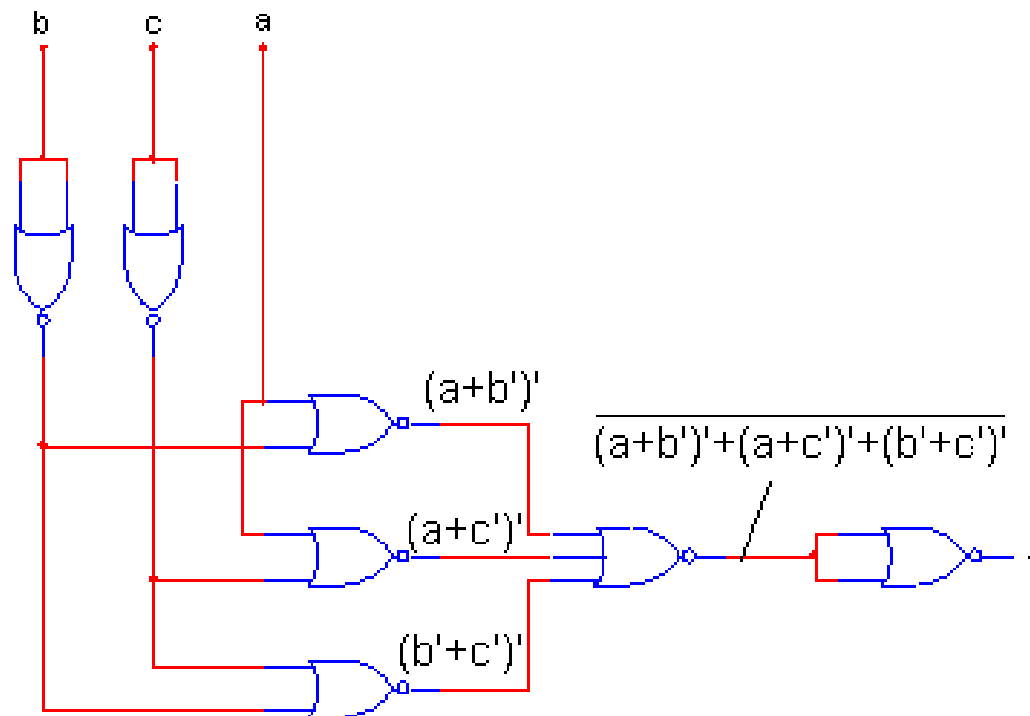
$$f(a,b,c) = [(a'c)']' + [(bc)']' + [(a'b)']' = (a+c')' + (b'+c')' + (a+b')'$$

f fonksiyonunun içerdiği her terim NOR formundadır. Ancak f fonksiyonunu da NOR formuna getirebilmek için ifadenin tamamının 2 kere değilinin alınması gerekir.

$$f(a,b,c) = \overline{\overline{(a + c')' + (b' + c')' + (a + b')'}}$$

Örnek: (devamı)

$$f(a,b,c) = \overline{\overline{(a + c')' + (b' + c')' + (a + b')'}}$$

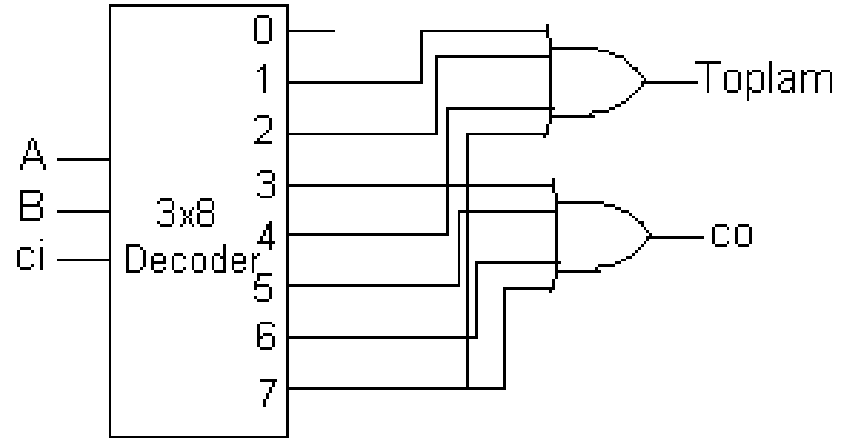


Örnek:

Tam toplayıcıyı, decoder ve VEYA kapıları kullanarak gerçekleyelim.

Tam toplayıcının doğruluk tablosu;

Girişler			Çıkışlar	
A	B	ci	Toplam	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$\text{Toplam} = \sum(1,2,4,7)$$

$$\text{co} = \sum(3,5,6,7)$$

Örnek:

$f(A,B,C,D)$ fonksiyonu aşağıdaki Karnaugh haritasında verilmiştir. Bu fonksiyonu 4×1 MUX ve diğer kapı elemanlarını kullanarak gerçekleyelim. (Not: MUX'un seçim uçlarını $S_1 S_0 = A B$ alınız.)

CD \ AB	00	01	11	10
00	1	1		1
01	1			
11				
10	1		1	1

CD \ AB	00	01	11	10
00	1	1		1
01	1			
11				
10	1		1	1

$A'C'D'$ (red line)
 $A'B'C'$ (blue line)
 $B'D'$ (red line)
 $AB'C$ (green line)

$$\begin{aligned}
 f(A,B,C,D) &= A'B'C' + AB'C + A'C'D' + B'D' \\
 &= A'B'C' + AB'C + A'(B+B')C'D' + (A+A')B'D' \\
 &= \underline{A'B'C'} + \underline{AB'C} + A'BC'D' + \underline{A'B'C'D'} + \underline{AB'D'} + \underline{A'B'D'} \\
 &= A'B'(C' + \underline{C'D'} + \underline{D'}) + AB'(C + D') + A'BC'D' \\
 &= A'B'(C' + D') + AB'(C + D') + A'BC'D'
 \end{aligned}$$

4×1 MUX'un tanım bağıntısı;

$A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$ olduğundan

$$I_0 = (C' + D') = (C.D)'$$

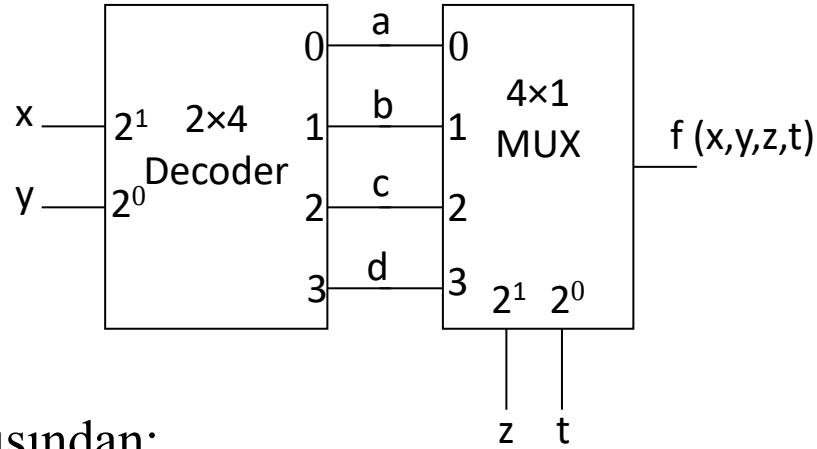
$$I_1 = C'D' = (C + D)'$$

$$I_2 = C + D'$$

$$I_3 = 0$$

Örnek:

Aşağıdaki devrenin çıkışı olan f fonksiyonunu mintermler cinsinden bulalım.



MUX'un tanım bağıntısından;

$$f = z't'.a + z't.b + zt'.c + zt.d$$

Decoder'in tanım bağıntısından;

$$a = x'y' \quad b = x'y \quad c = xy' \quad d = xy$$

Bu ifadeler MUX'un tanım bağıntısında yerine yazılırsa;

$$f = z't'.x'y' + z't.x'y + zt'.xy' + zt.xy$$

f fonksiyonu düzenlenirse;

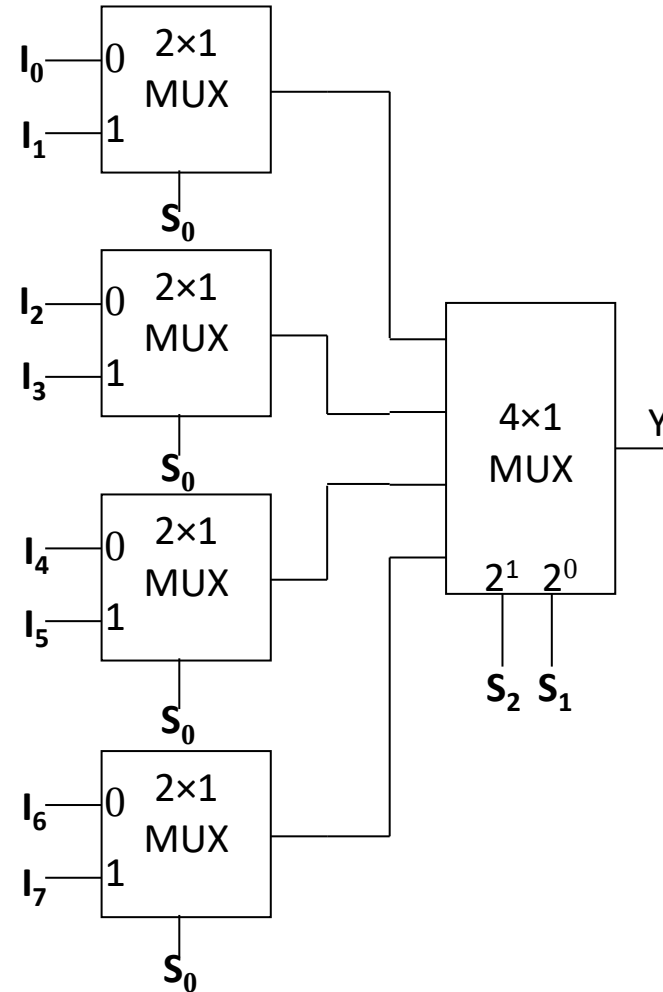
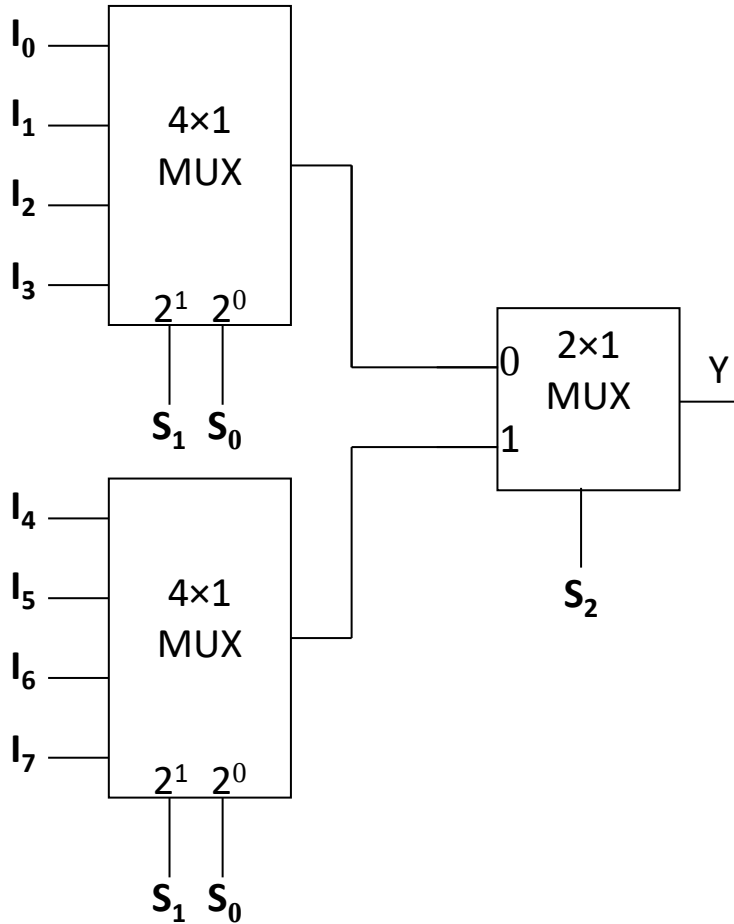
$$f(x,y,z,t) = x'y'z't' + x'yz't + xy'zt' + xyzt = \sum(0,5,10,15) \text{ olur.}$$

Örnek:

Sadece 4×1 ve 2×1 MUX'lar kullanarak 8×1 MUX elde edelim.

(Not: Seçim uçlarını $S_2 S_1 S_0$ alınız. MUX'lar Enable ucuna sahip değiller.)

2 çözüm vardır:



Örnek:

$$f(a,b,c,d) = \sum (1,2,4,7,8,11,13,14)$$

$f(a,b,c,d)$ fonksiyonunu 8×1 MUX ile sistematik olarak gerçekleyelim.

(Not: Seçim uçlarını a b c seçiniz.)

d \ abc	000	001	010	011	100	101	110	111
0	m_0	m_2	m_4	m_6	m_8	m_{10}	m_{12}	m_{14}
1	m_1	m_3	m_5	m_7	m_9	m_{11}	m_{13}	m_{15}
Girişler	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
Bağlantılar	d	d'	d'	d	d'	d	d	d'